Aprendizaje Computacional

Dr. José Francisco Martínez Trinidad

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Introducción al Curso **2**

2. Aprendizaje **4**

3. Clasificadores lineales y perceptrón **7**

4. Convergencia y generalización (perceptrón) **10**

5. Máquinas de vectores de soporte (SVM) **14**

6. SVM revisitada: forma dual y kernels **18**

7. Preparación de datos y redes neuronales **24**

8. Neurona artificial y separabilidad lineal **27**

9. Descenso de gradiente y backpropagation **32**

10. Redes convolucionales **39**

11. Redes residuales y autoencoders **44**

12. Clasificadores basados en distancia **49**

13. Clasificador Naive Bayes **56**

14. Medidas de desempeño **61**

15. Árboles de decisión **68**

16. Selección de atributos **80**

17. Clasificadores basados en reglas **94**

1. Introducción al Curso

25 de agosto de 2026

Resumen rápido. Sesión inicial: contenido y dinámica del curso, temario, criterios de evaluación y el calendario del proyecto. La segunda mitad fue la introducción propiamente dicha: qué es aprender, por qué generalizar no es memorizar, de qué disciplinas se alimenta el área y la formalización de la función f que se quiere aprender contra la hipótesis h.

Info general

Contacto:

- Dr. José Francisco Martínez Trinidad
- Correo: `fmartine@inaoep.mx`

Temario:

1. Introducción y clasificadores lineales
2. Perceptrón
3. SVM
4. Clasificadores basados en distancia
5. ANNs
6. Evaluación, validación y overfitting
7. Métodos bayesianos
8. Árboles de decisión
9. Reglas de clasificación
10. Reglas de asociación
11. Selección de atributos
12. Clustering (K-means, gaussian mixture, jerárquicos)

Idea clave

No se trata de usar las herramientas como una caja negra. El objetivo es, como investigadores, saber cómo usarlas para generar nuevo conocimiento, y por eso hay que conocerlas a detalle: para poder innovar y proponer alternativas.

Evaluación

80 % proyecto:

- Equipo de dos personas
- Seleccionar un artículo reciente
- Proponer una solución alternativa
- Tres presentaciones

- Reporte final

20 % examen oral, a la entrega del proyecto, sobre un tema aleatorio del temario.

Calendario

- **25 de agosto.** Buscar un artículo publicado en 2026 que sea de interés, en una revista JCR.
- **3 de septiembre.** Entregar por correo una lista que indique cuál artículo seleccionó cada estudiante o equipo.
- **29 de septiembre o 1 de octubre.** Primera presentación:
  - Descripción del problema abordado en el artículo
  - Breve descripción del trabajo relacionado que menciona el artículo
  - Breve descripción de la solución propuesta en el artículo
  - Descripción de los experimentos que soportan dicha solución, incluyendo tablas y gráficas tomadas del artículo
  - Conclusiones a las que llegan los autores
  - Análisis de debilidades y posibles modificaciones para mejorar la propuesta de los autores
- **27 o 29 de octubre.** Segunda presentación: resumen del problema abordado.
- **24 o 26 de noviembre.** Presentación final: resumen del problema abordado y de la solución final.
- **30 de noviembre.** Entrega del reporte final. Se entrega en forma de artículo, con formato de https://www.springer.com/.

El reporte debe tener más o menos 12 páginas e incluir título, resumen, introducción, trabajo relacionado, solución propuesta, experimentos, conclusiones y trabajo futuro, y referencias.

Revistas JCR

- Expert Systems with Applications
- Knowledge-Based Systems
- Journal of Machine Learning Research

Buscador: https://www.webofscience.com/wos/woscc/basic-research

Recursos

- Weka 3
- scikit-learn

Ojito...

Hay que tratar de contactar al autor del artículo para que nos dé sus recursos y poder compararnos.

2. Aprendizaje

Que es el aprendizaje?

Es una disciplina dentro de las ciencias de la computación que surge como motivación de hacer que las computadoras realicen tareas complejas como las que hacen los humanos.

Aprender significa adquirir conocimiento, entendimiento o una habilidad de manera empírica o por estudio. La pregunta que queda abierta es cómo determinamos el éxito o el fracaso de la tarea.

Se busca que se logre un proceso de aprendizaje para que el sistema sea capaz de generalizar.

Idea clave

Generalizar no es memorizar. Aprender permite responder a problemas nuevos con el conocimiento adquirido.

Al inicio hubo varias tendencias. Una de ellas fue ver cómo aprenden los animales y de ahí trasladarlo a la máquina, con la premisa de que quizá el aprendizaje en un animal es diferente al de un ser humano, pero no resultó cierto.

Ojito...

No hay una demostración universal de lo que es el aprendizaje. Hay muchas definiciones y muchas escuelas.

Tipos de aprendizaje

- Supervisado
- No supervisado
- Por refuerzo

Tareas relacionadas con el aprendizaje

- Reconocimiento
- Diagnóstico
- Planeación
- Control de robots
- Predicción

Aprender puede ser percibir cambios en el comportamiento gracias a la experiencia. En nuestro contexto serían mejoras en actividades ya conocidas, y aprender tareas nuevas.

Hacer que las máquinas aprendan es muy útil, ya que nos simplifica la vida en muchos contextos: nos ayuda a hacer tareas complejas más rápido y a analizar datos masivos de una manera rápida.

Disciplinas relacionadas

- Estadística
- Lógica

- Matemáticas discretas
- Y muchas más

Todas las teorías que hay sobre cómo funciona el cerebro han ayudado bastante, ya que se trata de emular lo que se cree que ocurre en el cerebro humano cuando ocurre el aprendizaje, y trasladarlo a la computadora.

Otra disciplina que se vincula mucho con el ML es el control, la teoría de control. Igual los modelos psicológicos y los modelos evolutivos, entre muchos más.

Qué es lo que se aprende?

Diferentes estructuras computacionales: funciones, programas lógicos, máquinas de estado finito, gramáticas y sistemas para la resolución de problemas.

La idea es que tengo mi problema y tengo algunos ejemplos de él. Le doy a la computadora los ejemplos y la computadora me da como resultado el programa que me ayuda a generalizar cómo son esos ejemplos, de tal manera que cuando llegue un dato nuevo pueda clasificarlo, identificarlo y reconocerlo correctamente.

Más formalmente, se tiene una función f, que es la que queremos aprender, y otra función h, la hipótesis. Ambas reciben un vector de valores con N componentes. Queremos encontrar la función h cuya salida sea $h(X)$, y que sea capaz de reconocer un dato nuevo.

Supervisado y no supervisado

Hay dos tipos de aprendizaje, bueno tres, pero estos son los más comunes.

- **Supervisado.** Los datos de entrenamiento están etiquetados. A la función h, que es la que voy a aprender, le voy a dar un dato x, y esa función tiene que ser capaz de reconocer la nueva entrada.
- **No supervisado.** Los datos de entrenamiento no están etiquetados, y el algoritmo encuentra la relación entre los datos.

Vector de entrada

Al vector de entrada se le llama patrón, feature vector, instancia o ejemplo. Sus componentes son las variables o características, y esas variables pueden tomar valores reales, discretos o categóricos.

La salida puede ser un valor real, y entonces se trata de un predictor, o discreto, y entonces se trata de un clasificador.

Modos de entrenamiento

Hay varios modos de entrenamiento, dependiendo de los datos:

- Por batch
- De manera incremental
- En línea

Evaluación

Cuando ya tengo mi función h, ¿cómo la evalúo? ¿Cómo sé que está bien? Para eso se necesita un conjunto de prueba, que nos ayuda a ver qué tan bien la función h realiza su tarea, midiendo el error de h sobre ese conjunto. El conjunto de prueba es una partición del dataset original.

3. Clasificadores lineales y perceptrón

27 de agosto de 2026

Resumen rápido. Retomamos la idea de aprender una función que generalice y la aterrizamos en un ejemplo de control de acceso por rostro. De ahí salieron los clasificadores lineales, las funciones de pérdida y el perceptrón como algoritmo para ajustar los parámetros.

Recap

ML se trata de encontrar una función que fitee unos datos de entrenamiento para que sea capaz de generalizar. El proceso es:

1. Suponemos una función f que modela lo que queremos
2. Construimos hipótesis
3. Queremos construir la función h que emule a f
4. Entrenamos a h para que sea capaz de generalizar
5. Testeamos

Ejemplo: control de acceso por rostro

Supongamos que queremos construir un sistema que permita pasar a personas cuyos rostros sean conocidos para entrar al instituto. Tenemos los datos de las caras, y una imagen es un arreglo de bits.

Tenemos que construir una función

$$f : \mathbb{R}^d \to \{-1, 1\}$$

Es discreta porque es una tarea de clasificación. El clasificador tendrá un conjunto de vectores de entrenamiento con etiquetas, y estos vectores nos ayudarán a entrenar la función. Entonces queremos construir la función solo con los datos de entrenamiento, o sea con nuestra experiencia empírica.

Una manera sería ver un píxel de la imagen, de tal forma que al estar presente acepta a esa persona y la rechaza en otro caso, pero esto tiene varias limitaciones. ¿La función es útil para otras imágenes que no estén en el dataset? ¿Es capaz de generalizar?

Idea clave

Lo que se trata es de encontrar un modelo, una función que nos resuelva esa tarea de aprendizaje. No queremos funciones que fiteen perfectamente al conjunto de datos, ni súper generales, queremos algo en medio.

Selección de modelo

Y cómo se encuentran clasificadores que generalicen bien. La clave es *restringir* el conjunto de funciones binarias que uno se permite considerar. Se busca una clase de funciones tal que, si una función de la clase funciona bien en el entrenamiento, es probable que también funcione bien en las imágenes que no se han visto.

Idea clave

El problema de selección de modelo. La clase correcta no puede ser demasiado grande, porque entonces contiene demasiadas funciones claramente distintas y una de ellas va a memorizar el entrenamiento por casualidad, como la del píxel único. Pero tampoco demasiado chica, porque entonces se corre el riesgo de no encontrar ninguna función que funcione bien ni siquiera en el entrenamiento. Encontrar esa clase es uno de los problemas centrales del aprendizaje automático.

Fijar la clase de funciones en los clasificadores lineales, que es lo que sigue, es exactamente tomar esa decisión.

Clasificadores lineales

Lo que hacemos es definir una función lineal:

$$f(x;\theta) = \text{sign}(\theta_1 x_1 + \cdots + \theta_d x_d) = \text{sign}(\theta^\mathsf{T} x)$$

Se toma el signo para nuestro problema porque solo estamos considerando el caso binario: acepta o rechaza la entrada al edificio.

El modelo encontrará una línea, un plano que permita separar a las personas que pueden o no entrar al edificio. Ese plano es justo el lugar donde $\theta^\mathsf{T} x = 0$, y θ es el vector normal a él, así que ajustar θ es girar y mover el plano.

El chiste es encontrar la función a partir de las entradas que tengo en el entrenamiento. Cuando no es posible construir la función, queremos entonces descubrir otra que cometa el menor error posible.

Ojito...

En nuestro problema el clasificador lineal no guarda info de los píxeles vecinos, solo evaluaría el píxel individual.

Funciones de pérdida

Mencionamos que queremos encontrar el error. Esto se mide con la diferencia del dato esperado con el dato obtenido, y hay diferentes medidas de error.

A las funciones de error se les llama **loss**, función de error o función de pérdida, y hacen lo que mencionábamos: dicen qué tanta diferencia hay entre el dato observado y el esperado. Son esas funciones las que se tratan de optimizar.

El perceptrón

Queremos encontrar un clasificador lineal que minimice el error, o sea encontrar θ que minimice el error de entrenamiento:

$$E(\theta) = \frac{1}{n}\sum_{i=1}^{n}\Big[1 - \delta(y^{(i)},\, f(x^{(i)};\theta))\Big]$$

Donde δ vale 1 si sus dos argumentos coinciden y 0 si no, así que $1-\delta$ vale 1 exactamente cuando el ejemplo quedó mal clasificado. Dicho de otra forma, $E(\theta)$ es la fracción de ejemplos de entrenamiento en los que el modelo se equivoca.

¿Pero cómo construir ahora esa función lineal? ¿Cómo ajustar los parámetros? ¿Hay alguna manera razonable de setear los parámetros de θ?

Una opción es ajustar incrementalmente cada parámetro para que veamos que está bien. Un algoritmo simple de este tipo es el **perceptron update rule**:

$$\text{si } y^{(i)}(\theta^{\mathsf{T}} x^{(i)}) \leq 0 \quad \text{entonces} \quad \theta' = \theta + y^{(i)} x^{(i)}$$

En otras palabras, los parámetros son actualizados solo si hay error, y estos updates tienden a corregir errores. En nuestro caso, la clasificación de caras, cuando el modelo hace un error el signo no coincide con el de la etiqueta.

Supongamos que hacemos un error, y que los parámetros actualizados están dados por θ'. Si consideramos clasificar la misma imagen después del update:

$$\begin{aligned} y^{(i)}(\theta'^{\mathsf{T}} x^{(i)}) &= y^{(i)}((\theta + y^{(i)} x^{(i)})^{\mathsf{T}} x^{(i)}) \\ &= y^{(i)}(\theta^{\mathsf{T}} x^{(i)}) + (y^{(i)})^2 \left\| x^{(i)} \right\|^2 \\ &= y^{(i)}(\theta^{\mathsf{T}} x^{(i)}) + \left\| x^{(i)} \right\|^2 \end{aligned}$$

Como $y^{(i)} \in \{-1, 1\}$ su cuadrado siempre es 1, y $\left\| x^{(i)} \right\|^2 > 0$. O sea que el update le suma algo positivo, empujando ese ejemplo hacia el lado correcto del plano. Eso es lo que quiere decir que los updates tienden a corregir errores.

Ojito...

Esto da origen al reconocedor del perceptrón, y este reconocedor es útil para problemas lineales: solo puede separar con una línea o un plano, pero jamás podrá separar datos que no sean linealmente separables.

4. Convergencia y generalización (perceptrón)

1 de septiembre de 2026

Resumen rápido. En esta sesion vimos la convergencia del perceptron, y la definicion formal, q era el margen geometrico y cerramos con la intro a SVM

Punto de partida y supuestos

Seguimos con clasificadores lineales que pasan por el origen:

$$f(x;\theta) = \text{sign}(\theta^\mathsf{T} x),$$

donde θ denota los parámetros que tenemos que estimar en base a los ejemplos de entrenamiento, con ejemplos x y etiquetas y

Sea k el número de parámetros actualizados y $\theta^{(k)}$ el vector de parámetros después de k actualizaciones. Cada vez que el modelo se equivoca hace un update con la regla del perceptrón, y ese update es un k más

Para poder demostrar algo hacen falta dos supuestos:

1. **Las normas están acotadas**: existe R tal que $\left\|x^{(t)}\right\| \leq R$ para todo ejemplo.
2. **El clasificador lineal existe**: hay un θ^*, el θ ideal, que separa los datos con margen. Formalmente,

$$y^{(t)}(\theta^*)^\mathsf{T} x^{(t)} \geq \gamma > 0 \qquad \text{para todo } t,$$

   donde γ es el margen del clasificador, que es lo que nos dice que lo hace bien

Ojito...

En el ejemplo de las imágenes de las caras reconocidas para entrar al edificio, el segundo supuesto es justamente suponer que ese clasificador existe. Si los datos no son linealmente separables, no hay θ^* y nada de lo que sigue aplica

Convergencia en un número finito de updates

La convergencia se demuestra combinando dos resultados. El primero dice que el producto interno con el ideal crece linealmente, y el segundo que la norma cuadrada crece a lo más linealmente. Tomamos $\theta^{(0)} = 0$.

Parte 1: el producto interno crece linealmente

Supongamos que el update k se hizo sobre el ejemplo $(x^{(t)}, y^{(t)})$, o sea $\theta^{(k)} = \theta^{(k-1)} + y^{(t)}x^{(t)}$. Entonces

$$\begin{aligned}(\theta^*)^\mathsf{T}\theta^{(k)} &= (\theta^*)^\mathsf{T}\theta^{(k-1)} + y^{(t)}(\theta^*)^\mathsf{T}x^{(t)} \\ &\geq (\theta^*)^\mathsf{T}\theta^{(k-1)} + \gamma,\end{aligned}$$

donde la desigualdad es el supuesto del margen. Cada update le suma al menos γ, así que aplicándolo k veces desde $\theta^{(0)} = 0$:

$$(\theta^*)^\mathsf{T}\theta^{(k)} \geq k\,\gamma.$$

Parte 2: la norma cuadrada crece a lo más linealmente

Con el mismo update,

$$\begin{aligned}\left\|\theta^{(k)}\right\|^2 &= \left\|\theta^{(k-1)} + y^{(t)}x^{(t)}\right\|^2 \\ &= \left\|\theta^{(k-1)}\right\|^2 + 2\,y^{(t)}\left(\theta^{(k-1)}\right)^{\mathsf{T}}x^{(t)} + \left(y^{(t)}\right)^2\left\|x^{(t)}\right\|^2 \\ &\leq \left\|\theta^{(k-1)}\right\|^2 + R^2.\end{aligned}$$

El término de en medio se va porque el update ocurre precisamente cuando el ejemplo estaba mal clasificado, o sea cuando $y^{(t)}\left(\theta^{(k-1)}\right)^{\mathsf{T}}x^{(t)} \leq 0$. Y el último se acota porque $\left(y^{(t)}\right)^2 = 1$ y $\left\|x^{(t)}\right\| \leq R$. Repitiendo k veces:

$$\left\|\theta^{(k)}\right\|^2 \leq k\,R^2$$

Combinación: qué pasa con el ángulo

Si combinamos esos dos resultados podemos ver el coseno del ángulo entre el vector de la actualización k y el ideal:

$$\cos\left(\theta^*, \theta^{(k)}\right) = \frac{\left(\theta^*\right)^{\mathsf{T}}\theta^{(k)}}{\|\theta^*\|\,\left\|\theta^{(k)}\right\|} \geq \frac{k\,\gamma}{\|\theta^*\|\,\sqrt{k}\,R} = \sqrt{k}\,\frac{\gamma}{\|\theta^*\|\,R}.$$

El numerador se acotó por abajo con la parte 1 y el denominador por arriba con la parte 2. Como el coseno está acotado por 1:

$$\sqrt{k}\,\frac{\gamma}{\|\theta^*\|\,R} \leq 1 \qquad \Longrightarrow \qquad k \leq \frac{R^2\,\|\theta^*\|^2}{\gamma^2}.$$

Idea clave

Cada update acerca el coseno a 1 en una cantidad finita, o sea acerca el vector actual al ideal. Como el coseno no puede pasar de 1, solo se puede hacer un número finito de updates. Ahí está la convergencia

Ojito...

Está acotado, pero la cota depende del valor de los pesos: puede ser muy grande o muy pequeña, y de antemano no sabemos cuál.

Margen geométrico

El cociente $\|\theta^*\|^2/\gamma^2$ nos dice qué tan difícil es el problema, o sea cuántos updates voy a tener que hacer. Su inverso es la distancia más chica que hay entre una instancia del conjunto de entrenamiento y mi región de decisión

Si esa distancia es grande hay bastante separación entre la superficie que divide las clases y las instancias, y es más fácil clasificar. Si es chica, es más difícil

Cómo se calcula

Se mide la distancia de la superficie de decisión a uno de los datos de entrenamiento. Sea $\tilde{x}$ el punto de la frontera más cercano a $x^{(t)}$, y ε esa distancia. Moverse de $x^{(t)}$ a la frontera es avanzar ε en la dirección normal, que es la de θ^*:

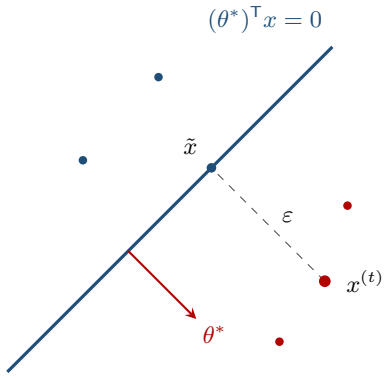
$$\tilde{x} = x^{(t)} - y^{(t)}\,\varepsilon\,\frac{\theta^*}{\|\theta^*\|}.$$

Evaluando el clasificador ideal en ese punto:

$$y^{(t)}(\theta^*)^\mathsf{T}\tilde{x} = \underbrace{y^{(t)}(\theta^*)^\mathsf{T}x^{(t)}}_{=\gamma} - \varepsilon\,\|\theta^*\| = \gamma - \varepsilon\,\|\theta^*\| = 0,$$

porque $\tilde{x}$ está sobre la frontera y ahí el clasificador vale cero. Despejando queda el **margen geométrico**:

$$\varepsilon = \frac{\gamma}{\|\theta^*\|}.$$



Por lo tanto el k, el número de veces que tengo que actualizar, se puede expresar en términos del margen geométrico:

$$k \leq \frac{R^2}{\varepsilon^2}.$$

Idea clave

Entre más ancho el pasillo que separa las clases, menos updates necesita el perceptrón. La dificultad del problema no está en cuántos datos hay ni en cuántas dimensiones, sino en qué tan apretado quedó ese pasillo

Generalización

Hasta ahorita asumimos que el conjunto de datos era linealmente separable, pero qué pasa si ahora no trabajo con mi conjunto de entrenamiento sino con datos que están fuera de él

Cuando llegue una instancia nueva, el clasificador lineal tendrá que ser capaz de clasificarla: se pone en el plano y se ve si la superficie separadora la separa correctamente

La garantía formal

El argumento se puede hacer preciso. Supongamos que *todas* las imágenes y etiquetas que podríamos llegar a encontrar, no solo las del entrenamiento, cumplen los mismos dos supuestos: $\|x_t\| < R$, y existe un θ^* finito con $y_t(\theta^*)^\top x_t \geq \gamma$ para toda t. O sea que existe un clasificador lineal que sirve para todo el problema, no solo para la muestra.

Ahora imaginemos que las instancias llegan una por una, y que por cada una hacemos a lo más un update si la clasificamos mal, y seguimos adelante sin volver a verla. ¿Cuántos errores vamos a cometer en esa secuencia arbitraria e indefinida?

Idea clave

El mismo número: $k \leq (R/\gamma_{\text{geom}})^2$. La cota no cambia. Una vez cometidos esos errores, todas las instancias nuevas se clasifican correctamente. Eso es una garantía de generalización, y no salió de un supuesto estadístico sobre la distribución de los datos, sino de la geometría del problema.

Ojito...

Con un pero: el algoritmo necesita *saber* cuándo se equivocó, porque la cota está expresada en número de updates, y los updates se hacen sobre errores. En un escenario real donde nadie te dice la etiqueta verdadera, no hay quién dispare el update.

Idea clave

Lo que queremos ahora es un clasificador lineal que nos dé el máximo margen geométrico. De todas las superficies que separan bien el entrenamiento, la que deja el pasillo más ancho es la que tiene más holgura para los datos nuevos. Ese clasificador son las máquinas de vectores de soporte

5. Máquinas de vectores de soporte (SVM)

3 de septiembre de 2026

Resumen rápido. En esta clase vimos como funcionan los SVM

El problema

Se plantea el problema de encontrar la superficie de separación entre los datos de una muestra, de tal manera que el margen geométrico que vimos sea el máximo posible

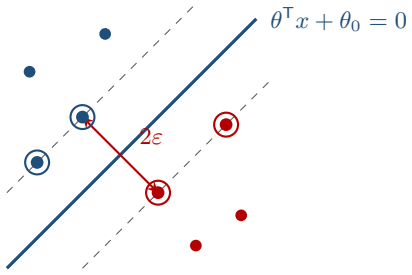


Figura 1: La superficie que deja el pasillo más ancho. Los puntos circulados, los que caen justo sobre el borde del margen, son los vectores de soporte.

Se plantea como un problema de optimización: lo que quiero optimizar son los parámetros de mi vector, de tal manera que exista un margen alrededor de mi superficie de separación que sea lo más grande posible.

Una manera de lograrlo sería encontrar una superficie que separe bien las instancias y luego tratar de que ese clasificador tenga el mayor margen posible. Se busca entonces maximizar el margen geométrico $\gamma / \|\theta\|$, cumpliendo las restricciones

$$y^{(t)}(\theta^\top x^{(t)}) \geq \gamma \qquad \text{para } t = 1, 2, \ldots, n.$$

Idea clave

Esa restricción dice dos cosas al mismo tiempo. El *signo* de $y^{(t)}\theta^\top x^{(t)}$ dice si la instancia quedó del lado correcto del plano, y su *magnitud* dice qué tan lejos quedó. Pedir que sea $\geq \gamma$ es pedir las dos: bien clasificada, y además a distancia de sobra, no pegada a la frontera.

Como problema de optimización

Maximizar el cociente que dijimos es lo mismo que minimizar su inverso al cuadrado:

$$\min_\theta \ \frac{1}{2}\frac{\|\theta\|^2}{\gamma^2} \qquad \text{sujeto a} \qquad y^{(t)}\theta^\top x^{(t)} \geq \gamma, \quad t = 1, \ldots, n.$$

Así escrito el problema depende de γ, y eso sobra. Si se escala θ y γ por la misma constante, ni las restricciones ni el margen geométrico cambian: la superficie es la misma. Se aprovecha esa libertad para fijar $\gamma = 1$, y el problema queda sin γ:

$$\min_\theta \ \frac{1}{2}\|\theta\|^2 \qquad \text{sujeto a} \qquad y^{(t)}\theta^\top x^{(t)} \geq 1, \quad t = 1, \ldots, n.$$

Con esa normalización el margen geométrico es $1/\|\theta\|$, así que minimizar la norma es literalmente ensanchar el pasillo.

El offset

Hasta ahora nuestra función está atada a pasar por el origen, y no queremos eso. Para lograrlo se suma un vector de offset θ_0:

$$f(x;\theta,\theta_0) = \text{sign}(\theta^\mathsf{T} x + \theta_0),$$

y el problema se vuelve

$$\min_{\theta,\theta_0} \frac{1}{2}\|\theta\|^2 \qquad \text{sujeto a} \qquad y^{(t)}(\theta^\mathsf{T} x^{(t)} + \theta_0) \geq 1.$$

Esto ayuda a que podamos encontrar una superficie con mayor margen geométrico, porque ya no obligamos al plano a pasar por el origen.

Vectores de soporte

Los elementos que caen justo a la distancia geométrica son los llamados **vectores de soporte**, y son elementos de la muestra. Son los que están circulados en la figura 1.

Propiedades del clasificador de margen máximo

A favor:

- **La solución es única** para cualquier conjunto de entrenamiento linealmente separable. El perceptrón, en cambio, se detiene en cualquier separador que funcione, y cuál depende del orden en que le llegaron los datos.
- **Es robusto a ejemplos ruidosos**, porque dejar la frontera lo más lejos posible de los datos da holgura para que una instancia corrida de lugar siga cayendo del lado correcto.
- **Depende solo de un subconjunto** de los ejemplos, los que caen exactamente sobre el margen. Los demás podrían estar en cualquier parte fuera del margen sin cambiar la solución.

En contra:

Ojito...

Es robusto a ejemplos ruidosos, pero **no a etiquetas ruidosas**. Un solo ejemplo mal etiquetado puede cambiar radicalmente el clasificador de margen máximo: si queda del lado equivocado, el pasillo tiene que encogerse hasta acomodarlo, y la frontera se va a otro lado. Esa fragilidad es la que motiva las variables de holgura.

Validación cruzada

Para evaluar este clasificador podemos hacer uso de la validación cruzada. Consiste en tomar el conjunto de entrenamiento, separar un cachito, construir el modelo con el resto y evaluar en ese cachito. El cachito se llama **fold**, y esto se repite k veces cambiando cuál fold se deja fuera.

Leave-one-out

Si dejamos solo un elemento fuera es el **leave-one-out cross validation**, pero eso se hace n veces, con n el número de elementos del training set. El error se mide así:

$$E_{\text{loo}} = \frac{1}{n}\sum_{i=1}^{n}\left[1 - \delta(y^{(i)},\, f(x^{(i)};\, \theta^{(-i)}))\right],$$

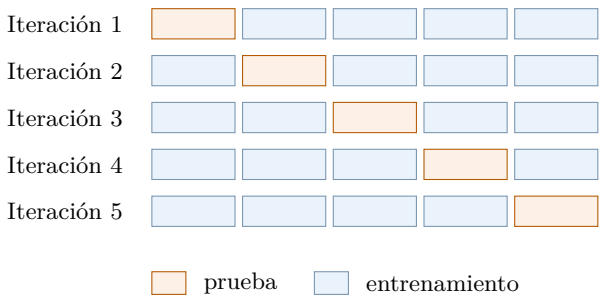


Figura 2: Validación cruzada con $k = 5$. En cada iteración un fold distinto hace de conjunto de prueba.

donde $\theta^{(-i)}$ son los parámetros entrenados con todo el conjunto *menos* el ejemplo i. O sea que cada ejemplo se predice con un modelo que nunca lo vio, y E_{loo} es la fracción de veces que ese modelo se equivoca.

Idea clave

El leave-one-out error está acotado por el número de vectores de soporte:

$$E_{\text{loo}} \leq \frac{\#\text{vectores de soporte}}{n}.$$

Cada vez que se quita una instancia que sea vector de soporte se puede cometer un error, porque quitarla mueve la superficie. Pero quitar una que no lo sea no cambia nada, así que ese ejemplo se sigue clasificando bien. Un clasificador con pocos vectores de soporte generaliza mejor.

Datos con ruido: variables de holgura

Uno de los problemas de este clasificador es que si las etiquetas del conjunto están mal, el clasificador va a estar incorrecto. También hay casos en los que el dato mismo trae error.

Para solucionar esto se introducen **variables de holgura**, dando una optimización relajada:

$$\min_{\theta,\theta_0,\xi} \frac{1}{2}\|\theta\|^2 + C\sum_{t=1}^{n}\xi_t$$

sujeta a

$$y^{(t)}\left(\theta^{\mathsf{T}}x^{(t)} + \theta_0\right) \geq 1 - \xi_t, \qquad \xi_t \geq 0, \qquad t = 1, \ldots, n.$$

Cada ξ_t mide cuánto se le perdona a la instancia t, y C es un parámetro de costo, o sea qué tanto le va a costar violar las restricciones.

Ojito...

Si el costo no es tan grande, le voy a dar mucho chance a que queden instancias dentro del margen geométrico. Y al revés: con C grande no se permiten violaciones, el modelo se aprieta contra el entrenamiento y vuelve a ser sensible al ruido.

Cuando los datos no son linealmente separables

Todo lo anterior es solo para instancias linealmente separables, pero qué pasa si las instancias no lo son. A alguien se le ocurrió llevar las instancias que no son linealmente separables a un espacio en el que sí lo sean.

Entonces, en lugar de trabajar con la instancia del vector original, se trabaja con su imagen $\varphi(x)$ en el nuevo espacio. Ahí se construye el clasificador lineal y ahí se obtiene la respuesta:

$$y^{(t)}\left(\theta^{\mathsf{T}}\varphi(x^{(t)}) + \theta_0\right) \geq 1 - \xi_t.$$

Idea clave

La superficie sigue siendo un plano, pero en el espacio nuevo. Al regresarla al espacio original se ve curva, y por eso el modelo puede separar datos que una recta no separaba.

6. SVM revisitada: forma dual y kernels

8 de septiembre de 2026

Resumen rápido. Retomamos el problema de mapear las instancias a un espacio donde sí sean separables, y vimos que el mapeo explícito es caro y normalmente ni siquiera lo conocemos. La salida es reescribir la SVM en su forma dual, donde los ejemplos aparecen solo dentro de productos internos, y ahí sustituir el producto por un kernel. Cerramos con cómo se extiende todo esto a más de dos clases.

El mapeo explícito y su costo

La idea de la clase pasada: si el conjunto no es linealmente separable en $\mathbb{R}^d$, se busca una función φ que lleve las instancias a un espacio de mayor dimensión donde sí lo sean,

$$\varphi : \mathbb{R}^d \to \mathbb{R}^{\mathcal{F}}, \qquad \mathcal{F} > d.$$

Ejemplo 6.1. De dos dimensiones a tres, elevando y multiplicando coordenadas:

$$[x_1, x_2] \;\longmapsto\; \varphi([x_1, x_2]) = [\, x_1^2,\; x_2^2,\; x_1 x_2 \,].$$

El producto interno en el espacio nuevo queda

$$\varphi(x)^\mathsf{T} \varphi(z) = x_1^2 z_1^2 + x_2^2 z_2^2 + x_1 x_2 z_1 z_2.$$

Con tres dimensiones de entrada el mismo esquema da nueve coordenadas, todos los productos $x_i x_j$.

Ojito...

Aquí está el problema. Encontrar la φ adecuada no es trivial, y aunque se encuentre, calcular productos internos en $\mathbb{R}^{\mathcal{F}}$ puede salir carísimo: la dimensión crece rápido. Peor aún, en general *no conocemos* φ explícitamente.

La salida no es buscar mejores mapeos, sino reformular la SVM de modo que φ nunca aparezca sola, solo dentro de productos internos. Eso es la forma dual.

La forma dual

El problema en el espacio de características es el de siempre:

$$\min_{\theta,\,\theta_0} \frac{\|\theta\|^2}{2} \qquad \text{sujeto a} \qquad y_t(\theta^\mathsf{T}\varphi(x_t) + \theta_0) \geq 1, \quad t = 1, \ldots, n.$$

Es convexo y con restricciones lineales, y ese tipo de problemas se puede pasar a su forma dual con multiplicadores de Lagrange. Se introduce un escalar no negativo α_t por cada restricción:

$$J(\theta, \theta_0; \alpha) = \frac{\|\theta\|^2}{2} - \sum_{t=1}^{n} \alpha_t \Big[\, y_t(\theta^\mathsf{T}\varphi(x_t) + \theta_0) - 1 \,\Big].$$

Idea clave

Cada α_t es el precio de violar la restricción t. Si la restricción se cumple con holgura, al maximizar sobre α le conviene poner $\alpha_t = 0$: esa restricción no está apretando y no cobra nada. Solo las que se cumplen con igualdad, o sea las que tocan el margen, terminan con $\alpha_t > 0$.

Bajo las condiciones de optimalidad de Lagrange se pueden intercambiar el mínimo y el máximo sin cambiar la respuesta:

$$\underbrace{\min_{\theta,\theta_0} \max_{\alpha \geq 0} J(\theta, \theta_0; \alpha)}_{\text{forma primal}} = \underbrace{\max_{\alpha \geq 0} \min_{\theta,\theta_0} J(\theta, \theta_0; \alpha)}_{\text{forma dual}}.$$

Resolviendo primero el mínimo interno, o sea derivando e igualando a cero:

$$\frac{\partial J}{\partial \theta_0} = -\sum_{t=1}^{n} \alpha_t y_t = 0,$$
$$\frac{\partial J}{\partial \theta} = \theta - \sum_{t=1}^{n} \alpha_t y_t \varphi(x_t) = 0 \implies \theta = \sum_{t=1}^{n} \alpha_t y_t \varphi(x_t).$$

Idea clave

Esa segunda ecuación dice algo fuerte: la solución θ siempre vive en el subespacio generado por los ejemplos de entrenamiento. No es un vector cualquiera de $\mathbb{R}^{\mathcal{F}}$, es una combinación lineal de las propias instancias, pesada por $\alpha_t y_t$.

Sustituyendo esa forma de θ en el objetivo se llega al dual:

$$\max_{\alpha} \sum_{t=1}^{n} \alpha_t - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j [\varphi(x_i)^\mathsf{T} \varphi(x_j)]$$

sujeto a

$$\alpha_t \geq 0, \qquad \sum_{t=1}^{n} \alpha_t y_t = 0.$$

Sigue siendo un problema de programación cuadrática, pero las restricciones son mucho más simples que las del primal.

La matriz de Gram

Fíjate en lo que quedó: la dimensión $\mathcal{F}$ del espacio de características **ya no aparece**. El problema está formulado enteramente sobre la matriz de productos internos, llamada matriz de Gram:

$$K = \begin{bmatrix} \varphi(x_1)^\mathsf{T} \varphi(x_1) & \cdots & \varphi(x_1)^\mathsf{T} \varphi(x_n) \\ \vdots & \ddots & \vdots \\ \varphi(x_n)^\mathsf{T} \varphi(x_1) & \cdots & \varphi(x_n)^\mathsf{T} \varphi(x_n) \end{bmatrix}.$$

Es de $n \times n$, o sea que su tamaño depende del número de ejemplos, no de la dimensión del espacio. Ahí está la puerta.

El truco del kernel

Idea clave

El truco del kernel. Si tienes un algoritmo donde los ejemplos aparecen *solo* dentro de productos internos, puedes reemplazar ese producto interno por otro, sin siquiera conocer φ.

Se define entonces una función kernel $K(x, x')$ tal que

$$K(x, x') = \varphi(x)^\mathsf{T}\varphi(x'),$$

y se sustituye. Nunca se calcula $\varphi(x)$: basta con conocer la forma funcional del kernel. Lo que costaba $\mathcal{F}$ operaciones ahora cuesta lo que cueste evaluar K, que suele ser $\mathcal{O}(d)$.

Teorema 6.2 (Mercer)**.** *Para toda función $K : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ simétrica y semidefinida positiva, existen un espacio de Hilbert $\mathcal{F}$ y una función $\varphi : \mathbb{R}^d \to \mathcal{F}$ tales que*

$$K(x, x') = \langle \varphi(x), \varphi(x') \rangle.$$

Idea clave

Este teorema es el permiso para trabajar al revés. No hace falta construir el kernel a partir de un conjunto de funciones base $\varphi_1(x), \ldots, \varphi_r(x)$: basta definir una K que cumpla las dos condiciones, simetría y semidefinida positiva, y el teorema garantiza que *alguna* φ existe. Nunca hay que verla.

Kernels estándar

Polinomial	$K(x, x') = (1 + x^\mathsf{T}x')^p, \quad p = 1, 2, \ldots$
Base radial (RBF)	$K(x, x') = \exp\left(-\frac{\beta}{2}\,\|x - x'\|^2\right), \quad \beta > 0$

Con $p = 1$ el polinomial es prácticamente el producto interno original, o sea la SVM lineal de siempre. El RBF corresponde a un espacio de características de dimensión infinita, y aun así se evalúa con una resta, una norma y una exponencial.

Clasificar con la solución dual

Ya con los α_t resueltos, una instancia nueva se clasifica con la función discriminante:

$$\hat{y}(x) = \theta^\mathsf{T}\varphi(x) + \theta_0 = \sum_{t \in \mathrm{SV}} \alpha_t y_t\, K(x_t, x) + \theta_0,$$

donde SV es el conjunto de vectores de soporte, o sea los que tienen $\alpha_t \neq 0$.

Idea clave

Aquí reaparece la dispersión que ya conocíamos. Muchos α_t salen exactamente cero de la optimización, y son distintos de cero *solo* para los vectores de soporte. Por eso la suma corre sobre SV y no sobre los n ejemplos: los demás no aportan nada al clasificar.

Ojito...

No sabemos qué ejemplos van a ser vectores de soporte hasta haber resuelto la optimización. Y su identidad depende del kernel: cambiar de kernel cambia quiénes son.

De dónde sale θ_0

El offset se cayó del problema dual, así que hay que recuperarlo aparte. Se usa que para todo vector de soporte la restricción se cumple con igualdad:

$$y_i\Big(\sum_{t \in \mathrm{SV}} \alpha_t y_t\, K(x_t, x_i) + \theta_0\Big) = 1, \qquad i \in \mathrm{SV},$$

y como $y_i \in \{-1, 1\}$ implica $y_i^2 = 1$, se despeja

$$\theta_0 = y_i - \sum_{t \in \mathrm{SV}} \alpha_t y_t \, K(x_t, x_i).$$

El margen geométrico en forma kernel

Sigue siendo $1/\|\theta\|$, solo que ahora $\|\theta\|$ se escribe con el kernel:

$$\hat{\gamma}_{\mathrm{geom}} = \left(\sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, K(x_i, x_j) \right)^{-1/2}.$$

La forma dual relajada

Todo lo anterior supone que los datos sí son separables en el espacio de características. Con variables de holgura el primal era

$$\text{mín} \quad \frac{\|\theta\|^2}{2} + C \sum_{t=1}^{n} \xi_t \qquad \text{sujeto a} \qquad y_t(\theta^{\mathsf{T}} \varphi(x_t) + \theta_0) \geq 1 - \xi_t.$$

Su dual es casi idéntico al de arriba. La *única* diferencia es que los multiplicadores quedan acotados por arriba:

$$\max_{\alpha} \quad \sum_{t=1}^{n} \alpha_t - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, K(x_i, x_j) \qquad \text{sujeto a} \qquad 0 \leq \alpha_t \leq C, \quad \sum_{t=1}^{n} \alpha_t y_t = 0.$$

Idea clave

Por qué la cota superior: sin ella, una restricción imposible de satisfacer haría crecer su α_t hasta infinito, porque el precio de violarla nunca dejaría de subir. El valor C es justamente el punto donde uno decide dejar de insistir con esa restricción. Es el mismo C del primal, visto desde el otro lado.

Ojito...

Con holgura, θ_0 ya no se despeja con cualquier $\alpha_t > 0$. Hay que usar los que cumplen $0 < \alpha_t < C$, o sea excluir los que están en el tope. Un $\alpha_t = C$ marca un punto que *viola* la restricción de margen, y para ese la igualdad no se cumple.

Valor de α_t	Dónde cae el ejemplo	Sirve para θ_0
$\alpha_t = 0$	Fuera del margen, bien clasificado	No
$0 < \alpha_t < C$	Justo sobre el margen	Sí
$\alpha_t = C$	Viola el margen	No

Clasificación multiclase

Hasta aquí todo fue binario. En multiclase cada punto de entrenamiento pertenece a una de N clases distintas, y se quiere una función que, dado un punto nuevo, determine correctamente a cuál pertenece.

Ojito...

Eso **no** es lo mismo que multietiqueta. Hay escenarios con varias categorías donde un punto puede pertenecer a varias a la vez, y ese es otro problema.

One vs All (OVA)

Se construyen N clasificadores binarios. Para el i-ésimo, los positivos son todos los puntos de la clase i y los negativos todos los que no. Si f_i es ese clasificador, se decide con

$$f(x) = \arg\max_i \ f_i(x).$$

All vs All (AVA)

Un clasificador por cada par de clases i, j, donde i son los positivos y j los negativos. Como $f_{ji} = -f_{ij}$, en realidad solo hay que entrenar $N(N-1)/2$ de ellos. Se decide sumando votos:

$$f(x) = \arg\max_i \sum_j f_{ij}(x).$$

También se le llama *all-pairs* o *one-vs-one*.

Idea clave

OVA y AVA son tan simples que mucha gente los inventó por separado. Y eligiendo buenos clasificadores binarios de base, funcionan tan bien como cualquier otra cosa que se te ocurra.

	OVA	**AVA**
Clasificadores	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$
Tamaño de cada uno	Todo el conjunto	Solo dos clases
Balance de clases	Desbalanceado	Balanceado

AVA sale más rápido y usa menos memoria pese a entrenar más clasificadores, porque cada uno es mucho más chico. Si el tiempo de entrenamiento es superlineal en el número de puntos, que es el caso de la SVM, AVA es la mejor opción. OVA además produce muestras de entrenamiento desbalanceadas: una clase contra las otras $N-1$ juntas.

SVM multiclase directa

En lugar de armar varios clasificadores binarios, lo natural es distinguir todas las clases en una sola optimización.

Weston y Watkins (1999). Un vector de pesos w_m por clase, y una holgura $\xi_{i,t}$ por cada ejemplo y cada clase equivocada:

$$\text{mín} \quad \frac{1}{2}\sum_{m=1}^{k} w_m^\mathsf{T} w_m + C\sum_{i=1}^{l}\sum_{t\neq y_i} \xi_{i,t}$$

sujeto a

$$w_{y_i}^\mathsf{T}\varphi(x_i) + b_{y_i} \geq w_t^\mathsf{T}\varphi(x_i) + b_t + 2 - \xi_{i,t}, \qquad \xi_{i,t} \geq 0.$$

Se decide con $\arg\max_m \left(w_m^\mathsf{T}\varphi(x) + b_m\right)$. Su desventaja principal es el tiempo de cómputo: el problema cuadrático resultante es enorme.

Crammer y Singer (2001). Misma idea pero con una sola holgura ξ_i por ejemplo, no una por clase:

$$\text{mín} \quad \frac{1}{2}\sum_{m=1}^{k} w_m^\mathsf{T} w_m + C\sum_{i=1}^{l}\xi_i$$

sujeto a

$$w_{y_i}^\mathsf{T}\varphi(x_i) - w_t^\mathsf{T}\varphi(x_i) \geq 1 - \delta_{y_i,t} - \xi_i,$$

donde $\delta_{i,j}$ es la delta de Kronecker, que vale 1 si $i = j$ y 0 si no. La optimización se hace sin los términos de offset b_m.

SimMSVM (He et al., 2012). Reduce el número de restricciones de margen a l, una por ejemplo, comparando la clase correcta contra el promedio de las demás:

$$w_{y_i}^\mathsf{T}\varphi(x_i) - \frac{1}{k-1}\sum_{m\neq y_i} w_m^\mathsf{T}\varphi(x_i) \geq 1 - \xi_i.$$

Sus autores probaron que su formulación dual es exactamente la de Crammer y Singer con una restricción adicional, y cada modelo de clase se construye sobre sus propios vectores de soporte.

7. Preparación de datos y redes neuronales

10 de septiembre de 2026

Resumen rápido. Antes de aplicar cualquier clasificador hay que preparar los datos: limpiarlos y ponerlos en una escala común, decidiendo entre normalizar y estandarizar. Luego, cómo partir la muestra para entrenar sin caer en overfitting ni en underfitting, y la variante estratificada de la validación cruzada para datos desbalanceados. Al final se abrió el tema nuevo, las redes neuronales artificiales, con la neurona biológica que les sirve de modelo.

Preparación de los datos

A veces queremos aplicar un clasificador, un algoritmo que nos permita separar, clasificar o estimar. Pero antes están los datos, y lo primero es limpiarlos: eliminar columnas vacías, llenar los valores faltantes y demás.

Después viene la escala. Los atributos suelen venir en unidades distintas, y para evitar que unos dominen a otros hay que **normalizar** o **estandarizar**.

Técnica	**Qué hace**	**Fórmula**
Estandarizar	Media 0, varianza 1	$z = \dfrac{x - \mu}{\sigma}$
Normalizar	Todo entre 0 y 1	$x' = \dfrac{x - \text{mín}}{\text{máx} - \text{mín}}$

Estandarizar reordena los datos en función de su media y su varianza. Eso uniforma el manejo, para que ninguna variable se salga del rango de las demás, y de paso aplasta a los datos atípicos: reduce esa diferencia en vez de dejar que arrastre todo.

Escoger entre las dos es una de las decisiones que hay que tomar, y depende de los datos y del objetivo. La regla que se dio en clase:

Distribución	**Qué aplicar**
No normal	Estandarizar
Normal	Normalizar

Idea clave

La razón de fondo es de dónde salen las constantes de cada fórmula. Normalizar usa el mínimo y el máximo, que son los dos estadísticos más sensibles que existen a un valor extremo: si la distribución tiene cola larga, un solo punto lejano estira el denominador y comprime a todos los demás en una franja diminuta. Estandarizar usa media y desviación, que no se dejan mover tanto por un punto. Por eso la distribución no normal pide estandarizar. Si la distribución es normal, el mínimo y el máximo se portan bien y normalizar es seguro.

Ojito...

El escalado se aplica **por variable**, no sobre todos los datos juntos. Cada columna tiene su propia media y desviación, o su propio mínimo y máximo. Mezclarlas destruye justo lo que se quería arreglar.

Todo esto sirve para que en el clasificador lineal no haya descompensaciones en los pesos. Si un atributo va de 0 a 1 y otro de 0 a 100 000, el peso del segundo tiene que volverse diminuto para compensar, y el entrenamiento se vuelve lento y mal condicionado.

Cómo dividir el conjunto de muestra

La siguiente decisión es cómo partir la muestra para el proceso de entrenamiento. Los dos extremos fallan por razones opuestas:

- Si le pasamos **todos** los datos al clasificador, se los va a aprender y no va a poder generalizar. Eso es **overfitting**, y además nos quedamos sin manera de medirlo, porque ya no hay datos que el modelo no haya visto.
- Si le pasamos **muy pocos**, no alcanza a aprender el patrón de manera adecuada. Eso es **underfitting**.

De ahí las tres particiones, cada una con su trabajo:

Partición	Para qué
Entrenamiento	Ajustar los parámetros del modelo
Validación	Escoger el modelo y sus hiperparámetros
Prueba	Estimar el error real, una sola vez

Los repartos usuales son 70/15/15 o 80/10/10 por ciento.

Ojito...

El conjunto de prueba se toca una vez, al final. Si se usa para decidir algo, deja de ser una estimación honesta del error: se convierte en parte del entrenamiento, y la generalización que reporta ya está contaminada.

Validación cruzada estratificada

La otra opción es la validación cruzada, que ya se vio en la clase del 3 de septiembre (figura 2): se parte el conjunto en k folds y se rota cuál se deja fuera.

Con datos desbalanceados eso no basta. Se parten **por clases** y los folds se arman con datos de cada clase, de modo que cada fold conserve la proporción original.

Idea clave

Sin estratificar, con una clase rara y folds aleatorios puede tocar un fold sin un solo ejemplo de esa clase. Ese fold no mide nada sobre ella, y el promedio de los k errores sale optimista por una razón que no tiene que ver con el clasificador.

Redes neuronales artificiales

Hasta aquí todos los modelos salieron de plantear un problema de optimización. Pero alguien preguntó por qué no tratamos de emular el modo en que aprende el cerebro humano, y de ahí nace la idea de las redes neuronales artificiales.

La neurona biológica

El cerebro está conectado por neuronas. Las conexiones entre neuronas se llaman **sinapsis**, y se dan entre axones y dendritas.

Parte	Qué hace
Dendritas	Reciben las señales de otras neuronas
Soma	Integra lo que llega y decide si dispara
Axón	Lleva la señal de salida hacia afuera
Sinapsis	Une el axón de una con la dendrita de otra, y es donde la señal se pondera

Por qué es eficiente el cerebro? Por el número: del orden de 10^{11} neuronas y unas 10^{14} conexiones entre ellas. No es que cada neurona sea potente, es que son muchísimas y están densamente conectadas.

La neurona artificial

El modelo artificial se queda con la estructura y tira la biología. Cada parte tiene su contraparte:

Biológica	Artificial
Dendritas	Las entradas $x_1, \ldots, x_d$
Sinapsis	Los pesos $w_1, \ldots, w_d$
Soma	La suma ponderada más el sesgo
Axón	La salida, pasada por una función de activación

O sea

$$y = g\left(\sum_{i=1}^{d} w_i x_i + w_0\right),$$

con g la función de activación.

Idea clave

Con g el signo, esa ecuación es exactamente el perceptrón de la clase del 27 de agosto. La neurona artificial no es un modelo nuevo, es el mismo clasificador lineal contado desde la biología. Lo nuevo viene de conectar muchas en capas, que es donde deja de ser lineal.

8. Neurona artificial y separabilidad lineal

15 de septiembre de 2026

Resumen rápido. Continuación de la neurona. El TLU de McCulloch y Pitts con su ejemplo numérico, por qué el escalón es demasiado rígido para modelar una neurona real y cómo lo suaviza la sigmoide, el catálogo de funciones de activación y por qué importa que sean derivables. Al final, la lectura geométrica: el TLU es un clasificador lineal y su frontera es un hiperplano perpendicular al vector de pesos.

El TLU de McCulloch y Pitts

La tarea es modelar algunos de los ingredientes de una red neuronal biológica. El carácter de *todo o nada* del potencial de acción se caracteriza con una señal de dos valores, 0 y 1. Si un nodo recibe n señales de entrada $x_1, \ldots, x_n$, cada una solo puede valer 0 o 1.

El efecto modulador de cada sinapsis se encapsula multiplicando la señal que entra por un peso. Con n pesos $w_1, \ldots, w_n$ se forman los n productos $w_i x_i$, y cada producto es el análogo de un **potencial postsináptico**, que puede ser positivo o negativo según el signo del peso.

Esos productos se combinan sumándolos, y eso emula lo que pasa en el axón. El resultado es la **activación**:

$$a = \sum_{i=1}^{n} w_i x_i.$$

Para emular la generación del potencial de acción hace falta un **umbral** θ: si la activación lo alcanza o lo supera, el nodo emite 1; si no, emite 0.

$$y = \begin{cases} 1 & \text{si } a \geq \theta, \\ 0 & \text{si } a < \theta. \end{cases}$$

Definición 8.1. Esa unidad, suma ponderada más umbral con función escalón, se llama **TLU**, *threshold logic unit*, y es de McCulloch y Pitts (1943).

Ejemplo 8.2. Una unidad de cinco entradas con pesos $(0.5,\ 1.0,\ -1.0,\ -0.5,\ 1.2)$, a la que se le presenta el patrón $(1, 1, 1, 0, 0)$.

La activación es

$$\begin{aligned} a &= (0.5 \times 1) + (1.0 \times 1) + (-1.0 \times 1) + (-0.5 \times 0) + (1.2 \times 0) \\ &= 0.5 + 1.0 - 1.0 + 0 + 0 \\ &= 0.5. \end{aligned}$$

Con umbral $\theta = 0.2$, como $a = 0.5 > 0.2$, la salida es $y = 1$.

Fíjate en el tercer término: el peso $w_3 = -1.0$ es negativo, así que esa entrada **inhibe** en vez de excitar, y se come completo el aporte de la segunda. Los pesos negativos son la contraparte de las sinapsis inhibitorias.

Comunicación con señales no binarias

El escalón es demasiado rígido para lo que hace una neurona real. Se cree que las neuronas codifican su señal en los *patrones* de disparo del potencial de acción, no en la simple presencia o ausencia de un

pulso. Si f es la frecuencia de disparo, está acotada por abajo en cero y por arriba en un $f_{\text{máx}}$ que impone la duración del periodo refractario entre disparos.

La salida es suavizar el escalón hasta una función continua que aplaste el rango, de modo que y dependa de forma suave de a. La forma conveniente es la **sigmoide logística**:

$$y = \sigma(a) = \frac{1}{1 + \exp\left(-\dfrac{a-\theta}{\rho}\right)}.$$

Desglosando la fórmula pieza por pieza:

- El término $a - \theta$ centra la curva en el umbral: cuando $a = \theta$ el exponente es cero, $\exp(0) = 1$ y queda $y = 1/2$. El umbral es el punto donde la neurona está indecisa.
- El signo menos hace que la curva sea creciente. Si a crece mucho, el exponente se va a $-\infty$, $\exp(-\infty) = 0$ y $y \to 1$. Si a se hace muy negativa, el exponente se va a $+\infty$ y $y \to 0$.
- El divisor ρ controla la forma. Valores grandes de ρ achican el exponente y aplanan la curva; valores pequeños la hacen subir más abruptamente. En el límite $\rho \to 0$ se recupera el escalón. En muchos textos se omite, lo que equivale a fijar $\rho = 1$.

Ojito...

La sigmoide tiende a 1 cuando a crece, pero **nunca lo alcanza**. Igual por abajo: se acerca a 0 sin tocarlo. Nunca satura del todo, y eso va a importar cuando se derive, porque la pendiente nunca es exactamente cero.

Idea clave

La derivada de la sigmoide se escribe en términos de ella misma:

$$\sigma'(a) = \sigma(a)\,[1 - \sigma(a)]\,.$$

Eso no es una curiosidad, es lo que hace barato el entrenamiento: para calcular la pendiente en un punto no hay que evaluar nada nuevo, se reusa la salida que ya se calculó en el paso hacia adelante. Nota que vale cero en los extremos, donde σ es 0 o 1, y es máxima en $\sigma = 1/2$, o sea justo en el umbral.

Elegir la función de activación

Hay varias opciones, y cada una cambia el comportamiento de la unidad.

Función	Definición	Rango
Escalón	$y = \begin{cases} 1 & a \geq \theta \\ 0 & a < \theta \end{cases}$	$\{0, 1\}$
Sigmoide	$y = \dfrac{1}{1 + e^{-a}}$	$(0, 1)$
ReLU	$y = \begin{cases} 0 & a < 0 \\ a & \text{si no} \end{cases}$	$[0, \infty)$
Tangente hiperbólica	$y = \dfrac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$	$(-1, 1)$
Softmax	$Y_k = \dfrac{e^{a_k}}{\sum_i e^{a_i}}$	$(0, 1)$, suman 1

Qué aporta cada una:

- **Escalón.** Decisión dura, sin grises. Es lo que define al TLU original.
- **Sigmoide.** El suavizado del escalón. Su salida en $(0, 1)$ se puede leer como una probabilidad.
- **ReLU.** Deja pasar lo positivo y corta lo negativo. Es barata de calcular y su derivada es 1 o 0, sin aplastarse para valores grandes.
- **Tangente hiperbólica.** Es una sigmoide centrada en cero, con salida en $(-1, 1)$. Que esté centrada ayuda porque las salidas no quedan todas del mismo signo.
- **Softmax.** No opera sobre una neurona sino sobre toda la capa: normaliza las activaciones para que sumen 1. Es la que se usa en la capa de salida de un clasificador multiclase.

Ojito...

El punto que hay que retener: **el escalón no es derivable**. Las otras sí, y eso es esencial para el cálculo eficiente de los pesos. Todo el entrenamiento por descenso de gradiente descansa en poder derivar la función de activación, así que el escalón deja a la unidad sin manera práctica de aprender.

El TLU como clasificador lineal

Un TLU parte sus patrones de entrada en dos categorías según su respuesta binaria. Esas categorías se pueden ver como regiones de un espacio multidimensional separadas por el equivalente en muchas dimensiones de una recta o un plano.

Ejemplo 8.3. Un TLU de dos entradas con $w_1 = 1$, $w_2 = 1$ y $\theta = 1.5$. Las cuatro entradas booleanas posibles dan:

x_1	x_2	Activación	Salida
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

Solo el patrón $(1, 1)$ supera el umbral de 1.5. El TLU está clasificando sus entradas en dos grupos: las que dan 1 y las que dan 0. Como cada patrón tiene dos componentes, se puede dibujar cada uno como un punto del **espacio de patrones**, y ahí $(1, 1)$ queda de un lado de una recta y los otros tres del otro.

Esa función es el AND lógico.

De dónde sale la recta

La condición crítica de clasificación ocurre cuando la activación iguala al umbral. Para dos entradas, igualando a y θ:

$$\begin{aligned} w_1 x_1 + w_2 x_2 &= \theta \\ w_2 x_2 &= -w_1 x_1 + \theta \\ x_2 &= -\left(\frac{w_1}{w_2}\right) x_1 + \left(\frac{\theta}{w_2}\right). \end{aligned}$$

Eso es una recta $x_2 = ax_1 + b$, con pendiente $a = -w_1/w_2$ y ordenada al origen $b = \theta/w_2$ sobre el eje x_2. Con los valores del ejemplo, $w_1 = w_2 = 1$ y $\theta = 1.5$, queda $x_2 = -x_1 + 1.5$: una recta que corta ambos ejes en 1.5 y deja a $(1, 1)$ arriba, solo.

Idea clave

Los pesos fijan la **inclinación** de la frontera y el umbral fija su **posición**. Entrenar es mover esa recta hasta que los puntos queden del lado correcto. Por eso a los TLU se les llama **clasificadores lineales** y a sus patrones, **linealmente separables**. Es el mismo objeto que el perceptrón de la clase del 27 de agosto, visto desde la biología.

En forma vectorial

Usando la definición de producto interno, la activación de un TLU de n entradas se escribe

$$a = W \cdot X,$$

y la condición crítica queda

$$W \cdot X = \theta.$$

Esa ecuación define un **hiperplano perpendicular al vector de pesos**. De un lado quedan todos los patrones que el TLU clasifica como 1; del otro, los que clasifica como 0.

Ojito...

Que el hiperplano sea perpendicular a W no es un detalle decorativo: quiere decir que W apunta hacia el lado de los positivos. Esa observación es la que va a justificar toda la regla de entrenamiento de la clase siguiente, porque corregir un error será *rotar* W hacia el patrón o alejarlo de él.

El umbral como un peso más

Hay un truco de notación que simplifica todo lo que viene. Partiendo de la condición de salida $W \cdot X \geq \theta$, se resta θ de ambos lados y se hace explícito el signo:

$$W \cdot X + (-1)\,\theta \geq 0.$$

Leído así, el umbral es **un peso más**, alimentado por una entrada fija en -1. El vector de pesos, que era de dimensión n para una unidad de n entradas, pasa a ser el vector de dimensión $n+1$

$$[\, w_1,\ w_2,\ \ldots,\ w_n,\ \theta\,],$$

que se llama **vector de pesos aumentado**. Con él, la función del nodo se escribe sin umbral a la vista:

$$W \cdot X \geq 0 \implies y = 1, \qquad W \cdot X < 0 \implies y = 0.$$

Idea clave

En el espacio aumentado, $W \cdot X = 0$ define el hiperplano de decisión, que sigue siendo ortogonal a W. Y la condición de umbral cero significa que el hiperplano **pasa por el origen**, porque esa es la única forma de que $W \cdot X = 0$ tenga solución en el origen. La ganancia práctica es que ya no hay que entrenar el umbral aparte: entra al mismo saco que los pesos y la misma regla lo ajusta.

9. Descenso de gradiente y backpropagation

17 de septiembre de 2026

Resumen rápido. Cómo se entrena. Primero la regla del perceptrón, que sale de un argumento puramente geométrico sobre rotar el vector de pesos. Luego el descenso de gradiente sobre una función de error, la regla delta con su derivación, y por qué converge donde el perceptrón oscila. Al final el problema del XOR, que obliga a meter capas ocultas, y backpropagation como la respuesta a quién tiene la culpa del error.

Entrenar una red

Ajustar los pesos y los umbrales se hace con un proceso iterativo: se presentan ejemplos de la tarea una y otra vez, y en cada presentación se hacen cambios pequeños para acercarlos a sus valores deseados. A eso se le llama **entrenar la red**, y al conjunto de ejemplos, **conjunto de entrenamiento**.

Trabajando con un solo nodo, el conjunto de entrenamiento son pares $\{V, t\}$, donde V es un vector de entrada y t es la clase objetivo, 1 o 0, a la que V pertenece. La condición de salida 1 se escribe $W \cdot V \geq \theta$.

Ajustar el vector de pesos

Aquí el razonamiento es geométrico y vale la pena seguirlo despacio, porque de él sale la regla.

Caso 1: el objetivo es $t = 1$ pero sale $y = 0$. Para producir un 0 la activación tuvo que ser negativa, así que el producto punto $W \cdot V$ fue negativo. Eso significa que los dos vectores apuntaban en direcciones opuestas, con ángulo mayor a 90°. Lo que hace falta es **rotar W para que apunte más hacia V**, y eso se logra sumándole una fracción de V:

$$W' = W + \alpha V, \qquad 0 < \alpha < 1.$$

Caso 2: el objetivo es $t = 0$ pero sale $y = 1$. Ahora la activación fue positiva cuando debió ser negativa, el ángulo es menor a 90°, y hay que **alejar W de V** restando una fracción:

$$W' = W - \alpha V.$$

Las dos situaciones se combinan en una sola regla:

$$\boxed{W' = W + \alpha\,(t - y)\,V}$$

Idea clave

El factor $(t - y)$ hace todo el trabajo de selección, y solo puede tomar tres valores. Si acertó, $t - y = 0$ y no se toca nada. Si $t = 1$ y $y = 0$, vale $+1$ y se suma. Si $t = 0$ y $y = 1$, vale -1 y se resta. Una sola expresión cubre los tres casos sin condicionales.

Esa es la **regla de entrenamiento del perceptrón**, de Frank Rosenblatt (1957-1958). El α se llama **tasa de aprendizaje** y controla qué tan grande es cada corrección.

Algoritmo 9.1 Aprendizaje del perceptrón

repetir
  para cada par de entrenamiento (V, t) **hacer**
    evaluar la salida y al presentar V al TLU
    si $y \neq t$ **entonces**
      $W \leftarrow W + \alpha\,(t - y)\,V$
    si no
      no hacer nada
    fin si
  fin para
hasta que $y = t$ para todos los vectores

Descenso de gradiente sobre un error

La regla anterior sale de la geometría. Ahora se llega a una parecida desde la optimización, y esa segunda ruta es la que generaliza a redes de varias capas.

Considerando una red de un solo TLU, cualquier función de error E que exprese la discrepancia entre la salida deseada y la real puede verse como una **función de los pesos**:

$$E = E(w_1, w_2, \ldots, w_{n+1}).$$

El $n + 1$ es porque el umbral ya va dentro, como peso aumentado.

El vector de pesos óptimo se encuentra minimizando E por descenso de gradiente:

$$\Delta w_i = -\,\alpha\,\frac{\partial E}{\partial w_i}.$$

Idea clave

El signo menos es el corazón del método. La derivada $\partial E/\partial w_i$ dice hacia dónde el error **crece** si muevo ese peso. Como queremos bajarlo, nos movemos en la dirección contraria. Si la pendiente es positiva, el peso baja; si es negativa, sube. Y donde la pendiente es cero, en el mínimo, el peso deja de moverse solo.

Si e^p es el error del patrón de entrenamiento p, el error total es el promedio sobre todos los patrones:

$$E = \frac{1}{N}\sum_{p=1}^{N} e^p.$$

Cómo definir el error de un patrón

Vale la pena ver los tres intentos, porque cada uno arregla un problema del anterior.

Primer intento: la diferencia. $e^p = t^p - y^p$. No sirve: la combinación $t^p = 0$, $y^p = 1$ da -1, mientras $t^p = 1$, $y^p = 0$ da $+1$. Los dos casos están igual de equivocados, pero uno cuenta como error negativo y al promediarlos se cancelarían.

Segundo intento: elevar al cuadrado. $e^p = (t^p - y^p)^2$. Ya es simétrico y siempre positivo. Pero hay un problema más fino: y^p depende de a^p a través de la **función escalón, que es discontinua**, así que no se puede derivar.

Tercer intento: medir contra la activación. Como la salida cambia justo cuando la activación cambia de signo, con $a \geq 0$ dando $y = 1$, se define el error respecto a la activación:

$$e^p = (t^p - a^p)^2.$$

Y se agrega un factor de 1/2, que no cambia dónde está el mínimo pero simplifica las derivadas:

$$e^p = \tfrac{1}{2}\,(t^p - a^p)^2.$$

La regla delta

Como E depende de todos los patrones, sus derivadas también, y eso obligaría a presentar el conjunto completo para evaluar un solo gradiente. Se evita presentando un patrón p, evaluando $\partial e^p/\partial w_i$ y usándolo como **estimación** del gradiente verdadero $\partial E/\partial w_i$.

La derivada, paso a paso

Las diapositivas dicen que se puede demostrar. Se demuestra así. Partiendo de $e^p = \frac{1}{2}(t^p - a^p)^2$ con $a^p = \sum_i w_i x_i^p$, y aplicando la regla de la cadena:

$$\begin{aligned}
\frac{\partial e^p}{\partial w_i} &= \frac{1}{2} \cdot 2\,(t^p - a^p) \cdot \frac{\partial (t^p - a^p)}{\partial w_i} && \text{cadena sobre el cuadrado} \\
&= (t^p - a^p) \cdot \left(-\frac{\partial a^p}{\partial w_i}\right) && t^p \text{ es constante} \\
&= -\,(t^p - a^p)\, x_i^p && \frac{\partial}{\partial w_i} \sum_j w_j x_j^p = x_i^p.
\end{aligned}$$

Ahí se ve para qué era el 1/2: se cancela con el 2 que baja del exponente y la expresión queda limpia.

Sustituyendo en $\Delta w_i = -\alpha\, \partial E/\partial w_i$, los dos signos menos se cancelan y queda la **regla delta**:

$$\boxed{\Delta w_i = \alpha\,(t^p - a^p)\, x_i^p}$$

Cómo se lee la fórmula

El gradiente depende de dos cosas, y las dos tienen sentido:

- **De la discrepancia** $(t^p - a^p)$**.** Entre más grande sea el error, más grande la corrección, y viceversa. Si ya acertó, la corrección es cero.
- **De la entrada** x_i^p**.** Si esa entrada vale cero, la i-ésima componente no está contribuyendo a la activación, y entonces su peso no tiene culpa de nada: no se toca. Si la entrada es grande, ese peso sí influye mucho y se corrige proporcionalmente.

Algoritmo 9.2 Entrenamiento con la regla delta

```
repetir
    para cada par de entrenamiento (V, t) hacer
        evaluar la activación a al presentar V al TLU
        Δw_i ← α (t^p − a^p) x_i^p        // para cada peso i
        w_i ← w_i + Δw_i
    fin para
hasta que la razón de cambio del error sea suficientemente pequeña
```

Esta regla la propusieron Widrow y Hoff (1960), así que originalmente se llamó **regla de Widrow-Hoff**. Se conoce más como **regla delta**, y al término $(t^p - a^p)$ se le dice *la* δ, porque es una diferencia.

Ojito...

La diferencia entre las dos reglas. Si **no existe solución**, o sea si el problema no es linealmente separable, bajo la regla del perceptrón los pesos se siguen alterando en cantidades significativas y el vector **oscila** sin parar. La regla delta, en cambio, siempre converge (con α suficientemente chica) a un vector w_0 donde el error es mínimo, aunque ese mínimo no sea cero y algunos patrones queden mal clasificados. Converger a la mejor respuesta imperfecta es mucho mejor que no converger.

Versión con sigmoide

Se puede reinstalar la salida y^p en la definición del error, ahora que se usa una sigmoide continua en vez del escalón:

$$e^p = \tfrac{1}{2}\,(t^p - y^p)^2, \qquad y^p = \sigma(a^p).$$

Al derivar aparece un eslabón más en la cadena, la derivada de la sigmoide:

$$\frac{\partial e^p}{\partial w_i} = -\,(t^p - y^p) \cdot \underbrace{\frac{d\sigma}{da}}_{\sigma'(a)} \cdot x_i^p,$$

y por lo tanto

$$\boxed{\Delta w_i = \alpha\,\sigma'(a)\,(t^p - y^p)\,x_i^p}$$

Idea clave

Ese factor $\sigma'(a)$ gobierna todo. El efecto de cambiar un peso depende de qué tan sensible sea la salida a la activación, y esa sensibilidad es justo la pendiente de la sigmoide. En las colas, donde la sigmoide está casi plana, $\sigma'(a) \approx 0$ y el peso casi no se mueve aunque el error sea grande: la neurona está saturada. El aprendizaje es más rápido cerca del umbral, donde la pendiente es máxima.

El límite: el XOR

Todo lo anterior descansa en que los patrones sean linealmente separables. Hay funciones booleanas elementales que no lo son.

Ejemplo 9.1. Tomando $(0,0)$ y $(1,1)$ con $t = 0$, y $(0,1)$ y $(1,0)$ con $t = 1$. Los positivos quedan en dos esquinas opuestas del cuadrado y los negativos en las otras dos.

No existe ninguna recta que separe unos de otros: hay que dejar dos esquinas diagonalmente opuestas de un lado, y cualquier recta que deje a $(0,1)$ y $(1,0)$ juntas necesariamente encierra también a una de las otras dos.

Esa función es el XOR, y el señalamiento es de Minsky y Papert en *Perceptrons* (1969).

Ojito...

Ese libro frenó la investigación en redes neuronales casi dos décadas. La limitación que señalaba era real, pero era la de *una sola* unidad, no la de la idea entera.

La salida: más capas

La solución se encuentra fácil si el espacio se parte en **tres regiones** en vez de dos. Con dos rectas paralelas se puede dejar a $(0,1)$ y $(1,0)$ en la franja de en medio y a las otras dos esquinas afuera, una de cada lado.

1. Si en lugar de una neurona de salida se usan **dos**, se obtienen dos rectas, y con ellas quedan delimitadas las tres regiones.

2. Para escoger entre una zona y otra de las tres hace falta **otra capa**, con una neurona cuyas entradas sean las salidas de las neuronas anteriores.

Eso da una red de tres neuronas repartidas en dos capas: dos neuronas que reciben x_1 y x_2 con pesos $w_{11}, w_{21}, w_{12}, w_{22}$, y una tercera que recibe las salidas de esas dos con pesos w_{31}, w_{32} y produce y.

Definición 9.2. Una capa está **totalmente conectada** cuando las entradas de cada unidad de la capa son las mismas. Una capa toma una entrada $x \in \mathbb{R}^m$ y produce una salida $y \in \mathbb{R}^n$.

Con m neuronas en una capa, el error se suma sobre todas:

$$e^p = \frac{1}{2}\sum_{j=1}^{m}(t_j^p - y_j^p)^2,$$

y el gradiente del i-ésimo peso de la j-ésima neurona solo involucra el término de error de *esa* neurona:

$$\Delta w_{ji} = \alpha\, \sigma'(a_j)\, (t_j^p - y_j^p)\, x_{ji}^p.$$

Backpropagation

La idea sigue siendo descenso de gradiente sobre E como función de los pesos, pero ahora hay que tomar en cuenta los pesos de dos capas.

Para la **capa de salida** es directo: se tiene acceso al desajuste entre el objetivo y la salida, y la fórmula de arriba aplica tal cual.

El problema son las neuronas de la **capa oculta**, y la pregunta se plantea así: cuánta *culpa* o responsabilidad hay que asignarle a los nodos ocultos por el error que aparece en la salida. Porque claramente, si la capa oculta le está pasando mala información a la de salida, las neuronas de salida no pueden aspirar a acertar.

Descomponer la regla conocida

El punto de partida es mirar la regla que ya se tiene y separarla en piezas:

$$\Delta w_{ji} = \alpha \underbrace{\sigma'(a_j)}_{\text{pendiente}} \underbrace{(t_j^p - y_j^p)}_{\delta^j} \underbrace{x_{ji}^p}_{\text{entrada}}.$$

La entrada y la pendiente de la sigmoide están determinadas únicamente por la **estructura de la neurona**, así que van a aparecer igual en la expresión de una neurona oculta. El término que sobra, $\delta = (t^p - y^p)$, es el específico de las neuronas de salida, y toda la tarea se reduce a encontrar su equivalente para una neurona oculta.

Con eso, la regla de la k-ésima neurona oculta se escribe ya:

$$\Delta w_{ki} = \alpha\, \sigma'(a_k)\, \delta^k\, x_{ki}^p,$$

a falta de saber qué es δ^k.

Encontrar la delta de una neurona oculta

Considera el enlace entre la k-ésima neurona oculta y la j-ésima de salida. Hay dos factores en juego:

- La contribución que la neurona j hace al error está expresada en su propia δ^j, que ya se sabe calcular.
- La influencia que k tiene sobre j está dada por el peso w_{jk} de la conexión entre ellas.

La interacción entre los dos se captura combinándolos **multiplicativamente**, como $\delta^j w_{jk}$. Y como la neurona k casi con seguridad alimenta a varias neuronas de salida, hay que sumar esos productos sobre todas:

$$\boxed{\delta^k = \sum_{j \in I_k} \delta^j \, w_{jk}}$$

donde I_k es el conjunto de neuronas de la capa de salida que reciben entrada de la neurona oculta k.

Idea clave

Ahí está el nombre del método. La δ de una neurona oculta no se mide contra ningún objetivo, porque no hay objetivo para una capa oculta. Se **arma hacia atrás**, juntando las deltas de las neuronas a las que alimenta, cada una pesada por la fuerza de la conexión. El error viaja de regreso por las mismas aristas que la señal usó para ir hacia adelante, y los pesos hacen de factor de reparto en las dos direcciones.

La regla unificada

Para cualquier neurona k, oculta o de salida, el descenso de gradiente queda en una sola línea:

$$\Delta w_{ki} = \alpha \, \delta^k \, x^p_{ki},$$

y lo único que cambia es cómo se calcula δ^k:

$$\begin{aligned} \delta^k &= \sigma'(a_k)\,(t^p_k - y^p_k) && \text{si } k \text{ es de salida,} \\ \delta^k &= \sigma'(a_k) \sum_{j \in I_k} \delta^j \, w_{jk} && \text{si } k \text{ es oculta.} \end{aligned}$$

Algoritmo 9.3 Backpropagation

1: presentar el patrón en la capa de entrada
2: que las neuronas ocultas evalúen su salida con ese patrón
3: que las neuronas de salida evalúen la suya con el resultado del paso anterior
4: aplicar el patrón objetivo a la capa de salida
5: calcular las δ de las neuronas de salida
6: entrenar cada neurona de salida por descenso de gradiente
7: para cada neurona oculta, calcular su δ^k
8: entrenar cada neurona oculta con la δ^k del paso anterior, por descenso de gradiente

Pasos	Nombre	Qué fluye
1 a 3	Paso hacia adelante	La información, de entrada a salida
4 a 8	Paso hacia atrás	Las δ, de salida a ocultas

El paso 7 es el que propaga las δ de vuelta desde los nodos de salida hacia las unidades ocultas, y de ahí el nombre **backpropagation**. En los años ochenta lo popularizaron Geoffrey Hinton, David Rumelhart y Ronald Williams.

Idea clave

Se puede tener más de una capa oculta, y se entrena exactamente igual: las δ de cada capa, salvo la de salida, se expresan en términos de las de la capa que tienen encima. La recursión es la misma sin importar la profundidad, y eso es lo que vuelve entrenable a una red arbitrariamente honda.

10. Redes convolucionales

22 de septiembre de 2026

Resumen rápido. La primera arquitectura que no conecta todo con todo. Se ve por qué una capa densa no sirve para imágenes, qué hace la convolución, cómo cambia la regla delta cuando un peso se reusa en muchos lugares, y cómo se arma una CNN completa. La segunda mitad es el problema que se destapó al querer apilar capas: el gradiente se desvanece, y ReLU solo lo arregla a medias.

Por qué no basta una capa densa

La clase pasada cerró diciendo que se puede tener cuantas capas ocultas se quiera. Con imágenes eso choca de inmediato con dos problemas.

El primero es el número de pesos. Una imagen de 224×224 en color aplanada da $150\,528$ entradas. Conectarla a una capa oculta de mil neuronas son 150 millones de pesos, en una sola capa.

El segundo es más de fondo: **aplanar destruye la estructura espacial**. Dos pixeles vecinos quedan como dos coordenadas cualquiera del vector, sin nada que diga que estaban juntos. Y una característica que la red aprende a detectar en una esquina no le sirve de nada en la otra, porque son pesos distintos.

La convolución

En la regla delta de una capa densa, la activación usa la entrada entera:

$$a = \sum_i w_i x_i.$$

En una capa convolucional la neurona no mira toda la imagen de golpe. Mira solo una ventana chica, llamada **filtro** o **kernel**:

$$a_{i,j} = \sum_m \sum_n w_{m,n} \cdot x_{i+m,\, j+n}.$$

Los índices se leen así:

- (i, j) es la posición de *salida*. Hay una activación por cada lugar donde se pueda apoyar el filtro.
- (m, n) recorre el filtro, no la imagen. Si el filtro es de 3×3, son nueve términos y ya.
- $x_{i+m,\, j+n}$ es la ventana de la imagen anclada en (i, j). El $+i$ y el $+j$ son lo que desliza la ventana.

Ejemplo 10.1. Una imagen de 5×5 con un borde vertical, las dos primeras columnas en 0 y las tres siguientes en 1, y un filtro de 3×3 que detecta bordes verticales:

$$X = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}, \qquad W = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

La activación en la esquina, con la ventana sobre las filas 0 a 2 y las columnas 0 a 2:

$$a_{0,0} = \underbrace{(1)(0) + (0)(0) + (-1)(1)}_{-1} + \underbrace{(1)(0) + (0)(0) + (-1)(1)}_{-1} + \underbrace{(1)(0) + (0)(0) + (-1)(1)}_{-1} = -3.$$

Corriendo la ventana por las nueve posiciones posibles queda un mapa de 3×3:

$$A = \begin{pmatrix} -3 & -3 & 0 \\ -3 & -3 & 0 \\ -3 & -3 & 0 \end{pmatrix}.$$

La última columna vale cero porque ahí la ventana cae entera dentro de la zona uniforme de unos, y el filtro suma tanto como resta. Las otras dos son distintas de cero porque la ventana cruza el borde. El filtro es ciego a lo plano y solo responde a los cambios.

Canales, paso y relleno

La fórmula esconde tres cosas que en la práctica hay que fijar.

Los canales. Una imagen no es una matriz, es un volumen de alto por ancho por *canales*: tres para color, uno en escala de grises. El filtro tiene la misma profundidad que su entrada, o sea es de $f \times f \times C$, y la suma de la fórmula corre también sobre los canales. Por eso un filtro, por grueso que sea, produce **un solo** mapa de salida de dos dimensiones. Si se quieren K mapas hacen falta K filtros distintos, y esos K mapas son los canales de la capa siguiente.

El paso, o *stride*, es de cuánto en cuánto se desliza la ventana. Con paso 1 se apoya en cada pixel; con paso 2 se salta uno y el mapa de salida sale a la mitad de tamaño.

El relleno, o *padding*, son ceros que se agregan alrededor de la imagen. Sin relleno el mapa encoge en cada capa, y los pixeles del borde participan en menos ventanas que los del centro, así que la red los ve menos.

Juntando los tres, el tamaño de salida de una convolución sobre una entrada de $n \times n$ con filtro de $f \times f$, relleno p y paso s es

$$\left\lfloor \frac{n - f + 2p}{s} \right\rfloor + 1.$$

En el ejemplo de arriba, $n = 5$, $f = 3$, sin relleno y con paso uno, da $5 - 3 + 1 = 3$: por eso la imagen de 5×5 produjo un mapa de 3×3. Con relleno $p = 1$ habría dado 5, o sea el mismo tamaño que la entrada, que es lo que se hace casi siempre para que la red pueda ser honda sin quedarse sin mapa.

Idea clave

El conteo de pesos de una capa convolucional sale de ahí: $K(f^2C + 1)$, contando un sesgo por filtro. Lo que *no* aparece en esa cuenta es n. El tamaño de la imagen afecta cuánto se tarda la capa, pero no cuántos parámetros tiene, y esa independencia es justo lo que la capa densa no tiene.

Lo que se gana

Tres propiedades, y las tres salen de la misma fórmula:

- **Conexión local.** Cada neurona ve solo su ventana, que se llama su campo receptivo. No hay un peso por pixel de la imagen, hay un peso por celda del filtro.
- **Pesos compartidos.** El *mismo* filtro se desliza por toda la imagen. Un filtro de 3×3 son nueve pesos sin importar si la imagen es de 28×28 o de 1000×1000.
- **Equivarianza a la traslación.** Si el borde se mueve, la respuesta se mueve con él. Lo que la red aprende a detectar en un lugar le sirve en todos.

Idea clave

Los números del principio con una capa convolucional: 64 filtros de $3 \times 3 \times 3$ son 1 728 pesos, contra los 150 millones de la capa densa. Cuatro órdenes de magnitud menos, y encima conserva la vecindad de los pixeles.

Backpropagation con pesos compartidos

La regla delta de la clase pasada, para una neurona cualquiera:

$$\Delta w_{ki} = \alpha \, \delta^k x^p_{ki}.$$

Se calcula el error local y se multiplica por la entrada asociada a ese peso. En una convolución hay un detalle: un solo peso $w_{m,n}$ contribuyó a *muchos* pixeles de salida durante el paso hacia adelante, así que recibe culpa desde muchos lugares a la vez:

$$\Delta w_{m,n} = \alpha \sum_i \sum_j \delta_{i,j} \cdot x_{i+m,\, j+n}.$$

La doble suma recorre exactamente las posiciones donde se aplicó el filtro. No es una regla nueva: es la misma regla delta, acumulada.

Idea clave

Eso no es un truco de redes, es cálculo. Cuando una variable aparece varias veces en una expresión, su derivada es la suma de las derivadas parciales de cada aparición. Compartir el peso obliga a sumar, y por eso la CNN es, palabras del profesor, la aplicación de la regla delta y el descenso de gradiente a un conjunto compacto de pesos reusados sobre toda la superficie de la imagen.

La arquitectura completa

	Etapa	Qué pasa
1	Entrada	La imagen, sin aplanar
2	Extracción de características	Capas convolucionales alternadas con capas de pooling, típicamente max pooling
3	Transición	Aplanado: los mapas finales se estiran en una columna de neuronas de una dimensión
4	Clasificación	Capas densas, y una capa de salida con una neurona por categoría, normalmente con softmax

El **max pooling** lo nombra la diapositiva pero vale la pena decir qué hace: toma el máximo sobre una ventana, casi siempre de 2×2 y avanzando de dos en dos, con lo que cada dimensión espacial se reduce a la mitad. Sirve para dos cosas. Achica el mapa, y da algo de invariancia a traslaciones chicas: si la característica se corre un pixel, el máximo de la ventana normalmente no cambia. No tiene parámetros que entrenar.

Ejemplo 10.2. Un mapa de 4×4 con ventanas de 2×2 y paso dos, o sea cuatro ventanas que no se

traslapan:

$$\begin{pmatrix} 1 & 3 & 2 & 0 \\ 5 & 2 & 1 & 4 \\ 0 & 1 & 7 & 2 \\ 3 & 2 & 1 & 0 \end{pmatrix} \longrightarrow \begin{pmatrix} 5 & 4 \\ 3 & 7 \end{pmatrix}.$$

De cada bloque queda solo el máximo. Se conserva *qué tan fuerte* respondió el filtro en esa zona y se tira *en qué pixel exacto* respondió, que es precisamente la información que no importa para reconocer el objeto.

Qué aprende cada capa

Apilar convolución y pooling tiene una consecuencia que da sentido a toda la arquitectura: cada capa ve una región más grande de la imagen original que la anterior, aunque su filtro siga siendo de 3×3. El campo receptivo crece con la profundidad, y con él crece lo que la capa puede reconocer.

Profundidad	**Qué detecta**	**Sobre qué**
Primeras capas	Bordes, esquinas, manchas de color	Unos pocos pixeles
Capas intermedias	Texturas y partes: una rueda, un ojo	Combinaciones de lo anterior
Capas hondas	Objetos completos	Combinaciones de partes

Nadie programa esa jerarquía. Sale sola del entrenamiento, y es la razón por la que en visión la profundidad importa tanto: cada nivel solo puede construirse con las piezas que le pasa el de abajo.

La arquitectura que fijó este esquema es **LeNet**, de Yann LeCun en 1998.

El límite de backpropagation

El problema serio es que **el error se diluye exponencialmente** conforme atraviesa capas de regreso hacia el principio de la red. En una red muy honda solo se entrenan las últimas capas; las primeras casi no se mueven.

La causa está en la regla de la cadena: propagar hacia atrás es multiplicar. De la clase del 15 de septiembre, $\sigma'(a) = \sigma(a)[1-\sigma(a)]$, que vale a lo más 0.25, en $a = 0$, y se va a cero en cuanto la neurona se satura.

Ejemplo 10.3. Diez capas con sigmoides, todas en su mejor caso:

$$0.25^{10} \approx 9.5 \times 10^{-7}.$$

El gradiente que llega a la primera capa es un millón de veces más chico que el de la última, y eso suponiendo que ninguna neurona esté saturada. Los números de la diapositiva van en la misma dirección: 0.1 en la capa 4, 0.05 en la 3, 0.005 en la 2 y 0.00025 en la 1.

Antes de 2012 todas las redes usaban sigmoide o tangente hiperbólica, y las dos se saturan: con entradas muy positivas o muy negativas la curva se aplana y su derivada se va a cero. De ahí el nombre, **desvanecimiento del gradiente**.

ReLU

Para resolverlo, los autores de AlexNet metieron ReLU:

$$f(x) = \text{máx}(0, x).$$

Sus dos propiedades, las que según la diapositiva salvaron al aprendizaje profundo:

- **Derivada constante e igual a 1 para $x > 0$.** No encoge nada al multiplicarse en cadena. Para $x < 0$ la derivada es 0 y el error simplemente no pasa.
- **Simplicidad de cómputo.** Una sola operación $\text{máx}(0, x)$, contra la exponencial de la sigmoide.

La referencia es Krizhevsky, Sutskever y Hinton (2012).

Por qué ReLU no alcanzó

Cambiar sigmoide por ReLU permitió que los gradientes fluyeran en redes de 16 o 19 capas, como la VGG. Pero al intentar 50 o 100 capas el gradiente seguía desvaneciéndose. Tres razones:

1. **ReLU moribunda.** Con más capas sube la probabilidad de que alguna neurona reciba solo entradas negativas. Su salida queda en cero, su derivada también, y como el gradiente que la atraviesa es cero, nunca se corrige. La neurona queda muerta.

2. **Gradientes explosivos.** La derivada de ReLU es exactamente 1 del lado positivo, pero el gradiente total en una capa temprana es el producto de los *pesos* de las conexiones por esas derivadas. Si los pesos son mayores que uno, el producto no se desvanece, se dispara.

3. **Degradación del flujo de información.** Apilar demasiadas operaciones no lineales hace que las capas finales pierdan correlación directa con las características básicas de la entrada.

Ojito...

Nótese que 1 y 2 apuntan en direcciones opuestas. El producto de muchos factores casi nunca se queda cerca de uno: o se colapsa o se dispara. Ese es el problema real, y no se arregla escogiendo mejor la función de activación.

Por eso durante años la recomendación fue al revés de lo que hoy se hace: pocas capas ocultas con muchas neuronas, en lugar de muchas capas con pocas neuronas. Así estuvo hasta la aparición del aprendizaje profundo.

11. Redes residuales y autoencoders

24 de septiembre de 2026

Resumen rápido. Las dos respuestas al problema que quedó abierto. Primero ResNet, que le abre al gradiente un camino directo de regreso con una suma. Después los autoencoders, que entrenan sin etiquetas poniendo la entrada como objetivo, y sus dos variantes: el disperso cuando la capa oculta sobra, y el apilado, que arma una red honda entrenando una capa a la vez.

Redes residuales

Una red residual, o **ResNet**, introduce **conexiones de salto**, también llamadas atajos.

La idea es cambiar lo que se le pide aprender a un bloque de capas. En vez de obligarlo a aprender directamente la función $H(x)$, se le pide la **función residual**

$$F(x) = H(x) - x,$$

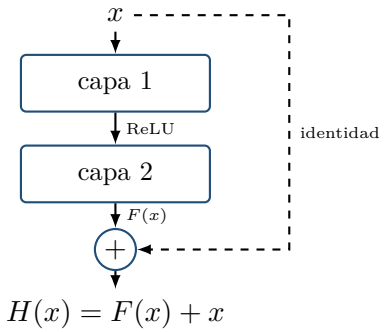
y entonces la salida que de verdad se busca queda

$$H(x) = F(x) + x.$$

Esa suma es la conexión de salto: la entrada x se brinca las capas intermedias y se suma directo a su salida.

Qué hay dentro del bloque

F no es una capa suelta, es un grupo de ellas. En la versión clásica son dos capas convolucionales con una ReLU en medio, y la suma va *antes* de la última ReLU:



Hay un detalle práctico que la fórmula esconde. Para poder sumar, $F(x)$ y x tienen que tener la misma forma, y eso deja de cumplirse en cuanto el bloque cambia el número de canales o reduce el tamaño espacial. Cuando pasa, al atajo se le mete una proyección lineal:

$$H(x) = F(x) + W_s\, x,$$

que en la práctica es una convolución de 1×1. Es el único lugar donde el atajo tiene parámetros; en todos los demás es literalmente un cable.

Por qué arregla el gradiente

Al aplicar backpropagation, si la salida es $H(x) = F(x) + x$, la derivada respecto a la entrada es

$$\frac{\partial H(x)}{\partial x} = \frac{\partial F(x)}{\partial x} + 1.$$

Ahí está todo. Siempre hay un término que vale exactamente 1 y que viaja limpio desde la última capa hasta la primera. Aunque las transformaciones del bloque lleven el gradiente a cero, o sea aunque $\partial F/\partial x \approx 0$, el $+1$ garantiza que lo que llega nunca baje de la señal original.

Comparado con la clase pasada, el contraste es exacto: sin el atajo el gradiente es un *producto* de factores menores que uno, y por eso se colapsa. Con el atajo es una *suma* en la que uno de los sumandos es uno.

Encadenando L bloques, con $x_{l+1} = x_l + F(x_l)$, la regla de la cadena da

$$\frac{\partial E}{\partial x_l} = \frac{\partial E}{\partial x_L} \prod_{i=l}^{L-1} \left(1 + \frac{\partial F(x_i)}{\partial x_i}\right).$$

Al desarrollar el producto, uno de los términos es el puro $\partial E/\partial x_L$, sin multiplicar por nada. Ese sumando sobrevive aunque todos los demás se anulen, y es la ruta directa de la que habla la diapositiva.

Ejemplo 11.1. Cincuenta capas donde cada una atenúa el gradiente a 0.8 de lo que recibe. Sin atajos:

$$0.8^{50} \approx 1.4 \times 10^{-5}.$$

Con atajos, cada factor es $1 + 0.8$ en vez de 0.8, y el término que solo acumula unos vale exactamente 1 sin importar cuántos bloques haya. El gradiente que llega a la primera capa es del mismo tamaño que el que salió de la última.

Aprender la identidad

El otro argumento es igual de importante. Si un bloque no aporta nada al problema, la red puede simplemente ajustar sus pesos para que $F(x) = 0$, y la salida queda $H(x) = x$, la identidad.

Idea clave

Eso explica la observación que originó el método. Una red honda debería ser al menos tan buena como una menos honda, porque siempre podría poner las capas de sobra en identidad. Y sin embargo las redes hondas simples salían *peores*. La conclusión de He y sus coautores fue que el problema no era la capacidad del modelo, era la optimización: aprender la identidad a través de una pila de no linealidades resulta difícil. Con el atajo, la identidad es el caso fácil, el que sale con los pesos en cero.

Antes de ResNet (2015), las redes de más de 20 capas sufrían degradación severa. Con conexiones residuales se volvió posible entrenar de forma estable redes de 152, 500 y hasta más de 1 000 capas. La referencia es He, Zhang, Ren y Sun (2016).

Autoencoders

Un **autoencoder** es un algoritmo de aprendizaje *no supervisado* que aplica backpropagation con un solo cambio: los valores objetivo se ponen iguales a las entradas.

$$t^p = x^p.$$

Con eso el error de la clase del 17 de septiembre se vuelve la distancia entre la entrada y su reconstrucción:

$$e^p = \tfrac{1}{2} \sum_i (x_i^p - \hat{x}_i^p)^2.$$

No hace falta etiquetar nada. El dato es su propia etiqueta, y por eso cuenta como no supervisado aunque el algoritmo de entrenamiento sea el mismo de siempre.

Las dos mitades, escritas

La red son dos funciones encadenadas. El **codificador** lleva la entrada al código, y el **decodificador** lo trae de vuelta:

$$h = \sigma(Wx + b), \qquad \hat{x} = \sigma(W'h + b').$$

A h se le llama el **código** o la representación latente, y a su dimensión el tamaño del cuello. Entrenar es minimizar $\|x - \hat{x}\|^2$ sobre W, W', b y b', con la misma backpropagation de siempre.

Una elección común es **amarrar los pesos**, o sea poner $W' = W^T$. Con eso el decodificador deja de tener parámetros propios, la red entrena la mitad de pesos y es más difícil que se memorice el conjunto en lugar de aprender de él.

Ojito...

El nombre de la capa oculta según su tamaño aparece en la literatura y conviene reconocerlo. Si el cuello es *menor* que la entrada, el autoencoder es **incompleto**; si es *mayor*, es **sobrecompleto**. Los dos aprenden, pero por razones distintas, y esa diferencia es justo la que separa esta sección de la que sigue.

La clave está en la capa oculta

Pedirle a una red que copie su entrada suena inútil, y lo sería si no fuera por el cuello de botella. Si la capa oculta tiene *menos* neuronas que las de entrada y salida, la información está obligada a pasar por ese embudo, y la red no tiene más remedio que encontrar una **representación intermedia** que quepa ahí y de la que se pueda reconstruir lo original.

Ya entrenada, la red se parte en dos:

Mitad	De dónde a dónde	Qué hace
Compresor, o codificador	De la entrada a la capa oculta	Produce la versión comprimida
Descompresor, o decodificador	De la capa oculta a la salida	Recupera la versión original a partir de la comprimida

Idea clave

Vale la pena ver con qué se emparenta esto. Un autoencoder *lineal* con un cuello de k neuronas y error cuadrático termina abarcando el mismo subespacio que las primeras k componentes principales, o sea lo mismo que daría una descomposición en valores singulares. Lo que lo vuelve más que eso es la no linealidad de las capas: con sigmoides o ReLU el codificador puede doblar el espacio, y encuentra estructura que una proyección lineal no alcanza.

Para qué se usan

El autoencoder casi nunca se entrena por su salida. Lo que se quiere es alguna de estas cuatro cosas:

- **Reducir la dimensión.** Quedarse con h y tirar el decodificador. Es el uso directo del cuello de botella.
- **Quitar ruido.** Entrenado con entradas sucias y salidas limpias, reconstruye la versión limpia de una entrada que nunca vio.

- **Detectar anomalías.** Aprende a reconstruir bien lo que se parece a sus datos de entrenamiento. Una entrada rara sale con error de reconstrucción alto, y ese error sirve directamente como puntaje de qué tan anómala es.
- **Preentrenar.** Usar los pesos del codificador como punto de arranque de una red supervisada, que es de lo que trata la última parte de la clase.

Autoencoders dispersos

Y si la capa oculta, en vez de tener menos neuronas, tiene *más* que las de entrada y salida?

El riesgo es que la red no aprenda nada: que copie la entrada a un subconjunto de las neuronas ocultas, deje las sobrantes sin usar, y las copie de vuelta a la salida. Reconstruye perfecto y no aprendió absolutamente nada.

La salida es forzar a las neuronas de la capa oculta a activarse *pocas veces*. Entonces la red tiene que volver a aprender un código alternativo, y además disperso. La idea es que los valores de las neuronas ocultas sean cero la mayor parte del tiempo.

Eso limita mucho la expresividad de la capa oculta, pero a cambio obliga a cada neurona a especializarse en cierto patrón de la entrada.

Dos maneras de conseguirlo

- **Autoencoder disperso.** Se agrega al error un factor de dispersión. Si la dispersión es baja, o sea si las neuronas se activan mucho, el factor suma bastante error; si es alta, suma poco. El castigo empuja a la red hacia el código disperso.
- **Autoencoder que quita ruido.** En lugar de dar el mismo valor en la entrada y en la salida, se le mete ruido al vector de entrada y se deja la salida limpia. La red queda obligada a generalizar, porque va a ver varios ejemplos de entrada ligeramente distintos que deben producir la misma salida.

El factor de dispersión no viene escrito en la diapositiva. La forma usual, la de las notas de Andrew Ng que están en las lecturas, mide la activación promedio $\hat{\rho}_j$ de cada neurona oculta contra un objetivo ρ chico, del orden de 0.05, y castiga la diferencia con

$$\beta \sum_j \left[\rho \log \frac{\rho}{\hat{\rho}_j} + (1-\rho) \log \frac{1-\rho}{1-\hat{\rho}_j} \right],$$

que vale cero cuando $\hat{\rho}_j = \rho$ y crece rápido en cuanto la neurona se activa más seguido de lo permitido.

Autoencoders apilados

Un **autoencoder apilado** es una red formada por varias capas de autoencoders dispersos, donde la salida de cada capa se conecta a la entrada de la siguiente. El entrenamiento va por etapas:

1. Entrenar un autoencoder disperso sobre las entradas crudas $x^{(k)}$, para aprender **características primarias** $h^{(1)(k)}$.
2. Usar esas características primarias como si fueran la entrada cruda de *otro* autoencoder disperso, y aprender **características secundarias** $h^{(2)(k)}$, que son más complejas que las primeras.

3. Tratar las secundarias como entrada cruda de un clasificador, y entrenarlo para mapear de ellas a las etiquetas de clase.

4. Juntar las tres piezas en una sola red con dos capas ocultas y una capa final de clasificación.

El método entrena los parámetros de cada capa por separado, mientras mantiene congelados los del resto del modelo. Al terminar esa fase se hace un **ajuste fino** con backpropagation sobre la red completa, para mejorar los resultados moviendo ya todos los parámetros a la vez.

Qué se gana

- **Aprende sin supervisión.** Solo necesita datos, y encuentra por su cuenta las características frecuentes con las cuales etiquetarlos. Las etiquetas hacen falta nada más para el clasificador del final.
- **Se puede ir tan hondo como se quiera.** El problema del error que se diluye no aparece, porque nunca se propaga a través de muchas capas a la vez: cada autoencoder tiene su propio objetivo y es poco profundo.

Idea clave

Las dos mitades de esta clase atacan el mismo problema por lados opuestos. El apilado lo *evita*: nunca propaga a lo largo de toda la red, así que el gradiente no tiene por dónde diluirse. ResNet lo *arregla*: deja que el gradiente recorra la red entera, pero le pone un camino donde no se encoge. Históricamente el preentrenamiento capa por capa fue lo que hizo viable el aprendizaje profundo a mediados de los dos mil. Dejó de usarse casi por completo cuando ReLU, mejores inicializaciones y más datos permitieron entrenar de punta a punta sin esa muleta.

Variante	Capa oculta	Qué la obliga a aprender
Básico	Más chica que la entrada	El cuello de botella: la información no cabe de otro modo
Disperso	Más grande que la entrada	El castigo por activarse seguido
Que quita ruido	De cualquier tamaño	El ruido en la entrada, que impide copiar
Apilado	Varias, una por etapa	Cada capa se entrena sola sobre la salida de la anterior

12. Clasificadores basados en distancia

6 de octubre de 2026

Resumen rápido. Una familia de clasificadores que no aprende pesos: guarda ejemplos con su clase y clasifica un dato nuevo según qué tan cerca queda de ellos. Para eso hace falta una distancia, así que primero se vio qué es un espacio métrico y las distancias más usadas. Luego las tres reglas, distancia mínima, vecino más cercano y k vecinos más cercanos, cómo son sus fronteras de decisión, y sus problemas: el costo de calcular tantas distancias y la maldición de la dimensionalidad.

La idea

Se supone que los datos se pueden representar en un espacio donde se puede medir, y que si dos objetos x y x' son parecidos, sus representaciones quedan cerca: $d(x, x')$ es chica. Las clases se representan con **prototipos**, puntos cuya clase ya se conoce.

Con eso, clasificar solo pide dos cosas: la distancia y una regla que diga cómo usarla, la *modalidad*. No hay entrenamiento en el sentido de la red neuronal; el modelo son los prototipos mismos.

Espacio métrico

Definición 12.1 (Espacio métrico)**.** Un par (E, d) formado por un conjunto de puntos E y una función $d : E \times E \to \mathbb{R}$, llamada **métrica** o **distancia**, que para todos $p, q, r \in E$ cumple:

1. **Positiva o nula**: $d(p, q) \geq 0$, y $d(p, q) = 0$ si y solo si $p = q$.
2. **Simétrica**: $d(p, q) = d(q, p)$.
3. **Desigualdad del triángulo**: $d(p, q) + d(q, r) \geq d(p, r)$.

Cada propiedad tiene su razón. La primera dice que solo un punto está a distancia cero de sí mismo: dos objetos distintos siempre se pueden distinguir. La segunda, que no importa desde dónde se mida. La tercera, que no hay atajos: ir de p a r pasando por q nunca sale más corto que ir directo. Esa última es la que permite podar búsquedas de vecinos, como se ve más abajo, y por eso importa que la función que se use las cumpla.

Si no se cumple alguna, se habla de un **espacio pseudométrico**: la versión "simple", donde no se pide simetría ni desigualdad del triángulo. En ese caso d se llama **medida de disimilitud**. En textos de matemáticas, "pseudométrica" se reserva para algo más específico, una métrica que permite $d(p, q) = 0$ con $p \neq q$; en el curso se usa en el sentido amplio.

Tampoco toda disimilitud sale de una distancia. Muchas se construyen a partir de una similitud, y el resultado no tiene por qué ser métrica.

Ejemplo 12.2 ($1 - \cos$ no es una métrica)**.** El coseno mide similitud, y $1 - \cos\theta$ es una disimilitud natural. Con tres vectores a 0°, 45° y 90°:

$$d(0°, 90°) = 1 - \cos 90° = 1, \qquad d(0°, 45°) = d(45°, 90°) = 1 - \cos 45° = 0.293.$$

Pasando por el de 45° se llega en $0.293 + 0.293 = 0.586$, menos que el 1 del camino directo: viola la desigualdad del triángulo. Sí es simétrica, así que es una disimilitud pero no una distancia. La divergencia de Kullback-Leibler entre distribuciones es el caso contrario: ni siquiera es simétrica.

Todo espacio vectorial con producto punto se vuelve espacio métrico con la **distancia euclidiana**:

$$d(\vec{p}, \vec{q}) = \|\vec{p} - \vec{q}\| = \sqrt{(\vec{p} - \vec{q})^T(\vec{p} - \vec{q})} = \sqrt{(p_1 - q_1)^2 + \cdots + (p_d - q_d)^2}.$$

Distancias usuales

De la euclidiana salen varias. Para vectores de d dimensiones:

Familia L_p. Para $p \geq 1$,

$$d_p(\vec{a}, \vec{b}) = \left(\sum_{i=1}^{d} |a_i - b_i|^p \right)^{1/p}.$$

Los tres casos que se usan:

- L_1, **Manhattan**: $d(\vec{a}, \vec{b}) = \sum_{i=1}^{d} |a_i - b_i|$. Suma las diferencias coordenada por coordenada, como caminar por las cuadras de una ciudad sin cruzar en diagonal.
- L_2, **euclidiana**: $d(\vec{a}, \vec{b}) = \left(\sum_{i=1}^{d} (a_i - b_i)^2 \right)^{1/2}$. La línea recta.
- L_∞, **Chebyshev**: $d(\vec{a}, \vec{b}) = \max_{1 \leq i \leq d} |a_i - b_i|$. Es el límite de L_p cuando $p \to \infty$: al elevar a una potencia enorme, la diferencia más grande se come a todas las demás. Es la distancia del rey en el ajedrez.

Euclidiana pesada. Con pesos $w_i \geq 0$ que dicen cuánto importa cada característica:

$$d(\vec{a}, \vec{b}) = \left(\sum_{i=1}^{d} w_i \, (a_i - b_i)^2 \right)^{1/2}.$$

Mahalanobis. Divide cada diferencia entre la desviación estándar σ_i de esa característica:

$$d(\vec{a}, \vec{b}) = \left(\sum_{i=1}^{d} \left(\frac{a_i - b_i}{\sigma_i} \right)^2 \right)^{1/2}.$$

Es el caso particular de una euclidiana pesada con $w_i = 1/\sigma_i^2$. La versión general usa la matriz de covarianza Σ completa, $\sqrt{(\vec{a} - \vec{b})^T \, \Sigma^{-1} \, (\vec{a} - \vec{b})}$, que además descuenta las correlaciones entre características; con Σ diagonal se reduce a la de arriba.

Ejemplo 12.3 (Por qué escalar)**.** Dos personas difieren en 20 años de edad y en 1000 pesos de ingreso. La euclidiana da $\sqrt{20^2 + 1000^2} = 1000.2$: la edad casi no cuenta, solo porque el ingreso se mide en números más grandes. Si la edad tiene $\sigma = 10$ y el ingreso $\sigma = 5000$, Mahalanobis da

$$\sqrt{\left(\tfrac{20}{10}\right)^2 + \left(\tfrac{1000}{5000}\right)^2} = \sqrt{4 + 0.04} = 2.01,$$

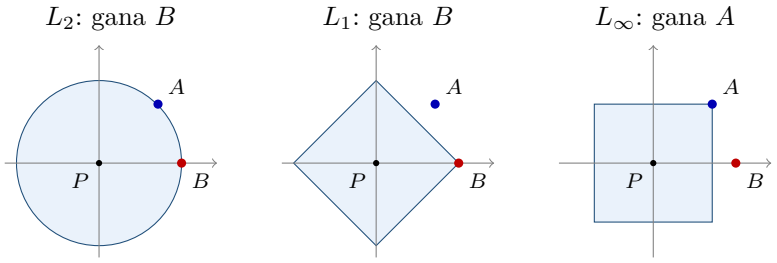
y ahora lo que separa a las dos personas es la edad: 20 años son dos desviaciones estándar, 1000 pesos apenas 0.2.

La métrica cambia al vecino

Distancias distintas dan vecindarios distintos (diapositiva 6). El conjunto de puntos a distancia menor que r de un centro es un círculo con L_2, un rombo con L_1 y un cuadrado con L_∞. Eso cambia quién es el vecino más cercano.

Ejemplo 12.4 (Un punto, tres métricas)**.** Se quiere clasificar $P = (0, 0)$ y hay dos prototipos: $A = (1, 1)$, de la clase azul, y $B = (1.4,\ 0)$, de la roja.

Métrica	$d(P, A)$	$d(P, B)$	**Vecino más cercano**
L_2	$\sqrt{2} = 1.414$	1.4	B, rojo
L_1	$1 + 1 = 2$	1.4	B, rojo
L_∞	$\max(1, 1) = 1$	1.4	A, azul



Cada figura dibuja la bola de radio igual a la distancia al vecino más cercano, que queda en la orilla; el otro prototipo queda fuera. A reparte su diferencia en dos coordenadas de 1, y B la concentra en una de 1.4. L_1 suma las dos de A y la castiga; L_∞ solo mira la mayor y la premia. La clasificación de P cambia sin que se mueva ningún punto.

Reglas de clasificación

Sea d una distancia y C el número de clases. Las tres modalidades de la clase:

Distancia mínima (MD)

Cada clase i se representa con un **único** prototipo p_i, normalmente la media de sus ejemplos. Un dato nuevo va a la clase del prototipo más cercano:

$$x \in w \iff d(x, p_w) < d(x, p_i) \quad \text{para todo } 1 \le i \le C,\ i \ne w.$$

Es lo más económico: se guardan C puntos y se calculan C distancias. El problema es que un solo punto resume toda la clase. Si la clase está muy extendida, tiene varios grupos separados, o un ejemplo raro jala su media, el prototipo deja de representarla.

Vecino más cercano (NN)

Cada clase i se representa con un **conjunto** de prototipos P_i, normalmente todos sus ejemplos de entrenamiento. El dato va a la clase que tenga el prototipo más cercano:

$$x \in w \iff \exists\, p \in P_w :\ d(x, p) \le d(x, p') \quad \forall\, p' \in P_i,\ 1 \le i \le C,\ i \ne w.$$

Si hay empate entre clases, se elige la que tenga más prototipos.

k vecinos más cercanos (k-NN)

En lugar del vecino más cercano se toman los k más cercanos, sin importar su clase. Sea $P = \bigcup_{i=1}^{C} P_i$ el conjunto de todos los prototipos y $P^k \subseteq P$ los k más cercanos a x. Gana la clase con más representantes entre ellos:

$$x \in w \iff \left|P^k \cap P_w\right| \ge \left|P^k \cap P_i\right| \quad 1 \le i \le C,\ i \ne w.$$

Si hay empate, se usa 1-NN entre las clases empatadas. Con $k > 1$ un ejemplo mal etiquetado ya no decide solo: tiene que convencer a la mayoría.

Ejemplo 12.5 (Tres reglas, tres respuestas)**.** Se clasifica $x = (3, 3)$ con dos clases:

$$A = \{(3,5),\ (3,0),\ (4,2)\}, \qquad B = \{(2,1),\ (4,3),\ (6,3)\}.$$

MD. Las medias son $\bar{A} = (10/3,\ 7/3)$ y $\bar{B} = (4,\ 7/3)$, así que

$$d(x, \bar{A}) = \sqrt{(1/3)^2 + (2/3)^2} = 0.745, \qquad d(x, \bar{B}) = \sqrt{1^2 + (2/3)^2} = 1.202.$$

Gana A.

NN y k-NN. Las distancias euclidianas de x a cada prototipo, de menor a mayor:

Prototipo	**Clase**	**Distancia**
$(4,3)$	B	1
$(4,2)$	A	$\sqrt{2} = 1.414$
$(3,5)$	A	2
$(2,1)$	B	$\sqrt{5} = 2.236$
$(3,0)$	A	3
$(6,3)$	B	3

Con 1-NN gana B, por $(4,3)$. Con 3-NN los tres más cercanos son uno de B y dos de A: gana A.

El mismo punto, con los mismos datos, cambia de clase según la regla. MD mira el centro de cada clase; 1-NN se deja llevar por un solo punto; 3-NN vota.

La diapositiva 11 muestra lo mismo con tres clases: MD elige B porque b_5 es el prototipo más cercano; NN elige A por a_7; y 7-NN elige C, porque entre los siete más cercanos $\{c_1, c_2, c_3, a_7, a_6, b_3, b_7\}$ hay tres de C y dos de cada una de las otras.

Fronteras de decisión

Distancia mínima: un hiperplano

Con la métrica euclidiana en $\mathbb{R}^d$, la frontera entre las clases j y k son los puntos $\vec{y}$ que quedan a la misma distancia de sus dos prototipos:

$$\begin{aligned} \|\vec{y} - \vec{p}_j\|^2 &= \|\vec{y} - \vec{p}_k\|^2 \\ \vec{y}^T\vec{y} - 2\vec{p}_j^T\vec{y} + \vec{p}_j^T\vec{p}_j &= \vec{y}^T\vec{y} - 2\vec{p}_k^T\vec{y} + \vec{p}_k^T\vec{p}_k \\ 2(\vec{p}_k - \vec{p}_j)^T\vec{y} + \vec{p}_j^T\vec{p}_j - \vec{p}_k^T\vec{p}_k &= 0. \end{aligned}$$

El término $\vec{y}^T\vec{y}$ se cancela, y lo que queda es lineal en $\vec{y}$:

$$\sum_{i=1}^{d} a_i y_i + a_0 = 0, \qquad a_i = 2(p_{k,i} - p_{j,i}), \quad a_0 = \vec{p}_j^T\vec{p}_j - \vec{p}_k^T\vec{p}_k.$$

Es un **hiperplano**, la mediatriz del segmento entre los dos prototipos: la misma clase de frontera que dan el perceptrón o una SVM lineal.

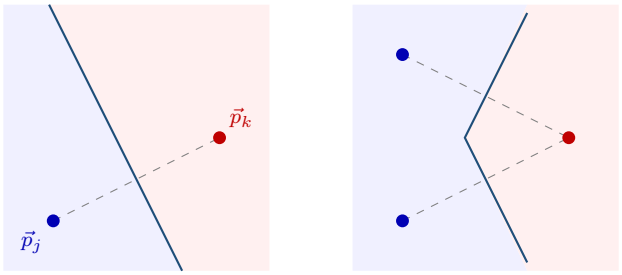
Vecino más cercano: lineal por tramos

Para NN la función discriminante de la clase j es

$$g_j(\vec{y}) = \max_{p \in P_j} (-d(\vec{y}, \vec{p})) \equiv \max_{p \in P_j} \left(- \|\vec{y} - \vec{p}\|^2 \right) \equiv \max_{p \in P_j} \left(2\vec{p}^T\vec{y} - \vec{p}^T\vec{p} \right),$$

el máximo de varias funciones lineales. La frontera queda **lineal por tramos**: cada tramo es la mediatriz entre un prototipo de una clase y uno de otra.

(a) MD: un hiperplano (b) 1-NN: lineal por tramos



En (b) hay dos prototipos azules, $(0,0)$ y $(0,2)$, y uno rojo, $(2,1)$. La frontera es la mediatriz con $(0,0)$, la recta $4y_1 + 2y_2 = 5$, hasta el punto $(0.75,\ 1)$, que está a la misma distancia de los tres; de ahí en adelante sigue la mediatriz con $(0,2)$, la recta $4y_1 - 2y_2 = 1$. La mediatriz entre los dos azules no se dibuja: separa puntos de la misma clase.

Para trazar a mano la frontera de 1-NN (diapositiva 15):

1. Ubicar la zona donde deberían estar las fronteras.
2. Unir con un segmento pares de puntos de clases opuestas que estén cerca.
3. Trazar la mediatriz de cada segmento.
4. Extender y unir las mediatrices, y borrar lo que sobre.
5. Llevar las fronteras hasta la orilla de la gráfica y marcarlo con flechas.
6. Revisar: una frontera de 1-NN tiene picos y esquinas; si sale curva, algo está mal.

k-NN

Su función discriminante es simplemente el número de votos, $g_j(\vec{y}) = \left|P^k \cap P_j\right|$, con P^k los k vecinos de $\vec{y}$ según d. La frontera también es lineal por tramos.

Las fronteras cambian con la **distancia**: con L_1 los tramos se vuelven horizontales, verticales o a 45° (diapositiva 18). Y cambian con el **número de vecinos**: con $k = 1$ la frontera rodea cada punto y se pliega alrededor de los aislados; con $k = 3$ sale más suave, porque un punto suelto ya no alcanza a ganar la votación de su zona (diapositiva 19). Un k demasiado grande se va al otro extremo: las clases chicas pierden todas las votaciones.

Variante: vecinos en una bola de radio ε

Mismo principio que k-NN, pero cambia quién vota. En lugar de los k más cercanos votan todos los ejemplos x' con $d(x, x') \leq \varepsilon$. En zonas densas vota mucha gente y en zonas vacías poca, así que se adapta a la densidad de los datos. A cambio, en una zona vacía la bola puede quedar sin nadie y hay que decidir qué hacer, y un buen ε depende de la escala de las características.

Limitaciones

El costo

Clasificar un dato pide calcular su distancia a los n prototipos, cada una de d coordenadas. No hay costo de entrenamiento, pero todo el costo se paga al clasificar, y con un conjunto grande puede ser prohibitivo. Se ataca de tres formas:

- **Reducir la dimensión** con técnicas como PCA, para que cada distancia cueste menos.
- **Estructuras de búsqueda**, como árboles, que descartan prototipos sin calcular su distancia aprovechando la desigualdad del triángulo.
- **Editar el conjunto de entrenamiento**: quitar puntos redundantes o casi redundantes, y también el ruido. Una forma de saber si un punto es ruido es mirar a sus vecinos: si la mayoría es de otra clase, probablemente está mal etiquetado. Es la edición de Wilson (1972).

Ejemplo 12.6 (Podar con la desigualdad del triángulo)**.** Se busca el vecino más cercano de q. Ya se calculó $d(q,p) = 10$ contra un prototipo de referencia p, y desde antes se tiene guardada $d(p,x) = 3$ para otro prototipo x. Por la desigualdad del triángulo,

$$d(q,x) \geq |d(q,p) - d(p,x)| = |10 - 3| = 7.$$

Si el mejor candidato hasta ahora está a distancia 5, x no puede ganarle, y se descarta sin calcular $d(q,x)$. Con una disimilitud que no cumple la desigualdad del triángulo esta poda sería incorrecta: podría tirar al vecino verdadero.

La maldición de la dimensionalidad

Cuando la distancia se calcula con muchas coordenadas, la diferencia entre las distancias de distintos pares de puntos se vuelve chica. Cada coordenada aporta un término a la suma, así que cambiar una sola de d variables mueve poco el total; y como todas aportan algo, todos los puntos terminan casi a la misma distancia de todos. El problema de fondo es que el volumen del espacio crece tan rápido con la dimensión que los datos disponibles quedan dispersos.

Ejemplo 12.7 (Los vecinos dejan de estar cerca)**.** En una simulación con 1000 puntos al azar en el cubo $[0,1]^d$, la distancia de un punto nuevo a su vecino más cercano y a su vecino más lejano fue:

d	Más cercano	Más lejano	Cociente
2	0.010	1.334	135.6
10	0.471	2.111	4.5
100	3.349	4.653	1.4
1000	12.166	13.553	1.1

En 2 dimensiones el más lejano está 135 veces más lejos que el más cercano. En 1000, solo 1.1 veces: el "vecino más cercano" está casi tan lejos como cualquier otro, y la regla pierde sentido.

Otra forma de verlo: para que un cubito alrededor de un punto atrape el 10 % de datos uniformes, su lado tiene que ser $0.1^{1/d}$ del rango. En $d = 1$ eso es 0.1; en $d = 10$ es 0.79; en $d = 100$, 0.98. El vecindario "local" ya abarca casi todo el rango de cada variable.

k-NN rápido

Hay toda una línea de trabajo para acelerar la búsqueda de vecinos (diapositivas 23 y 24):

- **Distancia parcial**: se va sumando coordenada por coordenada y se abandona en cuanto la suma parcial ya supera a la mejor distancia encontrada.
- **Preordenar el conjunto de entrenamiento**, para que los candidatos buenos salgan primero y la poda funcione antes.
- **Proyección de prototipos**: buscar primero en una o pocas coordenadas y solo calcular la distancia completa para los que pasan ese filtro.
- **Árboles y grafos**, que organizan los prototipos para no recorrerlos todos; varios métodos recientes son aproximados: aceptan no encontrar siempre al vecino exacto a cambio de mucha velocidad.
- **Aproximación y eliminación**: ramificación y acotamiento en espacios métricos, que descartan grupos enteros de prototipos con cotas como la del ejemplo de la poda.

Datos no numéricos

Las distancias de arriba piden números. Si los datos son categóricos hay que transformarlos a números, por ejemplo con codificación one-hot, o usar una disimilitud pensada para ellos, como contar en cuántos atributos difieren dos objetos.

Idea clave

Pese a su edad, el k-NN sigue vigente: es la forma de encontrar los documentos más parecidos a uno dado. Recuperar los documentos de mayor coseno con una consulta, como en las actividades de Fundamentos para el Análisis Automático de Lenguaje, es una búsqueda de vecinos más cercanos. Los buscadores sobre embeddings usan justo los métodos aproximados basados en grafos de la diapositiva 23 para hacerlo sobre millones de vectores.

13. Clasificador Naive Bayes

8 de octubre de 2026

Resumen rápido. Un clasificador probabilístico. Para un dato nuevo calcula la probabilidad de cada clase dado ese dato, con el teorema de Bayes, y elige la más probable. Es "ingenuo" porque supone que las características son condicionalmente independientes dada la clase; esa suposición casi nunca es cierta, pero vuelve el cálculo lineal y deja estimar el modelo con pocos datos, aun en alta dimensión. Se vio el problema de las probabilidades en cero, el suavizado de Laplace, y la versión gaussiana para variables continuas.

La intuición

Se quiere distinguir saltamontes (*grasshoppers*) de *katydids*, insectos parecidos pero de antenas largas, por el largo de sus antenas (diapositivas 4 a 11). Con muchos datos se arma un histograma de largos para cada especie, y se resume con una curva normal para cada una. Para un insecto nuevo con antenas de largo 3, se compara la altura de las dos curvas en $x = 3$: la de saltamontes mide 10 y la de katydid 2, así que

$$P(\text{saltamontes} \mid 3) = \frac{10}{10+2} = 0.833, \qquad P(\text{katydid} \mid 3) = \frac{2}{10+2} = 0.167.$$

Con antenas de 7 las alturas son 3 y 9, y gana katydid con 0.75. En 5 las curvas se cruzan y queda 0.5 contra 0.5.

Esa es toda la idea: calcular la probabilidad de que el dato nuevo pertenezca a cada clase y quedarse con la más probable.

El teorema de Bayes

$$p(c_j \mid d) = \frac{p(d \mid c_j)\, p(c_j)}{p(d)}$$

- $p(c_j \mid d)$, la **probabilidad a posteriori**: que el dato d sea de la clase c_j. Es lo que se quiere calcular.
- $p(d \mid c_j)$, la **verosimilitud**: la probabilidad de generar el dato d si se es de la clase c_j. Se puede pensar como que pertenecer a la clase causa tener esa característica con cierta probabilidad.
- $p(c_j)$, la **probabilidad a priori**: qué tan frecuente es la clase en los datos, antes de ver d.
- $p(d)$, la **evidencia**: la probabilidad de observar d. Es la misma para todas las clases, así que para decidir cuál gana se puede ignorar.

Ejemplo 13.1 (La oficial Drew)**.** "Drew" es nombre de hombre y de mujer. Con una base de 8 personas, 3 hombres y 5 mujeres, donde se llaman Drew un hombre y dos mujeres:

$$p(\text{hombre} \mid \text{drew}) \propto p(\text{drew} \mid \text{hombre})\, p(\text{hombre}) = \frac{1}{3} \cdot \frac{3}{8} = 0.125,$$

$$p(\text{mujer} \mid \text{drew}) \propto p(\text{drew} \mid \text{mujer})\, p(\text{mujer}) = \frac{2}{5} \cdot \frac{5}{8} = 0.250.$$

Gana mujer. Si se quieren las probabilidades de verdad se divide entre $p(\text{drew}) = 3/8$, y quedan $1/3$ y $2/3$; para decidir no hace falta.

Varias características: la suposición ingenua

Con un vector de características $x = (x^{(1)}, \ldots, x^{(n)})$ hace falta $P(x^{(1)}, \ldots, x^{(n)} \mid y)$. Por la definición de probabilidad condicional aplicada en cadena,

$$P(x^{(1)}, \ldots, x^{(n)} \mid y) = P(x^{(1)} \mid y)\, P(x^{(2)} \mid y, x^{(1)}) \cdots P(x^{(n)} \mid y, x^{(1)}, \ldots, x^{(n-1)}),$$

y el último factor depende de todas las demás características a la vez. Con n características binarias la tabla completa tiene $2^n - 1$ probabilidades por clase: con $n = 30$ son más de mil millones, y no hay datos para estimarlas.

En un filtro de spam cada correo es un vector con una coordenada por palabra del vocabulario, 1 si la palabra aparece y 0 si no (diapositiva 20). Con 50 000 palabras se vive en un espacio de 50 000 dimensiones, y la tabla completa es impensable.

Definición 13.2 (Suposición de Naive Bayes)**.** Las características son **condicionalmente independientes dada la clase**:

$$P(x^{(1)}, \ldots, x^{(n)} \mid y) = \prod_{j=1}^{n} P(x^{(j)} \mid y).$$

Por eso es "ingenuo". Ojo con lo que dice y lo que no: no dice que las características sean independientes. En el filtro de spam, *buy* y *price* aparecen juntas muy seguido, porque las dos son típicas del spam. Lo que se supone es que, *sabiendo ya que el correo es spam*, ver *buy* no cambia lo que se cree sobre *price*.

Juntando con Bayes y quitando la evidencia, que no depende de la clase, el clasificador queda

$$y_{NB} = \arg\max_{\tilde{y}} \; P(Y = \tilde{y}) \prod_{j=1}^{n} P\Big(X^{(j)} = x^{(j)}_{\text{test}} \mid Y = \tilde{y}\Big)\,.$$

Solo hacen falta $P(y)$ y cada $P(x^{(j)} \mid y)$ por separado, que se estiman contando frecuencias en los datos. En lugar de $2^n - 1$ probabilidades por clase son n, y clasificar cuesta una multiplicación por característica: el cálculo se vuelve lineal. Con tan pocos parámetros el modelo no se ajusta de más a los datos de entrenamiento, y por eso generaliza bien aun con pocos ejemplos.

Ejemplo 13.3 (Tres características)**.** Con la tabla de la diapositiva 18, sin usar el nombre, se clasifica a alguien que **no** mide más de 170 cm, tiene ojos **azules** y cabello **largo**. De los 3 hombres, 1 no pasa de 170, 2 tienen ojos azules y 1 cabello largo; de las 5 mujeres, 3, 3 y 4:

$$\text{hombre}: \quad \frac{3}{8} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{1}{36} = 0.028,$$

$$\text{mujer}: \quad \frac{5}{8} \cdot \frac{3}{5} \cdot \frac{3}{5} \cdot \frac{4}{5} = \frac{9}{50} = 0.180.$$

Gana mujer, con probabilidad $0.180/(0.180 + 0.028) = 0.87$.

Probabilidades en cero

Si una característica nunca tomó cierto valor dentro de una clase en los datos de entrenamiento, su frecuencia estimada es 0, y como todo se multiplica, esa clase queda con probabilidad 0 sin importar lo que digan las demás características. Con muchas dimensiones y pocos datos pasa todo el tiempo: basta con que un correo traiga una palabra que nunca apareció en un spam de entrenamiento.

Ejemplo 13.4 (Un cero lo decide todo)**.** Con la misma tabla, ahora usando también el nombre, se clasifica a alguien llamado Sergio, de más de 170 cm, ojos cafés y cabello largo. Ninguna mujer de la tabla se llama Sergio, así que $P(\text{sergio} \mid \text{mujer}) = 0$ y

$$\text{mujer}: \quad \frac{5}{8} \cdot 0 \cdot \frac{2}{5} \cdot \frac{2}{5} \cdot \frac{4}{5} = 0, \qquad \text{hombre}: \quad \frac{3}{8} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} = 0.009.$$

Hombre con probabilidad 1, aunque el cabello largo apunte a mujer: el cero borró todo lo demás.

Suavizado de Laplace

En lugar de dejar el cero, se le asigna a cada valor una probabilidad chica positiva. El **suavizado de Laplace** hace como si hubiera K ejemplos imaginarios repartidos parejo entre los K valores posibles de la característica:

$$\hat{P}\Big(X^{(j)} = x^{(j)}_{\text{test}} \mid Y = \tilde{y}\Big) = \frac{\sum_i \mathbf{1}\Big[x^{(j)}_i = x^{(j)}_{\text{test}},\ y_i = \tilde{y}\Big] + 1}{\sum_i \mathbf{1}[y_i = \tilde{y}] + K}.$$

Al conteo se le suma 1 y al total K. Las estimaciones se jalan hacia $1/K$, el valor que tendrían sin datos, y el jalón pesa menos entre más ejemplos tenga la clase.

Ejemplo 13.5 (Sergio, con suavizado)**.** El nombre toma $K = 6$ valores distintos y las otras tres características, $K = 2$. Para mujer, $\hat{P}(\text{sergio} \mid \text{mujer}) = (0+1)/(5+6) = 1/11$, y con el resto de factores suavizados igual:

$$\text{mujer}: \quad \frac{5}{8} \cdot \frac{1}{11} \cdot \frac{3}{7} \cdot \frac{3}{7} \cdot \frac{5}{7} = 0.0075, \qquad \text{hombre}: \quad \frac{3}{8} \cdot \frac{2}{9} \cdot \frac{3}{5} \cdot \frac{2}{5} \cdot \frac{2}{5} = 0.0080.$$

Sigue ganando hombre, pero 0.52 contra 0.48: el nombre ya no decide solo y las demás características vuelven a contar.

Otras técnicas para el mismo problema:

- **Suavizado aditivo** o de Lidstone: sumar un α entre 0 y 1 en lugar de 1, y αK al total. Laplace es el caso $\alpha = 1$; un α más chico altera menos las frecuencias reales.
- **Trabajar con logaritmos**. No resuelve los ceros, pero sí el problema hermano: multiplicar miles de probabilidades chicas da un número que la computadora redondea a 0. Se compara $\log P(y) + \sum_j \log P(x^{(j)} \mid y)$, que da la misma clase ganadora.

Por qué funciona

- **Es rápido y ocupa poco**: todas las probabilidades salen de una sola pasada por los datos y caben en una tabla chica.
- **Funciona en alta dimensión**, justo donde otros métodos sufren la maldición de la dimensionalidad: cada probabilidad se estima con una sola característica, así que los datos alcanzan.
- **No es sensible a características irrelevantes** (diapositiva 27).

Ejemplo 13.6 (Una característica que no dice nada)**.** Para clasificar a Jessica se usan varias características, entre ellas el color de ojos, que no tiene nada que ver con el sexo:

$$p(\text{ojos cafés} \mid \text{mujer}) = \frac{9000}{10\,000}, \qquad p(\text{ojos cafés} \mid \text{hombre}) = \frac{9001}{10\,000}.$$

El factor es casi el mismo en las dos clases, así que multiplica a las dos por casi lo mismo y no cambia cuál gana. Lo que decide es una característica relevante, como usar vestido: 9975/10 000 contra 2/10 000.

Ojito...

Una característica irrelevante no estorba si hay datos suficientes para estimarla: su factor sale casi igual en todas las clases. Con pocos datos el ruido de estimación sí puede inclinar un poco la balanza. Lo que de verdad lo daña son las características **redundantes**: si dos características dicen lo mismo, el modelo las cree independientes y cuenta esa evidencia dos veces.

Variables continuas: Naive Bayes gaussiano

Lo de arriba es para características discretas, donde $P(x^{(j)} \mid y)$ se estima contando. Con características continuas hay que suponer una distribución para los valores de cada una. Lo más común es suponer que, dentro de cada clase, cada característica sigue una **distribución normal**:

$$P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right),$$

donde μ_y y σ_y^2 son la media y la varianza de esa característica entre los ejemplos de la clase y. Entrenar es solo calcular una media y una varianza por clase y por característica. Estrictamente esto es una densidad y no una probabilidad, pero como todas las clases se evalúan en el mismo punto, la comparación sigue siendo válida.

Ejemplo 13.7 (Las antenas, con números)**.** Si los saltamontes tienen antenas con $\mu = 3$ y $\sigma = 1$, y los katydid con $\mu = 7$ y $\sigma = 1.5$, con las dos clases igual de frecuentes, para un insecto con antenas de 5:

$$P(5 \mid \text{saltamontes}) = \frac{1}{\sqrt{2\pi}}\, e^{-(5-3)^2/2} = 0.054, \qquad P(5 \mid \text{katydid}) = \frac{1}{1.5\sqrt{2\pi}}\, e^{-(5-7)^2/4.5} = 0.109.$$

Gana katydid, con $0.109/(0.054 + 0.109) = 0.67$, aunque 5 está justo a la mitad de las dos medias: la curva de katydid es más ancha y llega más lejos. La frontera queda en $x \approx 4.75$, no en 5. Con varianzas distintas la frontera ya no es lineal sino cuadrática, y tiene un segundo corte en $x \approx -5.15$, en la cola izquierda.

El teorema central del límite

En clase se justificó la normal con el teorema central del límite.

Teorema 13.8 (Teorema central del límite)**.** *Sean $X_1, \ldots, X_n$ variables aleatorias independientes e idénticamente distribuidas, con media μ y varianza σ^2 finitas, y sea $\bar{X}_n$ su promedio. Conforme n crece,*

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}}$$

tiende a una distribución normal estándar, sin importar la distribución original de las X_i.

Muchas mediciones naturales son el resultado de sumar muchos efectos chicos e independientes: la estatura, por ejemplo, depende de muchos genes y factores del ambiente. Por eso tantas características salen aproximadamente normales, y la suposición gaussiana suele funcionar.

Ojito...

El teorema habla de sumas y promedios, no de cualquier característica. Una variable muy sesgada, como el ingreso o un conteo que casi siempre vale 0, no se vuelve normal por el teorema, y ahí Naive Bayes gaussiano estima mal. Se puede transformar la variable, por ejemplo con logaritmo, partirla en intervalos y tratarla como discreta, o usar otra distribución.

Según el tipo de características hay tres variantes comunes: la **gaussiana**, para continuas; la de **Bernoulli**, para presencia o ausencia, como el vector del filtro de spam; y la **multinomial**, para conteos, como cuántas veces aparece cada palabra en un documento.

14. Medidas de desempeño

15 de octubre de 2026

Resumen rápido. Cómo saber qué tan bueno es un clasificador. Calidad depende de lo que se quiera medir, porque no todos los errores cuestan lo mismo. Hay dos familias de métricas: las basadas en la matriz de confusión, que evalúan al clasificador con un umbral de decisión fijo, y las basadas en la curva ROC (*receiver operating characteristic*), que lo evalúan en todos los umbrales a la vez. Se vio la matriz de confusión y las métricas que salen de ella: accuracy, precisión, recall y especificidad. Aquí se agregan el F1, el intercambio entre precisión y recall al mover el umbral, el caso multiclase y un adelanto de la curva ROC.

Qué significa calidad

"El clasificador es bueno" no dice nada hasta decir bueno para qué. Un clasificador se equivoca de dos formas, y casi nunca cuestan lo mismo:

- Un filtro de spam que manda a la basura un correo legítimo puede hacer que se pierda una oferta de trabajo. Uno que deja pasar un spam solo hace perder unos segundos.
- Una prueba médica que no detecta a un enfermo lo manda a su casa sin tratamiento. Una que da positivo a un sano lo manda a hacerse más estudios.

Por eso no hay una sola medida de calidad. Cada métrica contesta una pregunta distinta, y se escoge la que corresponde a lo que importa en el problema.

Las métricas se agrupan en dos familias:

- **Basadas en la matriz de confusión.** El clasificador ya tomó una decisión para cada ejemplo, con un umbral fijo, y se cuentan aciertos y errores de cada tipo.
- **Basadas en la curva ROC.** Muchos clasificadores dan un puntaje, como la probabilidad de Naive Bayes, y la decisión depende de dónde se ponga el umbral. La curva ROC muestra el desempeño en todos los umbrales a la vez.

Matriz de confusión

En un problema de dos clases se llama **positiva** a la clase que interesa detectar, el spam o el enfermo, y **negativa** a la otra. Cada ejemplo de prueba cae en una de cuatro casillas según su clase real y la que predijo el clasificador:

	predice positivo	**predice negativo**
real positivo	**VP** verdadero positivo	**FN** falso negativo
real negativo	**FP** falso positivo	**VN** verdadero negativo

- **Verdadero positivo** (VP): era positivo y lo detectó.
- **Falso negativo** (FN): era positivo y se le escapó. Es un error de omisión, o error tipo II.
- **Falso positivo** (FP): era negativo y lo marcó como positivo. Es una falsa alarma, o error tipo I.

- **Verdadero negativo** (VN): era negativo y lo dejó pasar.

La diagonal son los aciertos y fuera de ella los errores. Todas las métricas de esta familia son cocientes entre estas cuatro cuentas.

Ojito...

No hay una convención única para la orientación de la tabla. Aquí los renglones son la clase real y las columnas la predicha, como en scikit-learn; en otros libros va al revés. Antes de leer una matriz de confusión hay que revisar cuál es cuál.

Ejemplo 14.1 (Un filtro de spam)**.** Se prueba un filtro con 1000 correos que ya estaban etiquetados: 100 son spam y 900 legítimos.

	Predice spam	**Predice legítimo**	**Total**
Spam real	VP = 80	FN = 20	100
Legítimo real	FP = 30	VN = 870	900
Total	110	890	1000

Detectó 80 de los 100 spam y dejó pasar 20. De los 900 legítimos, mandó 30 a la basura por error. Las secciones que siguen sacan sus métricas de esta tabla.

Ojito...

La matriz se arma con ejemplos que el clasificador **no vio** al entrenar. Evaluado sobre sus propios datos de entrenamiento, un clasificador que memoriza saca métricas perfectas y no dice nada de cómo le irá con datos nuevos.

Accuracy

Definición 14.2 (Accuracy)**.** La fracción de ejemplos clasificados correctamente:

$$\text{accuracy} = \frac{VP + VN}{VP + VN + FP + FN}.$$

Su complemento, $1 - \text{accuracy}$, es la tasa de error.

En español se le llama también **exactitud**. Para el filtro:

$$\text{accuracy} = \frac{80 + 870}{1000} = 0.95.$$

Acierta en el 95 % de los correos. Suena muy bien, pero hay que compararlo contra algo.

Ejemplo 14.3 (La paradoja de la accuracy)**.** Un "filtro" que nunca marca nada como spam acierta en los 900 legítimos y falla en los 100 spam:

$$\text{accuracy} = \frac{0 + 900}{1000} = 0.90.$$

Un 90 % sin hacer nada. El filtro real apenas le gana por cinco puntos. En detección de fraude, donde el 1 % de las transacciones son fraude, decir siempre "no es fraude" da 99 % de accuracy y no detecta ni uno.

Ojito...

Con clases desbalanceadas la accuracy engaña: la domina la clase mayoritaria, y un clasificador inútil puede sacar un número alto. Sirve cuando las clases son de tamaño parecido y los dos errores cuestan lo mismo. Si no, hay que ver las métricas de cada clase.

Precisión

Definición 14.4 (Precisión)**.** De todos los ejemplos que el clasificador marcó como positivos, qué fracción de verdad lo era:

$$\text{precisión} = \frac{VP}{VP + FP}.$$

Contesta: *cuando el clasificador dice "positivo", ¿qué tan seguido tiene razón?* Para el filtro,

$$\text{precisión} = \frac{80}{80 + 30} = \frac{80}{110} = 0.727.$$

De cada 100 correos que manda a spam, 73 son spam y 27 eran legítimos. Solo mira la columna de "predice positivo".

Importa cuando los falsos positivos son caros: un filtro que borra correos legítimos, un sistema que bloquea tarjetas de crédito, un buscador cuyos primeros resultados tienen que ser buenos.

Ojito...

En español *precisión* y *exactitud* se usan casi como sinónimos, pero aquí son métricas distintas: exactitud es accuracy, la fracción de aciertos sobre todo; precisión es la de arriba, solo sobre lo que se predijo positivo. Mucha gente dice "precisión" cuando quiere decir accuracy.

Recall

Definición 14.5 (Recall)**.** De todos los ejemplos que de verdad son positivos, qué fracción detectó el clasificador:

$$\text{recall} = \frac{VP}{VP + FN}.$$

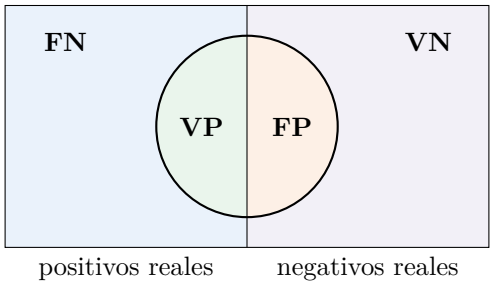
Contesta: *de todos los positivos que existen, ¿cuántos encontró?* Para el filtro,

$$\text{recall} = \frac{80}{80 + 20} = 0.80.$$

Atrapó el 80 % del spam. Solo mira el renglón de "real positivo". En español se le llama también **exhaustividad** o **sensibilidad**, y en la curva ROC aparece como **tasa de verdaderos positivos** (TPR).

Importa cuando los falsos negativos son caros: una prueba de cáncer, un detector de fraude, un sistema de búsqueda legal que no puede dejar fuera un documento relevante.

el círculo: lo que el clasificador marcó como positivo



La figura muestra la diferencia de un vistazo. La **precisión** es qué parte del círculo es VP: qué tan limpio está lo que se marcó. El **recall** es qué parte de la mitad izquierda quedó dentro del círculo: cuánto de lo que había se encontró.

Especificidad

Definición 14.6 (Especificidad)**.** De todos los ejemplos que de verdad son negativos, qué fracción clasificó correctamente como negativos:

$$\text{especificidad} = \frac{VN}{VN + FP}.$$

Es el recall de la clase negativa: *de todos los negativos, ¿a cuántos dejó en paz?* Para el filtro,

$$\text{especificidad} = \frac{870}{870 + 30} = 0.967.$$

Su complemento es la **tasa de falsos positivos**,

$$FPR = \frac{FP}{FP + VN} = 1 - \text{especificidad} = \frac{30}{900} = 0.033,$$

el otro eje de la curva ROC.

En medicina sensibilidad y especificidad van siempre en pareja: una prueba con sensibilidad alta casi no deja ir a un enfermo; una con especificidad alta casi no alarma a un sano.

Ojito...

Precisión y especificidad parecen hablar de lo mismo, los falsos positivos, pero no son iguales. La especificidad compara los FP contra todos los negativos reales, 900; la precisión, contra lo que se marcó positivo, 110. Por eso el filtro tiene especificidad de 0.967 y precisión de solo 0.727: 30 falsas alarmas son pocas entre 900 legítimos, pero muchas entre 110 correos marcados. Con clases desbalanceadas, la precisión es la que delata las falsas alarmas.

F1: precisión y recall en un solo número

Para comparar clasificadores conviene un solo número que tome en cuenta las dos. El **F1** es su media armónica:

$$F_1 = \frac{2 \cdot \text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}.$$

Para el filtro,

$$F_1 = \frac{2 \cdot 0.727 \cdot 0.80}{0.727 + 0.80} = \frac{1.164}{1.527} = 0.762.$$

Ejemplo 14.7 (Por qué la media armónica)**.** Un clasificador que solo marca como positivo el ejemplo del que está más seguro puede tener precisión 1 y recall 0.01. La media aritmética daría 0.505, como si fuera regular. La armónica da

$$F_1 = \frac{2 \cdot 1 \cdot 0.01}{1 + 0.01} = 0.020.$$

La media armónica se queda cerca del menor de los dos números: solo sale alta si precisión y recall son altas al mismo tiempo.

Mover el umbral

Un clasificador como Naive Bayes no da directamente "spam" o "legítimo": da una probabilidad, y se decide spam si pasa de un umbral u. Cambiar u cambia la matriz de confusión entera.

Ejemplo 14.8 (Diez correos con su puntaje)**.** El filtro le dio a cada correo su probabilidad de ser spam. Cuatro son spam de verdad (S) y seis legítimos (L):

Puntaje	0.95	0.85	0.80	0.70	0.55	0.45	0.40	0.30	0.20	0.10
Real	S	S	L	S	L	S	L	L	L	L

Se marca como spam todo lo que tenga puntaje mayor o igual a u:

u	VP	FP	FN	VN	**Precisión**	**Recall**	**F1**
0.90	1	0	3	6	1.000	0.25	0.400
0.75	2	1	2	5	0.667	0.50	0.571
0.60	3	1	1	5	0.750	0.75	0.750
0.42	4	2	0	4	0.667	1.00	0.800
0.15	4	5	0	1	0.444	1.00	0.615

Con el umbral alto el filtro es muy cuidadoso: lo que marca es spam seguro, pero se le escapa casi todo. Al bajarlo atrapa más spam y se le cuelan más legítimos. En $u = 0.42$ atrapa los cuatro con el mejor F1; más abajo ya no gana recall y solo pierde precisión.

Idea clave

Precisión y recall se jalan en sentidos contrarios: bajar el umbral sube el recall y casi siempre baja la precisión. No hay un umbral correcto en abstracto; se escoge según cuál error es más caro. Un filtro de spam prefiere umbral alto, porque perder un correo legítimo es peor; una prueba médica de tamizaje prefiere umbral bajo, porque dejar ir a un enfermo es peor.

Más de dos clases

Con k clases la matriz de confusión es de $k \times k$: el renglón es la clase real, la columna la predicha, y la diagonal son los aciertos.

Ejemplo 14.9 (Noticias en tres temas)**.** 90 noticias de prueba, 30 de cada tema:

Real \ Predicho	Deportes	Política	Tecnología
Deportes	**25**	3	2
Política	4	**20**	6
Tecnología	1	5	**24**

La accuracy es la diagonal entre el total, $69/90 = 0.767$. Para cada clase se calculan precisión y recall tratándola como la positiva y a las otras dos como negativas:

Clase	Precisión	Recall
Deportes	$25/30 = 0.833$	$25/30 = 0.833$
Política	$20/28 = 0.714$	$20/30 = 0.667$
Tecnología	$24/32 = 0.750$	$24/30 = 0.800$

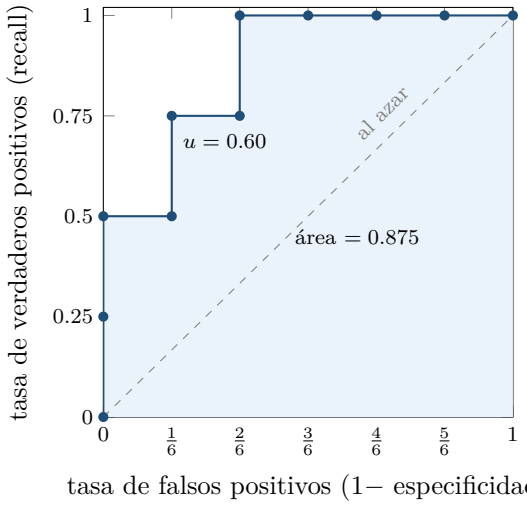
La precisión usa la suma de la columna y el recall la del renglón. Promediando las tres clases, la **macro-precisión** es 0.766 y el **macro-recall** 0.767.

Fuera de la diagonal se ve dónde se equivoca: la casilla más grande es 6, política confundida con tecnología. Ese número no sale en ninguna métrica resumida, y suele ser lo más útil para mejorar el clasificador.

Adelanto: la curva ROC

La segunda familia de métricas evalúa al clasificador en todos los umbrales a la vez. El nombre viene de los operadores de radar de la Segunda Guerra Mundial, que tenían que decidir qué tan fuerte debía ser una señal para declararla avión enemigo.

La curva ROC pone, para cada umbral, la tasa de falsos positivos en el eje horizontal y la de verdaderos positivos, el recall, en el vertical. Se recorren los correos del puntaje más alto al más bajo: cada spam sube la curva 1/4 y cada legítimo la mueve 1/6 a la derecha.



- La esquina superior izquierda, $(0, 1)$, es el clasificador perfecto: atrapa todos los positivos sin una sola falsa alarma.
- La diagonal punteada es un clasificador que adivina al azar. Una curva por debajo de ella es peor que tirar una moneda.
- Cada punto de la curva es un umbral. El punto $u = 0.60$ de la tabla anterior es $(1/6,\ 0.75)$.

El área bajo la curva, el **AUC**, resume la curva en un número. Aquí vale

$$\text{AUC} = \tfrac{1}{6} \cdot 0.5 + \tfrac{1}{6} \cdot 0.75 + \tfrac{4}{6} \cdot 1 = 0.083 + 0.125 + 0.667 = 0.875.$$

Idea clave

El AUC tiene una lectura directa: es la probabilidad de que un positivo tomado al azar reciba un puntaje más alto que un negativo tomado al azar. De los $4 \cdot 6 = 24$ pares spam-legítimo, en 21 el spam tiene el puntaje mayor, y $21/24 = 0.875$. A diferencia de la accuracy, no depende del umbral ni de cuántos ejemplos haya de cada clase: mide qué tan bien ordena el clasificador.

Resumen

Métrica	**Fórmula**	**Pregunta**	**Importa cuando**
Accuracy	$\frac{VP+VN}{\text{total}}$	¿Qué fracción acertó?	las clases están balanceadas
Precisión	$\frac{VP}{VP+FP}$	Cuando dice positivo, ¿qué tan seguido acierta?	las falsas alarmas son caras
Recall	$\frac{VP}{VP+FN}$	¿Cuántos de los positivos encontró?	dejar ir un positivo es caro
Especificidad	$\frac{VN}{VN+FP}$	¿A cuántos negativos dejó en paz?	se reporta junto con la sensibilidad
F1	$\frac{2PR}{P+R}$	¿Precisión y recall son altas a la vez?	se necesita un solo número
AUC	área bajo la ROC	¿Qué tan bien ordena positivos sobre negativos?	todavía no se fija el umbral

15. Árboles de decisión

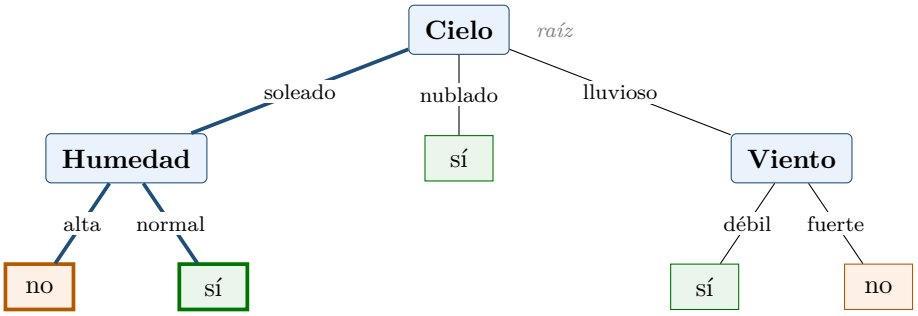
20 de octubre de 2026

Resumen rápido. Un árbol de decisión clasifica haciendo una pregunta por nodo: cada nodo interno prueba un atributo, cada rama es uno de sus valores y cada hoja es una clase. Se construye de arriba hacia abajo por particionamiento recursivo (ID3, de la familia TDIDT), y en cada nodo se escoge el atributo que deja los subconjuntos más puros, medido con la ganancia de información sobre la entropía. Se vio el defecto de esa medida, que prefiere a los atributos con muchos valores; el sobreajuste y sus dos remedios, la pre-poda y la post-poda; y los bosques aleatorios, que ponen a votar muchos árboles entrenados con bagging. Aquí se agregan el ejemplo completo de ID3 sobre el conjunto de jugar tenis, la razón de ganancia de C4.5, el índice de Gini, los atributos numéricos y el error fuera de la bolsa.

La estructura

Un árbol de decisión es una cadena de preguntas. Para clasificar un ejemplo se empieza en la raíz, se contesta la pregunta del nodo y se baja por la rama de la respuesta, hasta llegar a una hoja, que dice la clase.

- La **raíz** y los **nodos internos** son **nodos de decisión**: cada uno prueba un atributo.
- Las **ramas** que salen de un nodo son los valores que puede tomar su atributo.
- Las **hojas** son las clases.



El árbol de la figura decide si se juega tenis según el clima, y se construye paso a paso más abajo. Un día nuevo soleado, fresco, de humedad alta y viento fuerte baja por la ruta resaltada: Cielo es soleado, luego Humedad es alta, y la hoja dice que no se juega. La temperatura y el viento ni se consultaron.

Cada camino de la raíz a una hoja es una conjunción de pruebas sobre atributos, así que el árbol entero se lee como un conjunto de reglas, una por hoja:

si Cielo = soleado y Humedad = alta, entonces no se juega
si Cielo = soleado y Humedad = normal, entonces sí
si Cielo = nublado, entonces sí
si Cielo = lluvioso y Viento = débil, entonces sí
si Cielo = lluvioso y Viento = fuerte, entonces no

Idea clave

Esa legibilidad es la ventaja principal de un árbol frente a una red neuronal o una SVM: el modelo se puede leer, revisar con un experto del dominio y explicarle a quien se ve afectado por la decisión.

Construir el árbol a partir de un conjunto de entrenamiento se reduce a dos operaciones básicas:

- **Dividir** (*splitting*): escoger el atributo de un nodo y repartir sus ejemplos entre las ramas.

- **Podar** (*pruning*): quitar las ramas que solo le sirven a los ejemplos de entrenamiento y no ayudan con los nuevos.

ID3: particionamiento recursivo

ID3 (*Iterative Dichotomiser 3*, de Ross Quinlan, 1986) construye el árbol de arriba hacia abajo: escoge el atributo de la raíz, parte el conjunto según sus valores y repite lo mismo dentro de cada parte. Por eso a esta familia de algoritmos se le llama TDIDT, *Top-Down Induction of Decision Trees*.

Algoritmo 15.1 ID3$(S, \mathcal{A})$

Entrada: conjunto de entrenamiento S, atributos disponibles $\mathcal{A}$
Salida: un árbol de decisión

si todos los ejemplos de S son de la misma clase C **entonces**
    devolver una hoja con la clase C
fin si
si $\mathcal{A}$ está vacío **entonces**
    devolver una hoja con la clase mayoritaria de S
fin si
$A \leftarrow$ el atributo de $\mathcal{A}$ más informativo sobre S
crear un nodo de decisión que prueba A
para todo valor v de A **hacer**
    $S_v \leftarrow$ los ejemplos de S con $A = v$
    si S_v está vacío **entonces**
        colgar de la rama v una hoja con la clase mayoritaria de S
    si no
        colgar de la rama v el subárbol ID3$(S_v,\ \mathcal{A} \setminus \{A\})$
    fin si
fin para
devolver el nodo

En clase se vio el primer caso base y el paso recursivo. Los otros dos casos hacen falta para que el algoritmo siempre termine. Si ya no quedan atributos y los ejemplos siguen mezclados, como pasa cuando dos días idénticos tienen clases distintas, no queda nada que preguntar y se vota. Si ningún ejemplo toma un valor, esa rama no tiene datos y hereda la clase mayoritaria del padre.

Un atributo se usa a lo más una vez en cada camino: una vez que se sabe que el cielo está soleado, volver a preguntarlo no reparte nada.

Todo el algoritmo depende de una sola decisión, la de qué atributo es el "más informativo".

Entropía

El criterio es la **pureza**: el mejor atributo es el que reparte los ejemplos en subconjuntos donde casi todos son de la misma clase. Un subconjunto puro ya es una hoja; uno revuelto todavía necesita más preguntas. ID3 mide qué tan revuelto está un conjunto con la entropía de Shannon, que solo depende de cuántos ejemplos hay de cada clase.

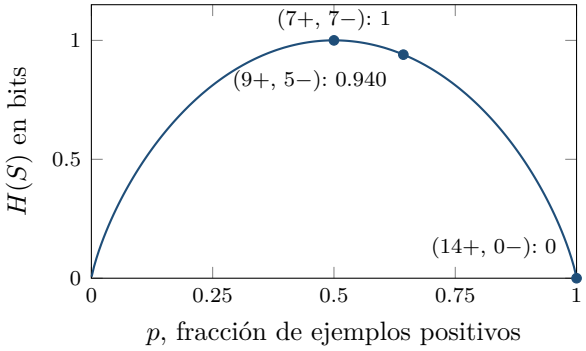
Definición 15.1 (Entropía)**.** Sea S un conjunto de ejemplos con k clases, y p_i la fracción de ejemplos

de S que son de la clase i. La entropía de S es

$$H(S) = -\sum_{i=1}^{k} p_i \log_2 p_i,$$

con la convención $0 \log_2 0 = 0$, porque una clase ausente no aporta incertidumbre.

Se lee como el número promedio de bits que hacen falta para comunicar la clase de un ejemplo tomado al azar de S. Si todos son de la misma clase no hay nada que comunicar, y $H = 0$. Si las clases están repartidas a partes iguales cada ejemplo es una sorpresa, y la entropía llega a su máximo, $\log_2 k$; con dos clases, 1 bit. El signo menos está porque $\log_2 p_i \leq 0$: sin él la entropía saldría negativa.



Ejemplo 15.2 (Tres conjuntos de 14 ejemplos)**.** Se escribe $(a+,\, b-)$ para un conjunto con a ejemplos positivos y b negativos.

$$\begin{aligned} H(9+,\, 5-) &= -\tfrac{9}{14} \log_2 \tfrac{9}{14} - \tfrac{5}{14} \log_2 \tfrac{5}{14} = 0.410 + 0.530 = 0.940, \\ H(7+,\, 7-) &= -\tfrac{1}{2} \log_2 \tfrac{1}{2} - \tfrac{1}{2} \log_2 \tfrac{1}{2} = 0.5 + 0.5 = 1, \\ H(14+,\, 0-) &= -1 \cdot \log_2 1 - 0 = 0. \end{aligned}$$

El primero está cargado hacia los positivos, así que es menos incierto que el de mitad y mitad, pero está lejos de ser puro.

Ganancia de información

Si un conjunto S se parte con un atributo A, cada subconjunto tiene su propia entropía. Lo que bajó la entropía al partir es la **ganancia de información**: cuánta incertidumbre sobre la clase quitó conocer el valor de A. Conviene que sea grande.

Definición 15.3 (Ganancia de información)**.** Sea S_v el subconjunto de S donde el atributo A vale v.

$$\text{Ganancia}(S, A) = H(S) - \sum_{v \in \text{valores}(A)} \frac{|S_v|}{|S|} H(S_v).$$

La suma es la entropía que queda después de partir, promediada según el tamaño de cada parte: un subconjunto revuelto de dos ejemplos pesa menos que uno de doce. ID3 escoge en cada nodo el atributo de **ganancia máxima**. Como $H(S)$ es la misma para todos los atributos, escoger la ganancia máxima es escoger la menor entropía restante.

Ejemplo completo: jugar tenis

El conjunto clásico de Quinlan, en la versión del libro de Mitchell: 14 días con cuatro atributos del clima y si ese día se jugó tenis.

Día	Cielo	Temperatura	Humedad	Viento	Juega
D1	soleado	calurosa	alta	débil	no
D2	soleado	calurosa	alta	fuerte	no
D3	nublado	calurosa	alta	débil	sí
D4	lluvioso	templada	alta	débil	sí
D5	lluvioso	fresca	normal	débil	sí
D6	lluvioso	fresca	normal	fuerte	no
D7	nublado	fresca	normal	fuerte	sí
D8	soleado	templada	alta	débil	no
D9	soleado	fresca	normal	débil	sí
D10	lluvioso	templada	normal	débil	sí
D11	soleado	templada	normal	fuerte	sí
D12	nublado	templada	alta	fuerte	sí
D13	nublado	calurosa	normal	débil	sí
D14	lluvioso	templada	alta	fuerte	no

Paso 1: entropía del conjunto completo. Hay 9 días en que se jugó y 5 en que no, así que $H(S) = H(9+,\, 5-) = 0.940$, el valor del ejemplo anterior.

Paso 2: ganancia de cada atributo en la raíz. Para Cielo, los tres subconjuntos son

$$\begin{aligned} S_{\text{soleado}} &= \{D1, D2, D8, D9, D11\} && (2+,\, 3-) && H = 0.971, \\ S_{\text{nublado}} &= \{D3, D7, D12, D13\} && (4+,\, 0-) && H = 0, \\ S_{\text{lluvioso}} &= \{D4, D5, D6, D10, D14\} && (3+,\, 2-) && H = 0.971, \end{aligned}$$

y la ganancia es

$$\begin{aligned} \text{Ganancia}(S, \text{Cielo}) &= H(S) - \tfrac{5}{14} H(S_{\text{soleado}}) - \tfrac{4}{14} H(S_{\text{nublado}}) - \tfrac{5}{14} H(S_{\text{lluvioso}}) \\ &= 0.9403 - \tfrac{5}{14}(0.9710) - \tfrac{4}{14}(0) - \tfrac{5}{14}(0.9710) \\ &= 0.9403 - 0.3468 - 0 - 0.3468 \\ &= 0.247. \end{aligned}$$

Con los otros tres atributos se hace la misma cuenta:

Atributo	Subconjuntos	Entropía restante	Ganancia
Cielo	soleado $(2+, 3-)$, nublado $(4+, 0-)$, lluvioso $(3+, 2-)$	0.694	**0.247**
Humedad	alta $(3+, 4-)$, normal $(6+, 1-)$	0.788	0.152
Viento	débil $(6+, 2-)$, fuerte $(3+, 3-)$	0.892	0.048
Temperatura	calurosa $(2+, 2-)$, templada $(4+, 2-)$, fresca $(3+, 1-)$	0.911	0.029

Gana Cielo y va en la raíz. La rama nublado ya es pura: todos los días nublados se jugó, y queda una hoja "sí". Las otras dos ramas siguen revueltas y se repite el procedimiento con los atributos que quedan.

Paso 3: la rama soleado. Ahora el conjunto es S_{soleado}, con $H = 0.971$:

$$\begin{aligned}
\text{Ganancia}(S_{\text{soleado}}, \text{Humedad}) &= 0.971 - \tfrac{3}{5}\,H(0+,\,3-) - \tfrac{2}{5}\,H(2+,\,0-) \\
&= 0.971 - 0 - 0 = 0.971, \\
\text{Ganancia}(S_{\text{soleado}}, \text{Temperatura}) &= 0.971 - \tfrac{2}{5}\,H(0+,\,2-) - \tfrac{2}{5}\,H(1+,\,1-) - \tfrac{1}{5}\,H(1+,\,0-) \\
&= 0.971 - 0 - 0.400 - 0 = 0.571, \\
\text{Ganancia}(S_{\text{soleado}}, \text{Viento}) &= 0.971 - \tfrac{3}{5}\,H(1+,\,2-) - \tfrac{2}{5}\,H(1+,\,1-) \\
&= 0.971 - 0.551 - 0.400 = 0.020.
\end{aligned}$$

Gana Humedad, y sus dos subconjuntos ya son puros: con humedad alta (D1, D2, D8) no se jugó, con humedad normal (D9, D11) sí.

Paso 4: la rama lluvioso. El conjunto es S_{lluvioso}, también con $H = 0.971$:

$$\begin{aligned}
\text{Ganancia}(S_{\text{lluvioso}}, \text{Viento}) &= 0.971 - \tfrac{3}{5}\,H(3+,\,0-) - \tfrac{2}{5}\,H(0+,\,2-) \\
&= 0.971 - 0 - 0 = 0.971, \\
\text{Ganancia}(S_{\text{lluvioso}}, \text{Humedad}) &= 0.971 - \tfrac{2}{5}\,H(1+,\,1-) - \tfrac{3}{5}\,H(2+,\,1-) \\
&= 0.971 - 0.400 - 0.551 = 0.020, \\
\text{Ganancia}(S_{\text{lluvioso}}, \text{Temperatura}) &= 0.971 - \tfrac{3}{5}\,H(2+,\,1-) - \tfrac{2}{5}\,H(1+,\,1-) \\
&= 0.971 - 0.551 - 0.400 = 0.020.
\end{aligned}$$

Gana Viento, con dos subconjuntos puros: con viento débil (D4, D5, D10) se jugó, con viento fuerte (D6, D14) no. Ningún día lluvioso fue caluroso, así que en Temperatura la rama calurosa quedaría vacía; si Temperatura hubiera ganado, esa rama heredaría la clase mayoritaria, el caso base de S_v vacío.

Todas las hojas son puras y el algoritmo termina. El resultado es el árbol del principio de la clase. La Temperatura no aparece en ningún nodo: ID3 descubrió por su cuenta que, sabiendo el cielo, la humedad y el viento, no aporta nada.

Ojito...

ID3 es **voraz**: en cada nodo toma el mejor atributo de ese momento y nunca se regresa a reconsiderarlo. No garantiza el árbol más chico que clasifica bien al entrenamiento; encontrar ese árbol es NP-completo (Hyafil y Rivest, 1976). La búsqueda voraz es lo que lo vuelve rápido.

El defecto: atributos con muchos valores

La ganancia de información favorece a los atributos con muchos valores. Con más ramas los subconjuntos son más chicos, y un subconjunto chico sale puro con facilidad, aunque sea por casualidad.

Ejemplo 15.4 (El número de día como atributo)**.** Si al conjunto se le agrega la columna Día, con valores D1 a D14, cada valor tiene un solo ejemplo y cada subconjunto es puro:

$$\text{Ganancia}(S, \text{Día}) = 0.940 - \sum_{v=1}^{14} \tfrac{1}{14} \cdot 0 = 0.940,$$

la ganancia más grande posible, muy arriba de los 0.247 de Cielo. ID3 pondría Día en la raíz con 14 hojas. Ese árbol clasifica perfecto el entrenamiento y no sirve para ningún día nuevo: un D15 no tiene rama por la cual bajar.

C4.5, el sucesor de ID3 del mismo Quinlan (1993), lo corrige con la **razón de ganancia**. Divide la ganancia entre la entropía de la partición misma, calculada sobre los tamaños de las ramas y sin mirar las clases:

$$\text{InfoDiv}(S, A) = -\sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \qquad \text{RazónGanancia}(S, A) = \frac{\text{Ganancia}(S, A)}{\text{InfoDiv}(S, A)}.$$

La información de división crece con el número de ramas: vale 1 para un atributo que parte en dos mitades y $\log_2 14 = 3.807$ para Día, que parte en 14 pedazos de uno.

Atributo	**Ganancia**	**InfoDiv**	**Razón de ganancia**
Día	0.940	3.807	0.247
Cielo	0.247	1.577	0.156
Humedad	0.152	1.000	0.152
Viento	0.048	0.985	0.049
Temperatura	0.029	1.557	0.019

Ojito...

La razón de ganancia reduce la ventaja de Día de 0.940 contra 0.247 a 0.247 contra 0.156, pero no se la quita. Un identificador (folio, matrícula, nombre) siempre gana porque parte en conjuntos de un solo ejemplo, y se elimina del conjunto antes de entrenar: no describe al ejemplo, solo lo nombra. El otro extremo también falla: con una información de división cercana a cero la razón se dispara sin que el atributo sirva, y por eso C4.5 solo compara razones entre los atributos cuya ganancia es por lo menos el promedio.

La entropía tampoco es la única medida de impureza. CART (Breiman y otros, 1984), y scikit-learn por omisión, usan el **índice de Gini**,

$$\text{Gini}(S) = 1 - \sum_{i=1}^{k} p_i^2,$$

que es la probabilidad de equivocarse si se etiqueta un ejemplo de S con una clase sorteada según las proporciones de S. Vale 0 en un conjunto puro y 0.5 con dos clases mitad y mitad. Para el conjunto de tenis, $1 - (9/14)^2 - (5/14)^2 = 0.459$. Gini y entropía casi siempre escogen el mismo atributo, y Gini se calcula sin logaritmos.

Atributos numéricos

ID3 solo maneja atributos con unos cuantos valores discretos. Con una temperatura en grados, cada valor distinto sería una rama, y se cae en el defecto de arriba. C4.5 convierte el atributo numérico en una pregunta binaria, $x < t$, y escoge el umbral t por ganancia:

1. Ordenar los ejemplos por el valor del atributo.
2. Tomar como candidatos los puntos medios entre valores consecutivos donde cambia la clase. Un umbral entre dos ejemplos de la misma clase nunca gana más que recorrerlo a la frontera de clase más cercana.
3. Calcular la ganancia de cada candidato y quedarse con el mejor.

Ejemplo 15.5 (Un umbral para la temperatura)**.** Seis días con su temperatura en grados Fahrenheit, ya ordenados:

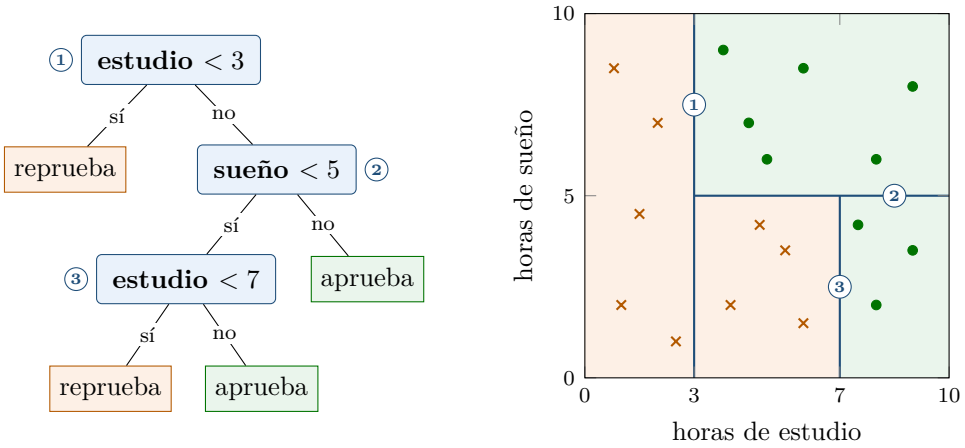
Temperatura	40	48	60	72	80	90
Juega	no	no	sí	sí	sí	no

La clase cambia entre 48 y 60, y entre 80 y 90, así que los candidatos son $t = 54$ y $t = 85$. El conjunto es (3+, 3−), con $H = 1$.

$$\text{Ganancia}(t = 54) = 1 - \tfrac{2}{6}\,H(0+,\,2-) - \tfrac{4}{6}\,H(3+,\,1-) = 1 - 0 - \tfrac{4}{6}(0.811) = 0.459,$$
$$\text{Ganancia}(t = 85) = 1 - \tfrac{5}{6}\,H(3+,\,2-) - \tfrac{1}{6}\,H(0+,\,1-) = 1 - \tfrac{5}{6}(0.971) - 0 = 0.191.$$

Gana $t = 54$: el nodo pregunta "¿temperatura < 54?". A diferencia de un atributo discreto, uno numérico se puede volver a usar más abajo en el mismo camino con otro umbral.

Con atributos numéricos cada nodo corta el espacio con una recta paralela a un eje, así que el árbol parte el espacio en rectángulos y asigna una clase a cada uno. En la figura, cada número marca el corte que hace el nodo correspondiente:



Ojito...

Una frontera diagonal, como "aprueba si estudio + sueño > 10", un árbol solo la puede aproximar con una escalera de muchos cortes. Un clasificador lineal la representa con un solo hiperplano. Cuando las clases se separan en diagonal, el árbol necesita muchos nodos y muchos datos para lo que un perceptrón hace con tres pesos.

Sobreajuste

ID3 sigue partiendo hasta que cada hoja es pura. Si los datos tienen ruido, una etiqueta mal capturada o un atributo mal medido, el árbol crece ramas solo para acomodar esos ejemplos. El árbol se aprende el entrenamiento de memoria, ruido incluido, y generaliza peor.

Definición 15.6 (Sobreajuste)**.** Un árbol h **sobreajusta** el conjunto de entrenamiento si existe otro árbol h' que se equivoca más que h en el entrenamiento, pero menos que h sobre la distribución completa de los ejemplos.

Ejemplo 15.7 (Un solo día mal etiquetado)**.** Al conjunto de tenis se le agrega

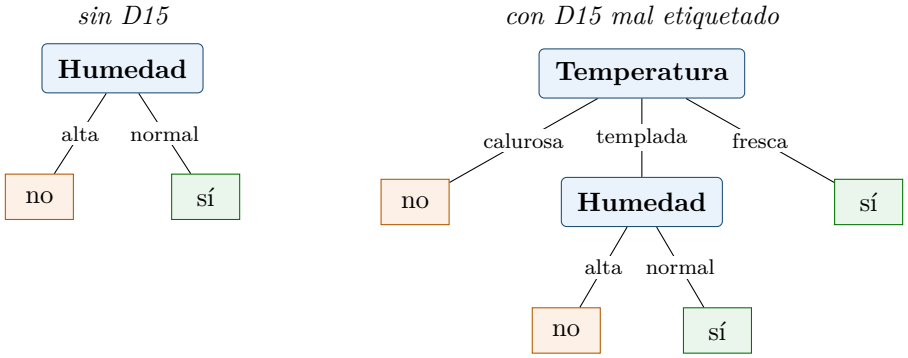
$$\text{D15} = (\text{soleado},\ \text{calurosa},\ \text{normal},\ \text{fuerte}), \quad \text{Juega} = \text{no},$$

un día que por la regla aprendida debió ser "sí" y que alguien anotó mal. Cielo sigue en la raíz, pero la

rama soleado ahora es (2+, 4−) con $H = 0.918$, y las ganancias cambian de orden:

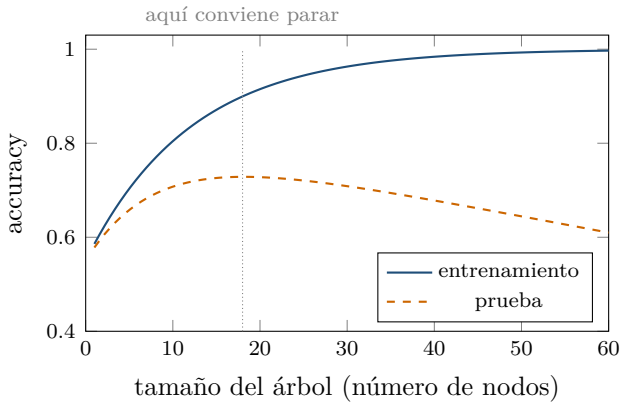
$$\begin{aligned}\text{Ganancia}(S_{\text{soleado}}, \text{Temperatura}) &= 0.918 - \tfrac{3}{6}\,H(0+,\,3-) - \tfrac{2}{6}\,H(1+,\,1-) - \tfrac{1}{6}\,H(1+,\,0-)\\ &= 0.918 - 0 - 0.333 - 0 = 0.585,\\ \text{Ganancia}(S_{\text{soleado}}, \text{Humedad}) &= 0.918 - \tfrac{3}{6}\,H(0+,\,3-) - \tfrac{3}{6}\,H(2+,\,1-)\\ &= 0.918 - 0 - 0.459 = 0.459.\end{aligned}$$

Ahora gana Temperatura, y la rama templada (D8 “no”, D11 “sí”) necesita todavía otro nodo. Humedad y Viento empatan ahí y se escoge Humedad:



Una sola etiqueta equivocada reorganizó la rama completa. El árbol nuevo sigue clasificando bien los 15 días, pero ahora dice que un día soleado, caluroso, de humedad normal y viento débil no se juega, cuando el árbol original decía que sí. La regla “soleado y caluroso, no se juega” salió de D1 y D2, que además tenían humedad alta, y del día mal anotado.

La forma típica del problema, con el tamaño del árbol creciendo conforme ID3 agrega nodos:



Curvas esquemáticas, en la forma de la figura 3.6 de Mitchell. En entrenamiento la accuracy sube siempre, porque cada nodo nuevo acomoda algún ejemplo más. En prueba sube mientras el árbol aprende la estructura real, y luego baja cuando los nodos nuevos solo acomodan ruido. Hay dos formas de quedarse cerca de la línea punteada: dejar de crecer el árbol a tiempo o crecerlo completo y luego recortarlo.

Pre-poda

La **pre-poda** (*pre-pruning*) detiene el crecimiento antes de que las hojas sean puras. Un nodo se vuelve hoja, con la clase mayoritaria de sus ejemplos, cuando se cumple alguna condición de paro:

- el árbol llegó a una profundidad máxima;
- al nodo llegan menos de cierto número de ejemplos;
- la mejor ganancia disponible es menor que un umbral;
- una prueba estadística, como la χ^2, no encuentra diferencia significativa entre la distribución de clases del nodo y la de sus hijos.

Es barata: el árbol grande nunca se construye. El costo es que decide mirando un solo nivel hacia adelante.

Ejemplo 15.8 (El efecto horizonte)**.** La clase es el o exclusivo de dos atributos binarios:

A	B	clase
0	0	no
0	1	sí
1	0	sí
1	1	no

Partir por A deja dos subconjuntos (1+, 1−), igual de revueltos que el original:

$$\text{Ganancia}(S, A) = 1 - \tfrac{2}{4}(1) - \tfrac{2}{4}(1) = 0,$$

y lo mismo pasa con B. Una pre-poda que exige ganancia mínima de 0.01 se detiene en la raíz con una hoja que acierta la mitad. Si se partiera por A y luego por B, el árbol acertaría todo. La pre-poda no puede ver que un corte inútil hoy hace posible uno perfecto en el siguiente nivel.

Post-poda

La **post-poda** (*post-pruning*) hace lo contrario: deja que ID3 crezca el árbol completo, con todo y sobreajuste, y después lo recorta de abajo hacia arriba. La versión más directa es la **poda por error reducido** (Quinlan, 1987), que aparta un conjunto de validación que el árbol no vio:

1. Para cada nodo interno, probar a reemplazar el subárbol que cuelga de él por una hoja con la clase mayoritaria de los ejemplos de entrenamiento que llegan a ese nodo.
2. Si con el reemplazo la accuracy en validación no baja, el reemplazo se queda.
3. Repetir con el nodo cuya poda más mejore la validación, hasta que cualquier poda adicional la empeore.

Ejemplo 15.9 (Podar un nodo)**.** A un nodo llegan 20 ejemplos de validación. El subárbol que cuelga de él, con sus 7 hojas, acierta 13. Una hoja sola con la clase mayoritaria acertaría 15. El subárbol se reemplaza por la hoja: es más chico y se equivoca menos con datos que no vio. Si la hoja acertara 12, el subárbol se queda, porque está capturando algo real.

Hay variantes que no gastan datos en un conjunto de validación. C4.5 estima el error de cada hoja con una cota pesimista calculada sobre el mismo entrenamiento, y CART poda por **costo-complejidad**, que penaliza el número de hojas con un parámetro α escogido por validación cruzada.

Idea clave

En la práctica la post-poda suele dar mejores árboles que la pre-poda, porque decide con el árbol completo a la vista y no cae en el efecto horizonte. El precio es construir primero el árbol grande. La pre-poda es la opción cuando el árbol completo sería demasiado caro de construir.

Bosques aleatorios

Un árbol tiene **alta varianza**: cambiar unos cuantos ejemplos de entrenamiento cambia un atributo cerca de la raíz y con él todo lo que cuelga, como pasó con D15. Un **bosque aleatorio** (*random forest*, Breiman, 2001) aprovecha esa inestabilidad: entrena muchos árboles distintos y los pone a votar. Cada árbol se equivoca a su manera, y en la votación esos errores tienden a cancelarse.

Para que los árboles salgan distintos hacen falta conjuntos de entrenamiento distintos, y solo hay uno. La solución es el **bagging**.

Definición 15.10 (Bagging)**.** *Bootstrap aggregating* (Breiman, 1996). Dado un conjunto S de n ejemplos:

1. Sortear n ejemplos de S **con reemplazo**. El resultado S_b es una **muestra bootstrap**: algunos ejemplos salen repetidos y otros no salen.
2. Entrenar un árbol h_b sobre S_b.
3. Repetir para $b = 1, \ldots, B$.

Un ejemplo nuevo x se clasifica por mayoría de votos entre $h_1(x), \ldots, h_B(x)$. En regresión se promedian.

Ejemplo 15.11 (Una muestra bootstrap de los días de tenis)**.** Se sortean 14 números del 1 al 14 con reemplazo y salen

$$2,\ 2,\ 3,\ 5,\ 7,\ 8,\ 8,\ 8,\ 10,\ 11,\ 13,\ 13,\ 13,\ 14.$$

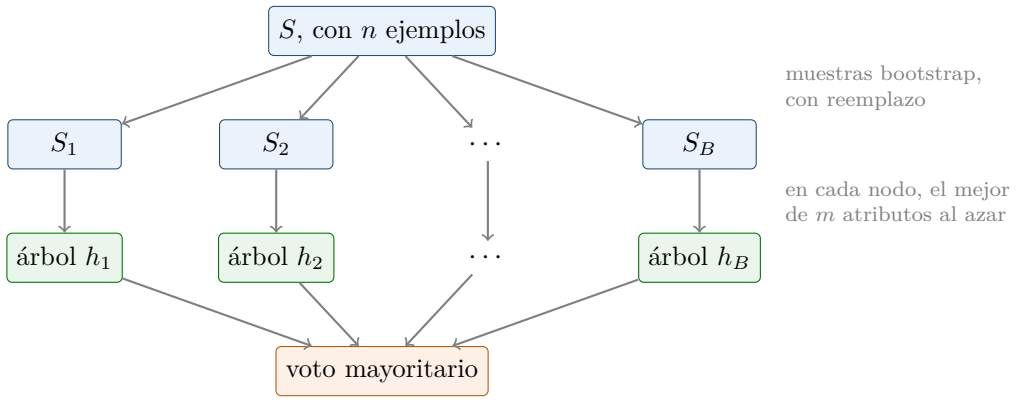
La muestra tiene 14 días, pero solo 9 distintos: D8 y D13 aparecen tres veces cada uno, D2 dos, y D1, D4, D6, D9 y D12 no aparecen. El árbol entrenado con esta muestra le da a D8 triple peso y nunca ve D1.

La probabilidad de que un ejemplo dado no salga en ninguno de los n sorteos es

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow[n\to\infty]{} \frac{1}{e} \approx 0.368,$$

así que cada muestra bootstrap contiene alrededor del 63.2 % de los ejemplos distintos. Con $n = 14$ la cuenta exacta da 64.6 %, o sea unos 9 días, como en el ejemplo.

Un bosque aleatorio es bagging de árboles más una segunda fuente de azar: en **cada nodo** se sortean m de los p atributos, y el mejor se escoge solo entre esos m. Para clasificación lo usual es $m \approx \sqrt{p}$; con los cuatro atributos de tenis, $m = 2$. Los árboles se crecen completos, sin podar, porque la varianza de cada uno ya la controla la votación.



Algoritmo 15.2 Bosque aleatorio

Entrada: S con n ejemplos y p atributos, número de árboles B, atributos por nodo m

1: **para** $b \leftarrow 1$ **hasta** B **hacer**
2: $\quad S_b \leftarrow n$ ejemplos sorteados de S con reemplazo
3: $\quad h_b \leftarrow$ árbol entrenado sobre S_b, sin podar; en cada nodo se sortean m atributos y se escoge el mejor de ellos
4: **fin para**
5: **devolver** $h(x) =$ la clase más votada entre $h_1(x), \ldots, h_B(x)$

Ejemplo 15.12 (La votación)**.** Un bosque de cinco árboles recibe un día soleado, fresco, de humedad alta y viento fuerte. Los votos salen no, no, sí, no, sí: gana "no" tres a dos. Ningún árbol tiene que ser perfecto; basta con que la mayoría acierte en cada ejemplo.

Idea clave

Por qué votar ayuda: si 25 árboles acertaran cada uno el 70 % de las veces y se equivocaran **de forma independiente**, la mayoría acertaría cuando al menos 13 aciertan, y eso pasa con probabilidad

$$\sum_{k=13}^{25} \binom{25}{k} 0.7^k \, 0.3^{25-k} = 0.983.$$

La independencia es la condición que importa. Con bagging solo, si un atributo es mucho mejor que los demás, como Cielo, todos los árboles lo ponen en la raíz, se parecen entre sí y se equivocan en los mismos días; votar no corrige un error que todos comparten. El sorteo de atributos por nodo obliga a que en muchos nodos Cielo ni siquiera esté disponible, y los árboles salen menos correlacionados.

Cada ejemplo se queda fuera de alrededor del 37 % de las muestras bootstrap. Clasificar cada ejemplo solo con los árboles que no lo vieron da el **error fuera de la bolsa** (*out-of-bag*), una estimación del error en datos nuevos que sale gratis, sin apartar un conjunto de validación. En la muestra del ejemplo, D1, D4, D6, D9 y D12 sirven para evaluar al árbol entrenado con ella.

Ojito...

Lo que se gana en accuracy se pierde en legibilidad. Un bosque de 500 árboles no se lee como reglas. Lo más cercano es la **importancia de cada atributo**: cuánto bajó la impureza en promedio en los nodos que lo usan, sumado sobre todos los árboles. Dice qué atributos pesan, pero no cómo se combinan.

Resumen

Concepto	Fórmula o regla	Para qué
Entropía	$-\sum_i p_i \log_2 p_i$	mide qué tan revuelto está un conjunto; 0 si es puro
Ganancia	$H(S) - \sum_v \frac{\lvert S_v \rvert}{\lvert S \rvert} H(S_v)$	criterio de ID3 para escoger el atributo; favorece a los de muchos valores
Razón de ganancia	Ganancia/InfoDiv	criterio de C4.5; castiga partir en muchas ramas
Gini	$1 - \sum_i p_i^2$	impureza de CART y scikit-learn, sin logaritmos
Pre-poda	parar al llegar a profundidad, tamaño o ganancia mínimos	barata; sufre el efecto horizonte
Post-poda	crecer completo y reemplazar subárboles por hojas si la validación no empeora	la que mejor funciona en la práctica
Bagging	B árboles sobre muestras bootstrap, voto mayoritario	baja la varianza de un árbol solo
Bosque aleatorio	bagging más $m \approx \sqrt{p}$ atributos al azar en cada nodo	árboles menos correlacionados; error fuera de la bolsa gratis

16. Selección de atributos

22 de octubre de 2026

Resumen rápido. Seleccionar atributos es quedarse con un subconjunto de las variables originales que discrimine igual o mejor que todas. No es lo mismo que extraer variables, que construye atributos nuevos combinando los originales. Se selecciona porque hay atributos irrelevantes o redundantes que estorban al clasificador, encarecen las mediciones e impiden visualizar los datos. Hay dos familias: los **filtros**, que califican los atributos con una medida sobre los datos sin usar el clasificador (correlación de Pearson, información mutua, RELIEF, clasificador de un solo atributo, árboles de decisión), y los **wrappers**, que usan al clasificador para evaluar subconjuntos y por eso son un problema combinatorio que se ataca con heurísticas: selección hacia adelante y hacia atrás, búsquedas secuenciales generalizadas, búsqueda flotante y algoritmos genéticos.

Qué es la selección de atributos

Definición 16.1 (Selección de atributos)**.** Dado un conjunto de n atributos $X = \{x_1, \ldots, x_n\}$ y un criterio J que mide qué tan bueno es un subconjunto, encontrar

$$S^* = \arg \max_{S \subseteq X,\ |S|=d} J(S), \qquad d < n.$$

Los atributos elegidos se usan tal cual; los demás se descartan.

El criterio J puede ser una medida calculada directo sobre los datos o la exactitud de un clasificador entrenado con el subconjunto. Esa elección separa a las dos familias de métodos.

La **extracción** de variables resuelve el mismo problema, bajar la dimensión, por otro camino: construye d atributos nuevos como funciones de los n originales, $y = f(x_1, \ldots, x_n)$. PCA y los autoencoders de la clase del 24 de septiembre son extracción.



Ejemplo 16.2 (Riesgo de diabetes)**.** Para predecir riesgo de diabetes se tienen altura, peso, edad y el folio del expediente. Seleccionar es quedarse con altura y peso: el médico sigue viendo dos medidas que entiende. Extraer es construir el índice de masa corporal, peso/altura^2, o una componente principal como $0.71\,\text{altura} + 0.70\,\text{peso}$: pueden discriminar mejor, pero ya no son algo que se mida directamente.

Idea clave

La selección conserva el significado de los atributos y ahorra medirlos: si el folio y la edad se descartan, ya no hay que capturarlos. La extracción suele comprimir mejor, pero necesita todos los atributos originales para calcular los nuevos, y lo que produce es más difícil de interpretar.

Por qué seleccionar

Lo natural sería pensar que más atributos nunca estorban: en el peor caso el clasificador ignora los que no sirven. En la práctica no los ignora.

- **Atributos irrelevantes.** No ayudan a discriminar entre clases y meten ruido. En un clasificador por distancia pesan lo mismo que los buenos.
- **Atributos redundantes.** Repiten información de otro, como la temperatura en Celsius y en Fahrenheit. No agregan nada y cuestan medirlos y procesarlos.
- **Pocos datos para muchas dimensiones.** Con N ejemplos fijos, cada atributo nuevo agrega parámetros que estimar con los mismos datos. Pasado cierto punto, el clasificador empeora.
- **Costo e interpretación.** Menos atributos es entrenar más rápido, medir menos y un modelo que se puede explicar.
- **Visualización.** Para ver los datos se escogen dos o tres atributos del total y se estudian en una gráfica. Escoger bien cuáles es un problema de selección.

Ejemplo 16.3 (Un atributo de ruido confunde al vecino más cercano)**.** La clase depende solo de x_1: los azules están cerca de $x_1 = 1$ y los rojos cerca de $x_1 = 3$. El atributo x_2 es ruido entre 0 y 100. Se clasifica $q = (1.0,\ 50)$, que es azul, con dos ejemplos de entrenamiento:

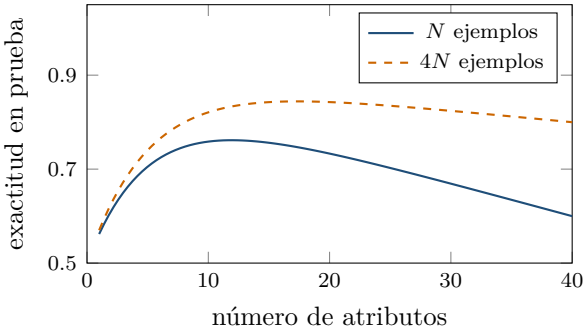
$$a = (1.2,\ 90) \text{ azul}, \qquad b = (3.0,\ 52) \text{ rojo}.$$

Con los dos atributos,

$$d(q, a) = \sqrt{0.2^2 + 40^2} = 40.0, \qquad d(q, b) = \sqrt{2^2 + 2^2} = 2.83,$$

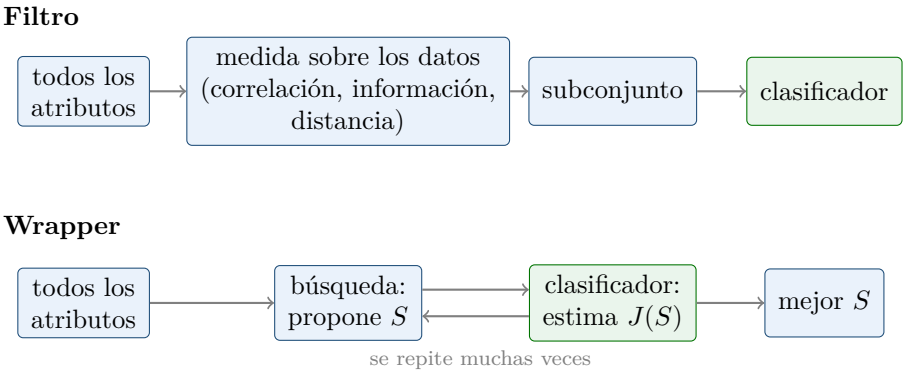
y 1-NN dice rojo. Con solo x_1, $d(q, a) = 0.2$ y $d(q, b) = 2$, y dice azul. Normalizar los rangos arregla la escala, pero no el problema: el ruido sigue sumando a la distancia una cantidad al azar.

El tercer punto se conoce como el **fenómeno de Hughes** o la **maldición de la dimensionalidad**. Con el número de ejemplos fijo, la exactitud primero sube al agregar atributos útiles y luego baja:



Curvas esquemáticas. Con más ejemplos el máximo se recorre a la derecha: se pueden usar más atributos antes de que empiecen a estorbar.

Dos familias: filtros y wrappers



Métodos de filtro. Califican cada atributo, o cada subconjunto, con una medida que sale de los datos: qué tanto se relaciona con la clase, cuánta información aporta, qué tan bien separa vecinos. El clasificador final no participa; se entrena después, con los atributos que pasaron el filtro.

+ Rápidos: casi siempre lineales en el número de atributos.
+ Independientes del clasificador: el mismo subconjunto sirve para k-NN, para Naive Bayes o para una red.
+ Menos propensos a sobreajustar, porque no optimizan la exactitud sobre los mismos datos.
− La medida puede no coincidir con lo que le sirve al clasificador.
− Los que califican atributo por atributo no ven las interacciones: dos atributos inútiles por separado pueden ser perfectos juntos.
− Tampoco ven la redundancia: dos copias del mismo atributo bueno reciben la misma calificación alta.

Métodos wrapper (envolventes). El clasificador se trata como caja negra: se le propone un subconjunto, se entrena con él y su exactitud, estimada con validación cruzada, es la calificación. Una estrategia de búsqueda decide qué subconjunto probar después.

+ Optimizan directo lo que importa: la exactitud del clasificador que se va a usar.
+ Evalúan subconjuntos completos, así que ven interacciones y redundancia.
− Caros: cada evaluación es entrenar y validar un clasificador.
− El subconjunto queda amarrado a ese clasificador.
− Riesgo de sobreajustar la selección: con muchas evaluaciones, alguna sale alta por suerte.

Filtro: coeficiente de correlación de Pearson

Definición 16.4 (Coeficiente de correlación de Pearson)**.** Para un atributo x y la clase c, codificada como 0 y 1, medidos en N ejemplos,

$$r = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(c_i - \bar{c})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2 \ \sum_{i=1}^{N}(c_i - \bar{c})^2}}, \qquad -1 \le r \le 1.$$

Mide qué tan bien se acomodan los puntos (x_i, c_i) a una recta. $|r|$ cerca de 1 es una relación lineal fuerte; cerca de 0, que no hay relación lineal. El signo solo dice la dirección, así que se ordenan los atributos por $|r|$ y se toman los primeros d.

Ejemplo 16.5 (Dos atributos contra la clase)**.** Seis ejemplos, tres de cada clase:

x_1	1	2	3	6	7	8
x_2	5	1	4	2	6	3
c	0	0	0	1	1	1

Para x_1: $\bar{x}_1 = 27/6 = 4.5$ y $\bar{c} = 0.5$. Las desviaciones de x_1 son $-3.5, -2.5, -1.5, 1.5, 2.5, 3.5$ y las de c son -0.5 tres veces y 0.5 tres veces.

$$\begin{aligned}
\textstyle\sum (x_i - \bar{x})(c_i - \bar{c}) &= 0.5\,(3.5 + 2.5 + 1.5) + 0.5\,(1.5 + 2.5 + 3.5) = 7.5,\\
\textstyle\sum (x_i - \bar{x})^2 &= 2\,(3.5^2 + 2.5^2 + 1.5^2) = 41.5,\\
\textstyle\sum (c_i - \bar{c})^2 &= 6 \cdot 0.5^2 = 1.5,\\
r_1 &= \frac{7.5}{\sqrt{41.5 \cdot 1.5}} = \frac{7.5}{7.890} = 0.951.
\end{aligned}$$

Con la misma cuenta, $r_2 = 0.5/\sqrt{17.5 \cdot 1.5} = 0.098$. El filtro se queda con x_1.

Pearson sirve también para medir **redundancia**: dos atributos con $|r|$ cercano a 1 entre sí dicen lo mismo, y basta con uno. Los buenos subconjuntos tienen atributos muy correlacionados con la clase y poco correlacionados entre sí.

Ojito...

Pearson solo ve relaciones **lineales** y de **un atributo a la vez**. Un atributo que separa las clases de forma no lineal puede tener $r = 0$; el ejemplo de la información mutua lo muestra. Y dos atributos que solo sirven juntos tienen $r = 0$ cada uno; el ejemplo de RELIEF lo muestra.

Filtro: información mutua

Definición 16.6 (Información mutua)**.** Para un atributo discreto X y la clase C,

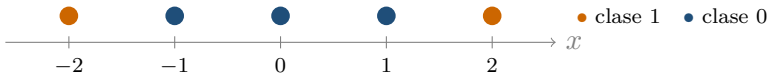
$$I(X;C) = \sum_x \sum_c p(x,c) \log_2 \frac{p(x,c)}{p(x)\,p(c)} = H(C) - H(C \mid X).$$

La segunda forma dice qué mide: cuánto baja la incertidumbre sobre la clase cuando se conoce el valor del atributo. Vale 0 si X y C son independientes, y $H(C)$ si X determina la clase por completo. Es exactamente la **ganancia de información** de la clase de árboles de decisión, ahora usada para ordenar atributos en lugar de escoger la pregunta de un nodo. Con el conjunto de jugar tenis el ranking ya está calculado:

	Cielo	Humedad	Viento	Temperatura
$I(X;\text{Juega})$	0.247	0.152	0.048	0.029

A diferencia de Pearson, la información mutua detecta cualquier tipo de dependencia, no solo la lineal.

Ejemplo 16.7 (Una relación que Pearson no ve)**.** Cinco ejemplos con $x \in \{-2, -1, 0, 1, 2\}$. La clase es 1 cuando $|x| = 2$, o sea en los extremos:



Con $\bar{x} = 0$ y $\bar{c} = 2/5$, las desviaciones de la clase son 0.6 en los extremos y -0.4 en el centro, y el numerador de Pearson es

$$(-2)(0.6) + (-1)(-0.4) + (0)(-0.4) + (1)(-0.4) + (2)(0.6) = 0.$$

Los términos se cancelan por simetría y $r = 0$. Pero cada valor de x tiene una sola clase, así que $H(C \mid X) = 0$ y

$$I(X;C) = H(C) = H\left(\tfrac{2}{5}, \tfrac{3}{5}\right) = 0.971 \text{ bits},$$

el máximo posible: conocer x dice la clase sin error.

Ojito...

Con atributos continuos, $p(x, c)$ no se cuenta directo: hay que discretizar en intervalos o usar un estimador de densidad, y el resultado depende de esa decisión. Y sigue siendo de un atributo a la vez: no ve interacciones ni redundancia. Variantes como mRMR (*máxima relevancia, mínima redundancia*) restan la información mutua entre los atributos ya elegidos.

Filtro: RELIEF

RELIEF (Kira y Rendell, 1992) califica los atributos mirando vecinos. Toma al azar un objeto de la muestra y busca sus dos vecinos más cercanos: el **amigo** (*nearest hit*), el más cercano de su misma clase, y el **enemigo** (*nearest miss*), el más cercano de otra clase. Un atributo bueno tiene valores parecidos entre el objeto y su amigo, y distintos entre el objeto y su enemigo.

Algoritmo 16.1 RELIEF

Entrada: muestra con N objetos y n atributos, número de iteraciones m

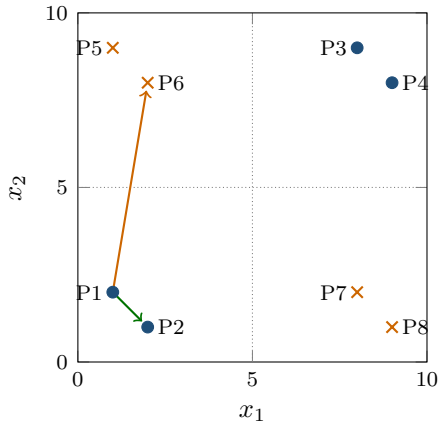
$W[A] \leftarrow 0$ para todo atributo A
para $i \leftarrow 1$ **hasta** m **hacer**
  $R \leftarrow$ un objeto tomado al azar
  $H \leftarrow$ su vecino más cercano de la misma clase
  $M \leftarrow$ su vecino más cercano de otra clase
  para todo atributo A **hacer**
    $W[A] \leftarrow W[A] - \text{dif}(A, R, H)/m + \text{dif}(A, R, M)/m$
  fin para
fin para
devolver W

Para atributos numéricos, $\text{dif}(A, R, P) = |R[A] - P[A]| \,/ (\text{máx}\, A - \text{mín}\, A)$, que queda entre 0 y 1; para nominales vale 0 si los valores son iguales y 1 si no. La distancia para encontrar los vecinos es la suma de las dif sobre todos los atributos. Al final se conservan los atributos con W mayor que un umbral τ.

Restar la diferencia con el amigo castiga a los atributos que cambian dentro de una misma clase; sumar la diferencia con el enemigo premia a los que cambian entre clases.

Ejemplo 16.8 (Dos atributos que solo sirven juntos)**.** Ocho objetos con tres atributos. La clase es el o exclusivo de "x_1 es grande" y "x_2 es grande", y x_3 es ruido:

	P1	P2	P3	P4	P5	P6	P7	P8
x_1	1	2	8	9	1	2	8	9
x_2	2	1	9	8	9	8	2	1
x_3	7	2	6	1	3	8	5	4
clase	0	0	0	0	1	1	1	1



Los círculos son la clase 0 y las cruces la clase 1; x_3 no se ve en la figura. Proyectados sobre un solo eje, los dos tipos de punto quedan revueltos: $x_1 = 1$ tiene un objeto de cada clase, y lo mismo cada valor. Pearson lo confirma: $r_1 = r_2 = 0$, mientras que el ruido x_3 sale con $r_3 = 0.218$. Un filtro por Pearson escogería el único atributo que no sirve.

Una iteración de RELIEF con R = P1. Los rangos son 8, 8 y 7. Las distancias de P1 a los demás:

Misma clase		
P2	$1/8 + 1/8 + 5/7$	$= 0.964$
P3	$7/8 + 7/8 + 1/7$	$= 1.893$
P4	$8/8 + 6/8 + 6/7$	$= 2.607$
Otra clase		
P5	$0/8 + 7/8 + 4/7$	$= 1.446$
P6	$1/8 + 6/8 + 1/7$	$= 1.018$
P7	$7/8 + 0/8 + 2/7$	$= 1.161$
P8	$8/8 + 1/8 + 3/7$	$= 1.554$

El amigo es P2 y el enemigo P6, las dos flechas de la figura. La actualización de cada atributo, antes de dividir entre m, es $-\text{dif}(R, H) + \text{dif}(R, M)$:

$$x_1 : -\tfrac{1}{8} + \tfrac{1}{8} = 0,$$
$$x_2 : -\tfrac{1}{8} + \tfrac{6}{8} = +0.625,$$
$$x_3 : -\tfrac{5}{7} + \tfrac{1}{7} = -0.571.$$

x_2 separa a P1 de su enemigo y no de su amigo: gana. x_3 hace lo contrario: pierde.

Todas las iteraciones. Tomando cada uno de los ocho objetos una vez, $m = 8$, las sumas son 1.125, 4.125 y -2.857, y

$$W = \left(\tfrac{1.125}{8},\ \tfrac{4.125}{8},\ \tfrac{-2.857}{8}\right) = (0.141,\ 0.516,\ -0.357).$$

	x_1	x_2	x_3
Pearson $\lvert r\rvert$	0	0	0.218
RELIEF W	0.141	0.516	-0.357

RELIEF descarta el ruido y conserva los dos atributos del XOR.

Idea clave

RELIEF ve interacciones porque busca a los vecinos con todos los atributos a la vez: el amigo de P1 es P2 porque se parecen en x_1 y en x_2 juntos. Pearson y la información mutua miran un atributo por separado y no pueden.

Ojito...

RELIEF no detecta redundancia: dos copias de x_2 recibirían las dos $W = 0.516$. Con pocos objetos el resultado depende de cuáles se sortean; aquí x_1 y x_2 juegan el mismo papel y aun así salieron con pesos distintos. ReliefF (Kononenko, 1994) promedia sobre k amigos y k enemigos, maneja más de dos clases y valores faltantes, y es la versión que se usa en la práctica.

Filtro: clasificador de un solo atributo

Se entrena un clasificador usando un atributo a la vez, se estima su exactitud con validación cruzada y se ordenan los atributos por esa exactitud. Mide la capacidad de discriminar en las mismas unidades que importan al final, pero sigue siendo de un atributo a la vez.

Ejemplo 16.9 (Una regla por atributo en jugar tenis)**.** El clasificador más simple con un solo atributo asigna a cada valor la clase mayoritaria de sus ejemplos; se le conoce como OneR. Con Cielo: soleado tiene $(2+, 3-)$ y predice no, nublado $(4+, 0-)$ predice sí, lluvioso $(3+, 2-)$ predice sí. Acierta $3 + 4 + 3 = 10$ de 14.

Atributo	Aciertos	Exactitud
Cielo	10/14	0.714
Humedad	10/14	0.714
Viento	9/14	0.643
Temperatura	9/14	0.643

Con 14 ejemplos se midió sobre el mismo entrenamiento; con datos reales se usa validación cruzada. Coincide con la información mutua en los dos primeros lugares, pero empata lo que ella separa: los rankings dependen de la medida.

En el ejemplo del XOR, un clasificador que solo ve x_1 acierta la mitad, igual que uno que solo ve x_2: este filtro también los descarta.

Cuenta como filtro aunque use un clasificador porque evalúa atributos sueltos y no subconjuntos, y el clasificador de un atributo no tiene que ser el que se va a usar al final.

Filtro: selección con árboles de decisión

Se entrena un árbol de decisión con todos los atributos y se conservan los que aparecen en algún nodo. Los que el árbol nunca usó no le sirvieron para separar las clases. Para ordenarlos se suma la reducción de impureza que logró cada atributo en todos sus nodos, pesada por la fracción de ejemplos que llega a cada nodo:

$$\text{importancia}(A) = \sum_{\text{nodos que prueban } A} \frac{|S_{\text{nodo}}|}{|S|} \,\text{Ganancia}(S_{\text{nodo}}, A).$$

Ejemplo 16.10 (El árbol de jugar tenis)**.** El árbol de la clase pasada usa Cielo en la raíz, Humedad

bajo soleado y Viento bajo lluvioso. Temperatura no aparece y se descarta. Las importancias:

$$\text{Cielo} : \tfrac{14}{14} \cdot 0.247 = 0.247,$$
$$\text{Humedad} : \tfrac{5}{14} \cdot 0.971 = 0.347,$$
$$\text{Viento} : \tfrac{5}{14} \cdot 0.971 = 0.347.$$

Suman $0.940 = H(S)$, toda la entropía inicial, porque las hojas quedaron puras. Normalizadas son 26 %, 37 % y 37 %: el atributo de la raíz no es el más importante. Cielo hace un corte parcial sobre los 14 días; Humedad y Viento terminan de separar sus ramas por completo.

A diferencia de los filtros anteriores, el árbol evalúa cada atributo en el contexto de las preguntas que ya se hicieron arriba, así que puede capturar algunas interacciones. Pero es voraz: en el XOR la raíz tiene ganancia 0 con cualquier atributo, el efecto horizonte de la clase pasada.

Ojito...

En otros textos esto no se llama filtro sino **método embebido** (*embedded*): la selección ocurre dentro del entrenamiento de un modelo. La importancia de un bosque aleatorio, promediada sobre cientos de árboles, es más estable que la de un árbol solo.

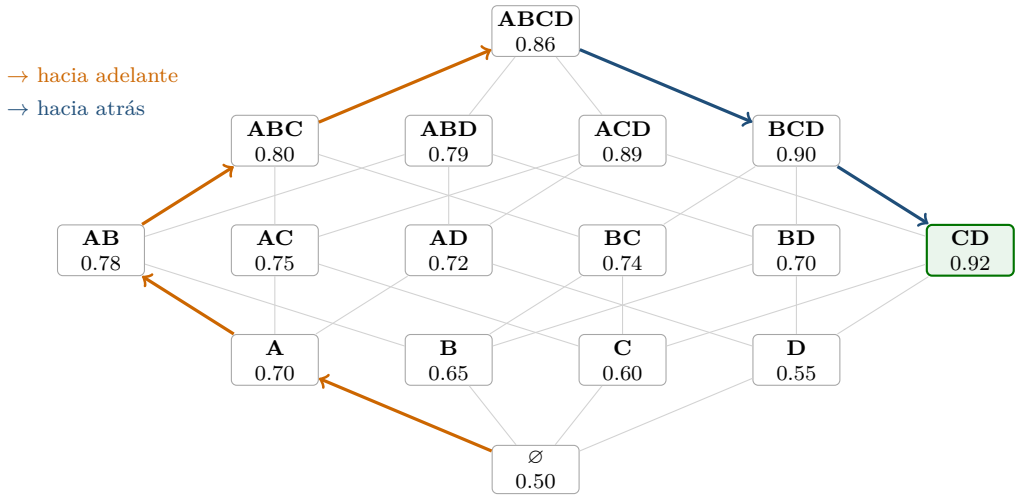
Wrappers: el espacio de búsqueda

Un wrapper evalúa subconjuntos, y con n atributos hay $2^n - 1$ subconjuntos no vacíos. Evaluar todos es la única forma de garantizar el mejor, y deja de ser posible muy pronto:

n	**Subconjuntos**	**A un segundo por evaluación**
10	1 023	17 minutos
20	1 048 575	12 días
30	1.07×10^9	34 años
100	1.27×10^{30}	mucho más que la edad del universo

Por eso los wrappers son problemas de combinatoria que se resuelven con heurísticas: recorren una parte pequeña del espacio buscando un subconjunto bueno, sin garantía de que sea el mejor.

Para comparar las heurísticas se usa un ejemplo con cuatro atributos, A, B, C y D. Cada subconjunto tiene una exactitud J, inventada para el ejemplo como si viniera de validación cruzada. C y D se comportan como el XOR de RELIEF: solos son débiles (0.60 y 0.55), juntos son el mejor subconjunto (0.92).



Cada nivel tiene los subconjuntos de un tamaño, y las líneas unen subconjuntos que difieren en un atributo. Las heurísticas se mueven por estas líneas.

Wrapper: selección hacia adelante

La **selección secuencial hacia adelante** (SFS, *sequential forward selection*) empieza con el conjunto vacío y en cada paso agrega el atributo que más sube J.

Algoritmo 16.2 Selección hacia adelante

Entrada: atributos X, tamaño deseado d

$S \leftarrow \varnothing$
mientras $|S| < d$ **hacer**
  $x^* \leftarrow \arg\max_{x \in X \setminus S} J(S \cup \{x\})$
  $S \leftarrow S \cup \{x^*\}$
fin mientras
devolver S

Ejemplo 16.11 (Hacia adelante en la tabla)**.** Los candidatos de cada paso y el que se queda:

Paso	Candidatos	Queda
1	A 0.70, B 0.65, C 0.60, D 0.55	A (0.70)
2	AB 0.78, AC 0.75, AD 0.72	AB (0.78)
3	ABC 0.80, ABD 0.79	ABC (0.80)
4	ABCD 0.86	ABCD (0.86)

El mejor que encuentra es ABCD con 0.86, usando los cuatro atributos. CD nunca se evaluó: en el paso 1 ganó A, y desde ahí todos los subconjuntos que se prueban contienen a A.

Hace $n + (n-1) + \cdots + 1 = n(n+1)/2$ evaluaciones: con $n = 100$ son 5 050 en lugar de 10^{30}. Es barato porque sus primeros pasos entrenan clasificadores con muy pocos atributos.

Ojito...

El defecto es el **efecto de anidamiento** (*nesting*): un atributo que se agregó ya no se puede quitar, aunque después deje de servir. Y como empieza con atributos sueltos, no ve interacciones: C y D solos son los peores, así que nunca entran primero.

Wrapper: selección hacia atrás

La **selección secuencial hacia atrás** (SBS, *sequential backward selection*) hace lo contrario: empieza con todos los atributos y en cada paso quita el que menos falta hace, el que al quitarlo deja la J más alta.

Algoritmo 16.3 Selección hacia atrás

Entrada: atributos X, tamaño deseado d

$S \leftarrow X$
mientras $|S| > d$ **hacer**
  $x^* \leftarrow \arg\max_{x \in S} J(S \setminus \{x\})$
  $S \leftarrow S \setminus \{x^*\}$
fin mientras
devolver S

Ejemplo 16.12 (Hacia atrás en la tabla)**.** Los candidatos de cada paso y el que se queda:

Paso	Candidatos	Queda
1	sin A: BCD 0.90, sin B: ACD 0.89, sin C: ABD 0.79, sin D: ABC 0.80	BCD (0.90)
2	sin B: CD 0.92, sin C: BD 0.70, sin D: BC 0.74	CD (0.92)
3	sin C: D 0.55, sin D: C 0.60	C (0.60)

Encuentra CD con 0.92, el mejor subconjunto de la tabla. Como empezó con todos los atributos, C y D estuvieron juntos desde el principio y nunca se separaron.

Idea clave

Hacia atrás ve interacciones que hacia adelante no ve, porque siempre evalúa subconjuntos grandes. El costo es ese mismo: sus primeros pasos entrenan clasificadores con casi todos los atributos, que es lo más caro. Y tiene el anidamiento al revés: un atributo que se quitó ya no regresa.

Wrapper: búsquedas secuenciales generalizadas

Las dos anteriores son casos extremos de una familia. Las más usadas:

- **Más l, menos r** (PTA(l, r), *plus-l take-away-r*, Stearns, 1976). Se alternan l pasos hacia adelante y r pasos hacia atrás. Si $l > r$ el subconjunto crece desde el vacío; si $l < r$, se encoge desde el completo. Quitar después de agregar deshace parte del anidamiento.
- **Generalizadas** (GSFS(g) y GSBS(g)). En cada paso se agrega o se quita el mejor grupo de g atributos, probando todas las combinaciones de g. Con $g = 2$ ya puede agregar C y D juntos. El costo crece como $\binom{n}{g}$ por paso.

Ejemplo 16.13 (PTA(2, 1) en la tabla)**.** Agrega dos, quita uno, y repite:

$$\varnothing \xrightarrow{+A} \text{A} \xrightarrow{+B} \text{AB} \xrightarrow{-B} \text{A} \xrightarrow{+B} \text{AB} \xrightarrow{+C} \text{ABC} \xrightarrow{-C} \text{AB} \xrightarrow{+C} \text{ABC} \xrightarrow{+D} \text{ABCD} \xrightarrow{-A} \text{BCD}.$$

Llega a BCD con 0.90: mejor que hacia adelante, pero no encuentra CD.

El problema de esta familia es que l, r y g se fijan antes de empezar, y ningún valor sirve bien en todos los problemas.

Wrapper: búsqueda flotante

La **búsqueda secuencial flotante hacia adelante** (SFFS, *sequential floating forward selection*; Pudil, Novovičová y Kittler, 1994) deja que los datos decidan cuántos pasos hacia atrás dar. Después de cada paso hacia adelante intenta quitar atributos, y quita mientras eso produzca un subconjunto mejor que el mejor que ya conocía de ese tamaño. El tamaño del subconjunto sube y baja: flota.

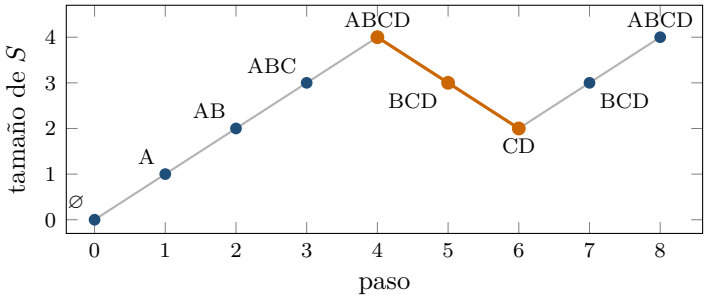
Algoritmo 16.4 Búsqueda flotante hacia adelante

Entrada: atributos X, tamaño máximo d

$S \leftarrow \varnothing$; mejor$[k] \leftarrow -\infty$ para todo k
mientras $|S| < d$ **hacer**
  agregar a S el atributo que más sube J *// paso hacia adelante*
  actualizar mejor$[|S|]$
  mientras $|S| > 2$ **hacer** *// pasos hacia atrás condicionales*
    $x^- \leftarrow$ el atributo de S cuya salida deja la J más alta
    si x^- es el recién agregado, o $J(S \setminus \{x^-\}) \leq$ mejor$[|S|-1]$ **entonces**
      salir del ciclo
    fin si
    $S \leftarrow S \setminus \{x^-\}$; actualizar mejor$[|S|]$
  fin mientras
fin mientras
devolver el subconjunto de mejor con la J más alta

Ejemplo 16.14 (Búsqueda flotante en la tabla)**.**

Paso	Movimiento	Queda	Por qué
1	+A	A 0.70	
2	+B	AB 0.78	
3	+C	ABC 0.80	quitar C es quitar el recién agregado
4	+D	ABCD 0.86	
5	−A	BCD 0.90	0.90 > mejor de tamaño 3 (0.80)
6	−B	CD 0.92	0.92 > mejor de tamaño 2 (0.78)
7	+B	BCD 0.90	quitar B es quitar el recién agregado
8	+A	ABCD 0.86	llegó a $d = 4$



En naranja, los dos pasos hacia atrás. Los mejores por tamaño quedan A (0.70), CD (0.92), BCD (0.90) y ABCD (0.86), y devuelve CD: el óptimo de la tabla, con la mitad de los atributos.

La versión simétrica, SFBS, empieza con todos los atributos y alterna quitar con agregar condicionalmente. Las búsquedas flotantes dan resultados cercanos a la búsqueda exhaustiva en muchos problemas, con un costo parecido al de las secuenciales simples.

Selección con algoritmos genéticos

Encontrar buenos subconjuntos de atributos es muy costoso, y las búsquedas secuenciales avanzan un atributo a la vez desde un solo punto de partida. Un **algoritmo genético** (Holland, 1975; aplicado a selección de atributos por Siedlecki y Sklansky, 1989) mantiene una **población** de muchos subconjuntos a la vez y los hace evolucionar: los mejores se reproducen, mezclan sus atributos y de vez en cuando

mutan. Con el paso de las generaciones la población se concentra en subconjuntos buenos.

Representación. Cada subconjunto es un **cromosoma**: una cadena de n bits, donde el bit i vale 1 si el atributo i está en el subconjunto.

A	B	C	D	
1	0	1	1	$= \{\text{A}, \text{C}, \text{D}\}$

Aptitud. Cada cromosoma se califica con la exactitud del clasificador entrenado con sus atributos, $J(S)$. Si además se quieren subconjuntos chicos, se resta un castigo por tamaño, $f(S) = J(S) - \lambda\,|S|\,/n$.

Algoritmo 16.5 Algoritmo genético para selección de atributos

Entrada: tamaño de población P, generaciones G, probabilidades de cruza p_c y de mutación p_m

```
generar P cromosomas al azar y calcular su aptitud
para g ← 1 hasta G hacer
    conservar los mejores cromosomas sin cambio          // elitismo
    mientras la nueva población tenga menos de P cromosomas hacer
        escoger dos padres según su aptitud              // selección
        con probabilidad p_c, cruzarlos para obtener dos hijos   // cruza
        invertir cada bit de los hijos con probabilidad p_m      // mutación
        agregar los hijos a la nueva población
    fin mientras
    calcular la aptitud de los nuevos cromosomas
fin para
devolver el cromosoma de mayor aptitud que se haya visto
```

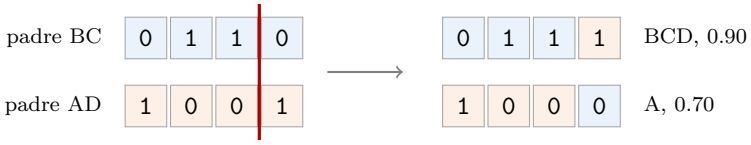
Selección. Decide quién se reproduce. En la **ruleta** cada cromosoma se escoge con probabilidad proporcional a su aptitud, $p_i = f_i / \sum_j f_j$. En el **torneo** se toman k cromosomas al azar y gana el de mayor aptitud; con $k = 2$ es el más común.

Ejemplo 16.15 (Una generación sobre la tabla)**.** Población inicial de cuatro cromosomas, con su aptitud tomada de la tabla:

Cromosoma	Subconjunto	Aptitud	Ruleta
1100	AB	0.78	$0.78/2.94 = 0.265$
0110	BC	0.74	0.252
1001	AD	0.72	0.245
0101	BD	0.70	0.238

Con ruleta las cuatro probabilidades son casi iguales, porque las aptitudes son parecidas: la selección casi no favorece a los mejores. El torneo no depende de qué tan grandes son las diferencias, solo de quién gana. Se hacen dos torneos: BC contra BD gana BC, y AD contra BD gana AD. Esos son los padres.

Cruza de un punto. Se escoge un punto de corte al azar, aquí después del tercer bit, y los hijos intercambian las colas:



El primer hijo junta B y C de un padre con D del otro, y vale más que los dos padres.

Mutación. Cada bit se invierte con probabilidad p_m. Si le toca al bit de B del primer hijo:



En una generación apareció CD, el subconjunto que la búsqueda hacia adelante nunca evaluó. Ninguno de los padres lo tenía; salió de combinar partes de dos padres y de un cambio al azar.

La cruza aprovecha lo que ya funciona, combinando trozos de cromosomas buenos. La mutación explora: mete atributos que ningún cromosoma de la población tiene, o quita alguno que todos comparten. Sin mutación la población puede perder un atributo para siempre.

Parámetros. Los valores son puntos de partida usuales, no reglas; se ajustan para cada problema.

Parámetro	**Valor usual**	**Efecto**
Tamaño de población P	20 a 100	chica converge rápido a un subconjunto mediocre; grande explora más y cuesta más evaluaciones
Generaciones G	50 a 500, o hasta que la mejor aptitud deje de subir	cada generación cuesta P entrenamientos
Probabilidad de cruza p_c	0.6 a 0.9	baja deja pasar a los padres casi sin cambio
Probabilidad de mutación p_m	$1/n$ por bit	alta vuelve la búsqueda aleatoria; cero la estanca
Selección	torneo con $k = 2$ o 3	k mayor presiona más hacia los mejores y pierde diversidad
Elitismo	los 1 o 2 mejores	garantiza que el mejor encontrado no se pierda
Castigo λ	según el problema	mayor prefiere subconjuntos más chicos

Ojito...

Un algoritmo genético no garantiza el óptimo: encuentra soluciones buenas, a menudo cercanas al óptimo, en espacios donde la búsqueda exhaustiva es imposible. El costo se mide en entrenamientos: con $P = 50$ y $G = 100$ son 5 000, parecido a la búsqueda hacia adelante con $n = 100$. Y como hace muchas evaluaciones sobre los mismos datos, el riesgo de sobreajustar la selección es mayor; el subconjunto final se valida con datos que la búsqueda no usó.

Resumen

Método	**Tipo**	**Ve interacciones**	**Comentario**
Pearson	filtro	no	solo relaciones lineales
Información mutua	filtro	no	cualquier dependencia; es la ganancia de información
RELIEF	filtro	sí	vecinos con todos los atributos; no ve redundancia
Clasificador de un atributo	filtro	no	ordena por exactitud de cada atributo solo
Árbol de decisión	filtro (o embebido)	en parte	se queda con los atributos que usa el árbol
Hacia adelante	wrapper	no	barato; anidamiento
Hacia atrás	wrapper	sí	caro al principio; anidamiento
PTA y generalizadas	wrapper	en parte	l, r o g fijos de antemano
Flotante	wrapper	sí	los pasos hacia atrás los decide J
Algoritmo genético	wrapper	sí	población de subconjuntos; sin garantía de óptimo

17. Clasificadores basados en reglas

3 de noviembre de 2026

Resumen rápido. Un clasificador basado en reglas guarda lo que aprendió como una lista de reglas *si-entonces*, que una persona puede leer y revisar. Cada regla se mide por su cobertura y su precisión. Para clasificar hay que decidir qué hacer cuando se disparan varias reglas o ninguna: listas ordenadas, votación y regla por defecto. Las reglas se obtienen de dos formas: leyéndolas de un árbol de decisión, o aprendiéndolas directo con covering secuencial, que aprende una regla, quita lo que ya cubre y repite. Con jugar tenis, dos reglas y una regla por defecto clasifican los 14 días.

Qué es una regla

Definición 17.1 (Regla de clasificación)**.** Una expresión de la forma

si antecedente **entonces** clase,

donde el antecedente es una conjunción de condiciones sobre los atributos, como Cielo = soleado o Temperatura ≤ 22, y el consecuente es una clase.

Se dice que la regla **cubre** a un ejemplo, o que **se dispara**, cuando el ejemplo cumple todas las condiciones del antecedente. Una regla con antecedente vacío cubre a todos los ejemplos.

Ejemplo 17.2 (Reglas para jugar tenis)**.** Con los 14 días de la clase de árboles de decisión:

- **si** Cielo = nublado **entonces** sí.
- **si** Cielo = soleado **y** Humedad = alta **entonces** no.

D1 (soleado, calurosa, alta, débil) dispara la segunda, porque cumple sus dos condiciones; no dispara la primera.

Cobertura y precisión de una regla

Cada regla se evalúa con dos números. Sea D el conjunto de entrenamiento y r una regla que predice la clase y:

$$\text{cobertura}(r) = \frac{|\text{ejemplos que cubre } r|}{|D|}, \qquad \text{precisión}(r) = \frac{|\text{ejemplos que cubre } r \text{ y son de clase } y|}{|\text{ejemplos que cubre } r|}.$$

La cobertura dice a cuántos ejemplos se aplica; la precisión, qué tan seguido acierta cuando se aplica. Es la misma precisión de la clase de medidas de desempeño, calculada solo sobre lo que cubre la regla.

Ejemplo 17.3 (Cuatro reglas sobre los 14 días)**.**

Regla	Cubre	Cobertura	Precisión
si Cielo = nublado entonces sí	4	$4/14 = 0.29$	$4/4 = 1.00$
si Humedad = normal entonces sí	7	$7/14 = 0.50$	$6/7 = 0.86$
si Humedad = alta entonces no	7	$7/14 = 0.50$	$4/7 = 0.57$
si soleado y Humedad = alta entonces no	3	$3/14 = 0.21$	$3/3 = 1.00$

Agregar condiciones baja la cobertura y casi siempre sube la precisión: *Humedad alta* sola acierta el 57 %; con *y soleado* acierta todo, pero en 3 días en vez de 7. Aprender reglas es negociar entre los dos números.

Clasificar con un conjunto de reglas

Un clasificador es un conjunto de reglas. Para un ejemplo nuevo se revisa cuáles se disparan, y pueden pasar tres cosas: se dispara exactamente una, que es lo fácil; se disparan varias que no están de acuerdo; o no se dispara ninguna.

Dos propiedades evitan los casos difíciles:

- **Mutuamente excluyentes**: ningún ejemplo dispara dos reglas a la vez. Así no hay conflictos.
- **Exhaustivas**: todo ejemplo dispara al menos una. Así nadie se queda sin clase.

Cuando no se cumplen hacen falta dos mecanismos:

- **Para los conflictos**, se ordenan las reglas y gana la primera que se dispare. A un conjunto ordenado se le llama **lista de decisión**. La otra opción es no ordenarlas y dejar que voten las que se disparan, pesando cada voto por la precisión de la regla.
- **Para los no cubiertos**, se agrega al final una **regla por defecto**, con antecedente vacío, que predice la clase mayoritaria de los ejemplos que ninguna regla cubrió.

Ejemplo 17.4 (Un conflicto)**.** Con las reglas

- r_1: si Cielo = lluvioso y Viento = fuerte entonces no, con precisión 1.00 y 2 días cubiertos;
- r_2: si Humedad = normal entonces sí, con precisión 0.86 y 7 días,

el día nuevo (lluvioso, templada, normal, fuerte) dispara las dos. En una lista de decisión con r_1 primero, gana r_1 y se predice no. Votando por precisión también gana r_1, 1.00 contra 0.86. Con otro orden la respuesta cambiaría: en una lista de decisión el orden es parte del modelo.

Reglas a partir de un árbol

La forma más directa de obtener reglas es leerlas de un árbol de decisión: cada camino de la raíz a una hoja es una regla, con las condiciones del camino como antecedente y la clase de la hoja como consecuente.

Ejemplo 17.5 (El árbol de jugar tenis)**.** El árbol que construyó ID3 tiene cinco hojas, así que da cinco reglas:

1. si Cielo = soleado y Humedad = alta entonces no;
2. si Cielo = soleado y Humedad = normal entonces sí;
3. si Cielo = nublado entonces sí;
4. si Cielo = lluvioso y Viento = débil entonces sí;
5. si Cielo = lluvioso y Viento = fuerte entonces no.

Las reglas que salen de un árbol son mutuamente excluyentes y exhaustivas, porque cada ejemplo baja por un solo camino. No necesitan orden ni regla por defecto.

Después se pueden **simplificar**: se prueba quitar cada condición de cada regla y, si la precisión estimada no empeora, se quita. Es lo que hace C4.5, el sucesor de ID3. En este ejemplo ninguna condición sobra: quitar *Cielo = soleado* de la primera regla deja *si Humedad = alta entonces no*, que baja de 3/3 a 4/7.

Al simplificar, las reglas pueden dejar de ser excluyentes, y entonces vuelven a hacer falta el orden y la regla por defecto.

Covering secuencial

La otra forma es aprender las reglas directamente de los datos, sin construir un árbol. La estrategia se llama **covering** (de cubrir): aprender una regla que cubra muchos ejemplos de una clase y casi ninguno de las otras, quitar los ejemplos que ya cubre, y repetir con los que quedan.

Algoritmo 17.1 Covering-Secuencial(D, y)

```
R ← lista vacía
mientras en D queden ejemplos de la clase y hacer
    r ← Aprender-Una-Regla(D, y)
    agregar r al final de R
    quitar de D los ejemplos que cubre r
fin mientras
devolver R
```

Algoritmo 17.2 Aprender-Una-Regla(D, y)

```
antecedente ← vacío                                   // cubre todo D
mientras la regla cubra ejemplos de otra clase y queden atributos sin usar hacer
    de todas las condiciones atributo = valor con atributos sin usar,
     agregar la que dé mayor precisión sobre lo que sigue cubierto
    en empate, la que cubra más ejemplos de la clase y
fin mientras
devolver si antecedente entonces y
```

Cada regla se construye de lo general a lo específico: empieza cubriendo todo y cada condición la vuelve más estrecha y más pura. Esta versión, con la precisión como criterio, es el algoritmo **PRISM** (Cendrowska, 1987).

Ejemplo 17.6 (Covering para la clase "no" en jugar tenis)**.** Hay 5 días de la clase no: D1, D2, D6, D8 y D14.

Regla 1, primera condición. Para cada condición posible se cuenta, entre los días que cumple, cuántos son "no":

Condición	"No" / cubiertos	Precisión
Cielo = soleado	3/5	0.60
Humedad = alta	4/7	0.57
Viento = fuerte	3/6	0.50
Temperatura = calurosa	2/4	0.50
Cielo = lluvioso	2/5	0.40
	⋮	

Gana *Cielo = soleado*. La regla ahora cubre D1, D2, D8, D9 y D11: tres "no" y dos "sí". Todavía no es pura.

Regla 1, segunda condición. Solo entre esos cinco días:

Condición agregada	"No" / cubiertos	Precisión
Humedad = alta	3/3	1.00
Temperatura = calurosa	2/2	1.00
Viento = débil	2/3	0.67
Temperatura = templada	1/2	0.50

Hay empate en 1.00 y se rompe por cobertura: *Humedad = alta* cubre tres "no" y *calurosa* solo dos. La regla queda pura:

$$r_1 : \textbf{ si } \text{Cielo} = \text{soleado } \textbf{y } \text{Humedad} = \text{alta } \textbf{entonces } \text{no},$$

y cubre D1, D2 y D8. Se quitan del conjunto; quedan 11 días, con dos "no": D6 y D14.

Regla 2. Con los 11 días que quedan, las mejores condiciones empatan: *Cielo = lluvioso* y *Viento = fuerte* tienen las dos 2/5. Se toma cualquiera; con *lluvioso* la regla cubre D4, D5, D6, D10 y D14, y al agregar *Viento = fuerte* quedan D6 y D14, 2/2:

$$r_2 : \textbf{ si } \text{Cielo} = \text{lluvioso } \textbf{y } \text{Viento} = \text{fuerte } \textbf{entonces } \text{no}.$$

Empezando por *Viento = fuerte* se llega a la misma regla. Ya no quedan "no" y el algoritmo termina.

Una clase con reglas y la otra por defecto

Con las reglas de "no" y una regla por defecto para todo lo demás, el clasificador completo es una lista de decisión de tres reglas:

Orden	Regla
1	si Cielo = soleado y Humedad = alta entonces no
2	si Cielo = lluvioso y Viento = fuerte entonces no
3	en otro caso, sí

Clasifica bien los 14 días, igual que el árbol, con tres reglas en lugar de cinco hojas. Es la estrategia de **RIPPER** (Cohen, 1995), el algoritmo de reglas más usado: aprende reglas para la clase minoritaria, que suele ser la que interesa detectar, y deja la mayoritaria como regla por defecto.

Ejemplo 17.7 (Dos días nuevos)**. (soleado, fresca, alta, fuerte).** La regla 1 se dispara: soleado y humedad alta. Se predice no, y no se revisa nada más.

(lluvioso, templada, normal, débil). La 1 no, porque no está soleado; la 2 tampoco, porque el viento es débil. Llega a la regla por defecto: sí.

Observación. PRISM, tal cual, aprende reglas para *cada* clase por separado. Para "sí" en jugar tenis le salen cuatro reglas más: nublado; humedad normal y viento débil; templada y humedad normal; lluvioso y viento débil. Las dos últimas cubren un solo día cada una, D11 y D4. Con todas juntas el conjunto ya no es excluyente: el día del ejemplo del conflicto dispara una regla de cada clase.

Precisión contra cobertura

Elegir condiciones solo por precisión tiene un defecto: premia las reglas diminutas. Una condición que cubre un ejemplo y acierta tiene precisión $1/1 = 1$, y le gana a una que cubre 31 y acierta 30, con $30/31 = 0.97$. La segunda es mucho más confiable: la primera puede ser pura casualidad.

La corrección más simple es el **estimador de Laplace**, la misma idea que el suavizado de Naive Bayes. Con p aciertos de t cubiertos y k clases,

$$\text{Laplace}(r) = \frac{p+1}{t+k}.$$

Ejemplo 17.8. Con dos clases:

$$\frac{1+1}{1+2} = 0.67, \qquad \frac{30+1}{31+2} = 0.94.$$

Ahora gana la regla grande. Con pocos datos el estimador se acerca a $1/k$, adivinar; con muchos, a la precisión.

RIPPER usa otra medida, la ganancia de FOIL, que también favorece las condiciones que conservan muchos ejemplos positivos.

Ojito...

Las reglas sobreajustan igual que los árboles. Aprender hasta que cada regla sea pura produce reglas que cubren uno o dos ejemplos, como las de D11 y D4 de la observación anterior, que probablemente describen ruido y no un patrón. Las defensas son las mismas que la poda: exigir una cobertura mínima, dejar de agregar condiciones cuando ya no mejoran lo suficiente, y podar cada regla contra un conjunto de validación, que es lo que hace RIPPER.

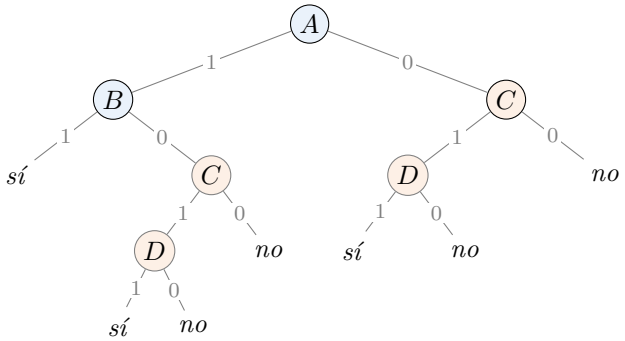
Reglas contra árboles

Toda regla de un árbol empieza con la condición de la raíz, aunque esa condición no tenga nada que ver con lo que la regla describe. Por eso hay conceptos que en reglas son cortos y en árboles son enormes.

Ejemplo 17.9 (El problema del subárbol replicado)**.** Con cuatro atributos binarios, el concepto

$$(A \wedge B) \vee (C \wedge D)$$

son dos reglas: **si** A **y** B **entonces** sí; **si** C **y** D **entonces** sí; en otro caso, no. El árbol más chico, empezando por A, necesita siete hojas:



El subárbol de C y D, en naranja, aparece dos veces: una para $A = 0$ y otra para $A = 1, B = 0$. El árbol tiene que repetir la regla de C y D en cada rama donde la de A y B no se cumplió. Con más términos en la disyunción, el árbol crece mucho más rápido que el conjunto de reglas.

	Árbol de decisión	**Reglas**
Se lee como	un camino desde la raíz	condiciones sueltas, cada regla por su cuenta
Traslapes	nunca: un ejemplo baja por un camino	posibles; hace falta orden o votación
Tamaño	puede repetir subárboles	suele ser más compacto
Modificar	cambiar un nodo afecta a todo lo de abajo	se puede agregar o quitar una regla sola
Aprendizaje	divide y vencerás, rápido	covering, más lento; aprende una regla a la vez

Idea clave

La gran ventaja de los dos es la misma: se pueden leer. Un médico puede revisar *si fiebre alta y tos seca entonces sospechoso* y decir si tiene sentido, cosa imposible con los pesos de una red neuronal. Por eso los clasificadores de reglas se siguen usando donde hay que explicar o auditar las decisiones, como en créditos y diagnóstico, aunque otros modelos clasifiquen un poco mejor.

Resumen

Concepto	**En una línea**
Regla	si conjunción de condiciones entonces clase
Cobertura	fracción de ejemplos a los que se aplica
Precisión	fracción de aciertos entre los que cubre
Lista de decisión	reglas ordenadas; gana la primera que se dispara
Regla por defecto	antecedente vacío al final; atrapa lo no cubierto
Reglas de un árbol	un camino, una regla; excluyentes y exhaustivas
Covering	aprender una regla, quitar lo que cubre, repetir
PRISM	covering con precisión, una lista por clase
RIPPER	covering para la minoritaria, poda con validación, mayoritaria por defecto
OneR	una sola condición por regla, sobre un solo atributo (clase del 22 de octubre)